Erkenntnis
https://doi.org/10.1007/s10670-019-00140-3

ORIGINAL RESEARCH

Cognitive and Computational Complexity: Considerations from Mathematical Problem Solving

Markus Pantsar[1]

Received: 3 January 2018 / Accepted: 17 June 2019
© The Author(s) 2019

Abstract

Following Marr's famous three-level distinction between explanations in cognitive science, it is often accepted that focus on modeling cognitive tasks should be on the computational level rather than the algorithmic level. When it comes to mathematical problem solving, this approach suggests that the complexity of the task of solving a problem can be characterized by the computational complexity of that problem. In this paper, I argue that human cognizers use heuristic and didactic tools and thus engage in cognitive processes that make their problem solving algorithms computationally suboptimal, in contrast with the optimal algorithms studied in the computational approach. Therefore, in order to accurately model the human cognitive tasks involved in mathematical problem solving, we need to expand our methodology to also include aspects relevant to the algorithmic level. This allows us to study algorithms that are cognitively optimal for human problem solvers. Since problem solving methods are not universal, I propose that they should be studied in the framework of enculturation, which can explain the expected cultural variance in the humanly optimal algorithms. While mathematical problem solving is used as the case study, the considerations in this paper concern modeling of cognitive tasks in general.

1 Introduction

This paper addresses a question of great theoretical interest in philosophy, psychology and cognitive science: how should we assess the complexity of cognitive tasks? Although the analysis carried out in this paper is applicable to cognitive tasks in general, here I am particularly interested in mathematical problem solving as a case study. In addition to the theoretical aspect, although not usually framed in these terms, this is also a practical question that mathematics educators face constantly. In

✉ Markus Pantsar
markus.pantsar@gmail.com

[1] Department of Philosophy, History and Art, University of Helsinki, Unioninkatu 40A, 00014 Helsinki, Finland

Published online: 02 July 2019

Erkenntnis
https://doi.org/10.1007/s10670-019-00140-3

PESQUISA ORIGINAL

Complexidade Cognitiva e Computacional: Considerações sobre a Resolução de Problemas Matemáticos

Markus Pantsar[1]

Recebido: 3 de janeiro de 2018 / Aceito: 17 de junho de 2019 © O(s) Autor(es) 2019

Resumo

Seguindo a famosa distinção de três níveis de Marr entre explicações na ciência cognitiva, é frequentemente aceite que o foco na modelação de tarefas cognitivas deve estar no nível computacional e não no nível algorítmico. Quando se trata de resolução de problemas matemáticos, esta abordagem sugere que a complexidade da tarefa de resolver um problema pode ser caracterizada pela complexidade computacional desse problema. Neste artigo, argumento que os cognitores humanos usam ferramentas heurísticas e didáticas e, portanto, se envolvem em processos cognitivos que tornam seus algoritmos de resolução de problemas computacionalmente abaixo do ideal, em contraste com os algoritmos ótimos estudados na abordagem computacional. Portanto, para modelar com precisão as tarefas cognitivas humanas envolvidas na resolução de problemas matemáticos, precisamos expandir nossa metodologia para incluir também aspectos relevantes ao nível algorítmico. Isso nos permite estudar algoritmos que são cognitivamente ideais para solucionadores de problemas humanos. Dado que os métodos de resolução de problemas não são universais, proponho que sejam est udados no âmbito da enculturação, o que pode explicar a variação cultural esperada nos algoritmos humanamente óptimos. Embora a resolução de problemas matemáticos seja usada como estudo de caso, as considerações neste artigo dizem respeito à modelagem de tarefas cognitivas em geral.

1 Introdução

Este artigo aborda uma questão de grande interesse teórico em filosofia, psicologia e ciência cognitiva: como devemos avaliar a complexidade das tarefas cognitivas? Embora a análise realizada neste artigo seja aplicável a tarefas cognitivas em geral, aqui estou particularmente interessado na resolução de problemas matemáticos como estudo de caso. Além do aspecto teórico, embora normalmente não enquadrado nestes termos, esta é também uma questão prática que os educadores matemáticos e nfrentam constantemente. Em

* Markus Pantsar
markus.pantsar@gmail.com

[1] Departamento de Filosofia, História e Arte, Universidade de Helsinque, Unioninkatu 40A, 00014 Helsinque, Finlândia

Published online: 02 July 2019

designing exams, for example, the teacher has to use some kind of (perhaps implicit) measure for evaluating problems in terms of the complexity of the cognitive task of solving them. The required cognitive tasks cannot be too complex, but they should be complex enough in order to assess the knowledge and skills relevant to the mathematical concepts.

So how can the complexity of the cognitive task of problem solving be assessed? In mathematics education, this is often a question tested in practice. Low performance by students in a particular problem is easy to interpret as showing high cognitive complexity of the task of solving it. However, from a theoretical perspective, this practical approach is unsatisfactory, as it does not necessarily distinguish between the cognitive processes involved. From the perspective of cognitive science, we are primarily interested in identifying those processes, after which we can assess their complexity.

One common method for this is modeling the cognitive tasks computationally, i.e., identifying a mathematical function that models the particular cognitive task. To assess the complexity of the task, we then assess the computational resources it takes for an algorithm to compute the function. When it comes to mathematical problem solving, this approach gives rise to a clear research paradigm. In theoretical computer science, the complexity of mathematical problems is studied in terms of their complexity, which is characterized by the resources (time or space) required for running algorithms that solve the problems. It is thus understandable to characterize the complexity of the cognitive task of solving a mathematical problem in terms of the complexity of that problem. In this paper, however, I will show that this approach is flawed and we need distinct concepts of cognitive complexity and computational complexity.

In particular, I will study the question of cognitive complexity from two directions. In the first part of the paper, I will study what we can achieve with the computational complexity approach to cognitive complexity. I will show that this is an important research paradigm because we can establish explicit complexity measures in it, which we can then use to discuss the possible characteristics of cognitive processes. However, I will also argue that this approach alone is not sufficient for studying cognitive complexity. In the second part of the paper, I will show that the reason for this is that it fails to take into consideration the plurality of processes used by human cognizers when solving a mathematical problem. The method of assessing the computational complexity of mathematical problems is closely related to the idea of *optimal* algorithms, i.e., algorithms requiring a minimum of computational resources for solving the problem. Here I will argue, however, that human cognizers use problem solving algorithms that may in fact be computationally suboptimal.[1] I will show, among other examples, that diagrams and other heuristic and didactic tools play an important role in mathematical problem solving, yet they add to the complexity of the problem solving algorithm. On these grounds, I will argue that we

[1] These algorithms can be either conscious or unconscious, as will be seen in the examples presented in Sect. 6.

ao conceber exames, por exemplo, o professor tem de utilizar algum tipo de medida (talvez implícita) para avaliar problemas em termos da complexidade da tarefa cognitiva de os resolver. As tarefas cognitivas necessárias não podem ser demasiado complexas, mas devem ser suficientemente complexas para avaliar os conhecimentos e competências relevantes para os conceitos matemáticos.

Então, como avaliar a complexidade da tarefa cognitiva de resolução de proble mas? Na educação matemática, esta é frequentemente uma questão testada na prática. O baixo desempenho dos alunos em determinado problema é fácil de interpretar como demonstrando alta complexidade cognitiva da tarefa de resolvê-lo. Contudo, do ponto de vista teórico, esta abordagem prática é insatisfatória, pois não distingue necessariamente entre os processos cognitivos envolvidos. Da perspectiva da ciência cognitiva, estamos principalmente interessados em identificar esses processos, após o que podemos avaliar a sua complexidade.

Um método comum para isso é modelar computacionalmente as tarefas cognitivas, ou seja, identificar uma função matemática que modele a tarefa cognitiva específica. Para avaliar a complexidade da tarefa, avaliamos então os recursos computacionais necessários para um algoritmo calcular a função. Quando se trata de resolução de problemas matemáticos, esta abordagem dá origem a um paradigma de pesquisa claro. Na ciência da computação teórica, a complexidade dos problemas matemáticos é estudada em termos de sua complexidade, que é caracterizada pelos recursos (tempo ou espaço) necessários para a execução de algoritmos que resolvem os problemas. É, portanto, compreensível caracterizar a complexidade da tarefa cognitiva de resolver um problema matemático em termos da complexidade desse problema. Neste artigo, contudo, mostrarei que esta abordagem é falha e que precisamos de conceitos distintos de complexidade cognitiva e complexidade computacional.

Em particular, estudarei a questão da complexidade cognitiva em duas direções. Na primeira parte do artigo, estudarei o que podemos alcançar com a abordagem da complexidade computacional para a complexidade cognitiva. Mostrarei que este é um paradigma de pesquisa importante porque podemos estabelecer nele medidas de complexidade explícitas, que podemos então usar para discutir as possíveis características dos processos cognitivos. No entanto, também argumentarei que esta abordagem por si só não é suficiente para estudar a complexidade cognitiva. Na segunda parte do artigo, mostrarei que a razão para isso é que ele não leva em consideração a pluralidade de processos usados pelos conhecedores humanos ao resolver um problema matemático. O método de avaliação da complexidade computacional de problemas matemáticos está intimamente relacionado à ideia de algoritmos ótimos, ou seja, algoritmos que requerem um mínimo de recursos computacionais para resolver o problema. Aqui argumentarei, no entanto, que os conhecedores humanos usam algoritmos de resolução de problemas que podem, na verdade, ser computacionalmente abaixo do ideal.[1] Mostrarei, entre outros exemplos, que diagramas e outras ferramentas heurísticas e didáticas desempenham um papel importante na resolução de problemas matemáticos, mas aumentam a complexidade do algoritmo de resolução de problemas. Por estas razões, argumentarei que

[1] Esses algoritmos podem ser conscientes ou inconscientes, como será visto nos exemplos apresentados na Seção. 6.

should distinguish between computationally optimal and what I will call *humanly* optimal algorithms.[2]

The first part of this paper (Sects. 2–5) studies the approach based on computational complexity. In the first section, I will present the modern computational-level paradigm for studying cognitive complexity, which will then be given a more detailed treatment in Sect. 3. In Sects. 4 and 5, I will present the possibilities and the limits of this paradigm in explaining cognitive phenomena. In the second part of the paper (Sects. 6, 7), I expand the methodology beyond the computational complexity approach. In Sect. 6, I aim to show how the human problem solving algorithms may be different from the optimal algorithms studied in the computational complexity approach. Finally, in Sect. 7, I discuss the notion of humanly optimal algorithm, concluding that rather than looking for a uniform notion of mathematical competence, we should expect both cross-cultural and intra-cultural variation in the problem solving methods.

While my focus is on mathematical problem solving, the approach applies also to other domains of cognitive modeling. This paper targets mathematical cognition partly because it works as a case study of modeling cognitive tasks. However, an important reason for the focus on mathematical problem solving is that there is a well-established paradigm in place for characterizing mathematical problems in terms of their computational complexity. Finally, given the importance of mathematics in our culture, I believe that there is a great need to treat mathematical problem solving explicitly when it comes to modeling cognitive capacities. This should include philosophical questions concerning that research.

2 What is Cognitive Complexity?

Cognitive complexity as a scientific term does not have a fixed cross-disciplinary meaning. If we take the dictionary definition of “cognitive” as referring to conscious or conscious mental processes such as remembering, reasoning and knowing (among others), cognitive complexity can be understood as the measure of simplicity of such activity. However, both the relevant activity and its simplicity can be understood in different ways.

Let us start by establishing a general sense in which one cognitive process can be more complex than another. Think of two people reasoning about the weather. The sky is red during sunrise and John remembers the saying “Red sky at morning, sailors take warning”, thus concluding that he should not go boating. Mary, on the other hand, reasons that since the sun is in the eastern horizon, the red color must mean that there is clear sky in the east which allows sunlight to color the clouds in the west. Since winds in that particular region (e.g., the Atlantic) tend to be from west to east, Mary concludes that more clouds and perhaps storms are coming and thus it is

[2] We will later get a definition of algorithm, but for now it is enough to think of algorithms as step-by-step procedures, such as computer programs.

devemos distinguir entre algoritmos computacionalmente ótimos e o que chamarei de algoritmos humanamente ótimos.[2]

A primeira parte deste artigo (seções 2 a 5) estuda a abordagem baseada na complexidade computacional. Na primeira seção, apresentarei o paradigma moderno de nível computacional para o estudo da complexidade cognitiva, que receberá um tratamento mais detalhado na Seção. 3. Nas seitas. 4 e 5, apresentarei as possibilidades e os limites deste paradigma na explicação dos fenômenos cognitivos. Na segunda parte do artigo (seções 6 e 7), amplio a metodologia além da abordagem da complexidade computacional. Na Seita. 6, pretendo mostrar como os algoritmos humanos de resol ução de problemas podem ser diferentes dos algoritmos ótimos estudados na abordagem da complexidade computacional. Finalmente, na Seção. 7, discuto a noção de algoritmo humanamente ótimo, concluindo que, em vez de procurar uma noção uniforme de competência matemática, deveríamos esperar variação tanto intercultural quanto intracultural nos métodos de resolução de problemas.

Embora meu foco esteja na resolução de problemas matemáticos, a abordagem também se aplica a outros domínios da modelagem cognitiva. Este artigo tem como alvo a cognição matemática em parte porque funciona como um estudo de caso de modelagem de tarefas cognitivas. No entanto, uma razão importante para o foco na resolução de problemas matemáticos é que existe um paradigma bem estabelecido para caracterizar problemas matemáticos em termos de sua complexidade computacional. Finalmente, dada a importância da matemática na nossa cultura, acredito que há uma grande necessidade de tratar explicitamente a resolução de problemas matemáticos quando se trata de modelar capacidades cognitivas. Isto deve incluir questões fil osóficas relativas a essa pesquisa.

2 O que é Complexidade Cognitiva?

A complexidade cognitiva como termo científico não tem um significado i nterdisciplinar fixo. Se considerarmos a definição do dicionário de “cognitivo” como referindo-se a processos mentais conscientes ou conscientes, como lembrar, raciocinar e conhecer (entre outros), a complexidade cognitiva pode ser entendida como a medida da simplicidade de tal atividade. Contudo, tanto a atividade relevante como a sua simplicidade podem ser entendidas de diferentes maneiras.

Comecemos por estabelecer um sentido geral em que um processo cognitivo pode ser mais complexo que outro. Pense em duas pessoas raciocinando sobre o tempo. O cé u fica vermelho durante o nascer do sol e João lembra-se do ditado “Céu vermelho de manhã, marinheiros avisem”, concluindo assim que não deveria andar de barco. Maria, por outro lado, raciocina que, como o sol está no horizonte oriental, a cor vermelha deve significar que há céu claro no leste, o que permite que a luz solar colora as nuvens no oeste. Como os ventos naquela região específica (por exemplo, o Atlântico) tendem a ser de oeste para leste, Mary conclui que mais nuvens e talvez tempestades estão chegando e, portanto, é

[2] Mais tarde obteremos uma definição de algoritmo, mas por enquanto é suficiente pensar em algoritmos como procedimentos passo a passo, como programas de computador.

better to stay ashore. This way, both John and Mary make the same observation and end up with the same action, yet it is clear that Mary's reasoning is more complex.

Even though there is quite some variation in how cognitive complexity is understood in psychology, the account of John and Mary can be seen as representative of the traditional psychological understanding of the term. Importantly, cognitive complexity is seen as a property of the process of the individual cognizing subject (Bieri 1955; van Hiel and Mervielde 2003). But of course this approach to cognitive complexity is not limited to determining individual differences in cognitive processing. We can also ask more general questions about cognitive tasks. Even with individual differences, it is clear that human beings have great inter-individual similarity in their cognitive processes, which makes it feasible to study the complexity of those *tasks*, rather than individual cognitive processes.

In the cognitive task of mathematical problem solving, we can see the usefulness of both approaches clearly. Let us think of a simple mathematical problem, such as finding a solution to the equation $n^2 = 2$ when $n \in \mathbb{N}$. Or equivalently, to frame it as a decision problem (i.e., yes/no-problem), $\exists n(n^2 = 2)$ when $n \in \mathbb{N}$. Let's call this proposition p. Clearly p is false, but the way two people establish this falsehood may be quite different, even if both were valid. This way, the cognitive task of John concluding that p is false can be less complex than Mary's cognitive task of establishing that p is false. But now it is natural to ask what would be the *least* complex process of establishing the truth-value of p. This approach is taken in theoretical computer science: complexity is understood to be a computational property of a *problem*, measuring the simplest possible way of correctly solving a decision problem.[3]

With this brief account, we have arrived at three different aspects of cognitive complexity: the complexity of individual cognitive processes, the complexity of general cognitive tasks, and the complexity of problems. There is an intuitive way in which the three aspects are connected: the more complex the problem, the more complex the cognitive tasks required, and thus the more complex the individual cognitive processes. Equally intuitively, the converse chain of implications from the individual cognitive processes to the problem does not necessarily hold. It is possible for a problem to be simple and yet for the individual human processing of it to be highly complex. This way, to study the complexity of mathematical problem solving, it would seem that we need to work on all three aspects.

In cognitive science, however, that has not been seen as a satisfactory state of affairs. Since cognitive scientists are interested in the general features of human cognition, they have understandably largely disregarded the kind of different cognitive

[3] In this paper, we will work in the paradigm of decision problems. In computer science, many other types of problems are also studied, such as counting, search, function and optimization problems (Goldreich 2008). The focus on decision problems is not meant to suggest that other types of problems are not relevant for computational complexity. Algorithms for solving optimization problems, for example, may well provide better models for some human cognitive tasks. There are two main reasons for focusing on decision problems. First, research on decision problems is the most developed field in complexity theory. Second, the discussion in cognitive science and its philosophy has focused mostly on decision problems when it comes to computational modeling. It should also be noted that these distinctions between problems are somewhat arbitrary, as many problems can be framed in different ways.

é melhor ficar em terra. Dessa forma, tanto João quanto Maria fazem a mesma observação e terminam com a mesma ação, mas fica claro que o raciocínio de Maria é mais complexo.

Embora exista alguma variação na forma como a complexidade cognitiva é entendida em psicologia, o relato de João e Maria pode ser visto como representativo da compreensão psicológica tradicional do termo. É importante ressaltar que a complexidade cognitiva é vista como uma propriedade do processo do sujeito cognitivo individual (Bieri 1955; van Hiel e Mervielde 2003). Mas é claro que esta abordagem da complexidade cognitiva não se limita a determinar diferenças individuais no processamento cognitivo. Também podemos fazer perguntas mais gerais sobre tarefas cognitivas. Mesmo com diferenças individuais, fica claro que os seres humanos apresentam grande semelhança interindividual em seus processos cognitivos, o que torna viável estudar a complexidade dessas tarefas, e não os processos cognitivos individuais.

Na tarefa cognitiva de resolução de problemas matemáticos, podemos ver claramente a utilidade de ambas as abordagens. Vamos pensar em um problema matemático simples, co mo encontrar uma solução para a equação $n^2 = 2$ quando $n \in \mathbb{N}$. Ou equivalentemente, para enquadrá-lo como um problema de decisão (ou seja, problema sim/não), $\exists n(n^2 = 2)$ quando $n \in \mathbb{N}$. Vamos chamar essa proposição de p. É evidente que p é falso, mas a forma como duas pessoas estabelecem esta falsidade pode ser bastante diferente, mesmo que ambas sejam válidas. Desta forma, a tarefa cognitiva de John de concluir que p é falso pode ser menos complexa do que a tarefa cognitiva de Mary de estabelecer que p é falso. Mas agora é natural perguntar qual seria o processo menos complexo de estabelecer o valor de verdade de p. Esta abordagem é adotada na ciência da computação teórica: a complexidade é entendida como uma propriedade computacional de um problema, medindo a maneira mais simples possível de resolver corretamente um problema de decisão.[3]

Com este breve relato, chegamos a três aspectos diferentes da complexidade cognitiva: a complexidade dos processos cognitivos individuais, a complexidade das tarefas cognitivas gerais e a complexidade dos problemas. Existe uma forma intuitiva de ligar os três aspectos: quanto mais complexo o problema, mais complexas são as tarefas cognitivas necessárias e, portanto, mais complexos são os processos cognitivos individuais. De forma igualmente intuitiva, a cadeia inversa de implicações dos processos cognitivos individuais para o problema não é necessariamente válida. É possível que um problema seja simples e, ainda assim, que o seu processamento humano individual seja altamente complexo. Dessa forma, para estudar a c omplexidade da resolução de problemas matemáticos, parece que precisamos trabalhar os três aspectos.

Na ciência cognitiva, contudo, isso não tem sido visto como uma situação satisfatória. Como os cientistas cognitivos estão interessados nas características gerais da cognição humana, é compreensível que tenham desconsiderado em grande parte o tipo de diferentes aspectos cognitivos.

[3] Neste artigo, trabalharemos no paradigma de problemas de decisão. Na ciência da computação, muitos outros tipos de problemas também são estudados, como problemas de contagem, busca, função e otimização (Goldreich 2008). O foco em problemas de decisão não pretende sugerir que outros tipos de problemas não sejam relevantes para a complexidade computacional. Algoritmos para resolver problemas de otimização, por exemplo, podem fornecer modelos melhores para algumas tarefas cognitivas humanas. Existem duas razões principais para focar em problemas de decisão. Primeiro, a pesquisa sobre problemas de decisão é o campo mais desenvolvido na teoria da complexidade. Em segundo lugar, a discussão na ciência cognitiva e na sua filosofia tem-se centrado principalmente em problemas de decisão quando se trata de modelação computacional. Deve-se também notar que estas distinções entre problemas são um tanto arbitrárias, uma vez que muitos problemas podem ser enquadrados de diferentes maneiras.

styles that make humans use needlessly complex cognitive processes. Chomsky (1965) presented a fundamental distinction in linguistics (in particular syntax) between *competence* and *performance*, which has later been widely transferred to the study of cognitive phenomena. In studying the nature of cognitive tasks, we are not primarily interested in the actual performance of the cognizers. Rather, we want to study an idealized version of his/her cognitive abilities, i.e., what we would expect from a fully competent user of that cognitive capacity.

This approach leaves us with two aspects: the complexity of cognitive tasks in general and the complexity of problems. Intriguingly, as we will see, in the subject of mathematical problem solving these two levels can come to be equated when focusing on the computational level of explanation in cognitive science. The complexity of decision problems is a widely researched topic in theoretical computer science and in recent times, it has also become influential in the discussion on modeling cognitive tasks in cognitive science (see, e.g., Frixione 2001; van Rooij 2008). In what is called the *computational level* of explanation, cognitive tasks are understood as employing cognitive capacities to transfer input states (e.g., perceptions) into output states (e.g., decisions) (Cummins 2000). In the widely-used distinction by Marr (1977, 1982), three levels of explanation of cognitive tasks are identified: the computational level, the algorithmic level, and the implementation level (Table 1). In the context of mathematical problems, the three levels can be understood, respectively, as the computational characterization of the problem, the algorithm used to solve the problem, and the neuronal action involved in solving it. Marr's distinction has become widely accepted as the basic framework for studying cognitive capacities (see, e.g., Newell 1982; Pylyshyn 1984; Horgan and Tienson 1996; Frixione 2001; van Rooij 2008).

Marr (1982) argued that for maximal progress in cognitive science, the focus should be on the computational level. His view (p. 27) was that by studying the computational level, we also get a better understanding of the algorithmic and implementational levels. According to the *Principle of Rationality,* as presented by Newell (1982), agents choose actions they know to lead to their goals. Following that, Anderson (1990) has argued that through the evolutionary process human competence in cognitive tasks has been optimized, thus explaining why studying the computational level will also provide explanations on the algorithmic and implementational levels. The evolutionary process and rationality are thought to ensure that human cognitive acts are generally optimized for the task and thus the focus can be on the computational level of explanation. As cognitive scientists have developed the computational modeling of cognitive tasks, the focus has indeed moved to the computational level more than to the algorithmic and implementation levels (see, e.g., Isaac et al. 2014; Szymanik 2016; for a recent example, see Piantadosi et al. 2016).[4]

[4] This is not to suggest that researchers working in cognitive modeling are ignorant of, or uninterested in, the algorithmic and implementational levels, only that computational-level explanations currently form the most important area in computational modeling of cognitive tasks.

estilos que fazem os humanos usar processos cognitivos desnecessariamente c omplexos. Chomsky (1965) apresentou uma distinção fundamental em linguística (em particular na sintaxe) entre competência e desempenho, que mais tarde foi a mplamente transferida para o estudo dos fenômenos cognitivos. Ao estudar a natureza das tarefas cognitivas, não estamos interessados principalmente no desempenho real dos cognitores. Em vez disso, queremos estudar uma versão idealizada das suas capacidades cognitivas, ou seja, o que esperaríamos de um utilizador totalmente competente dessa capacidade cognitiva.

Esta abordagem deixa-nos com dois aspectos: a complexidade das tarefas cognitivas em geral e a complexidade dos problemas. Curiosamente, como veremos, no tema da resolução de problemas matemáticos estes dois níveis podem vir a ser equiparados quando nos concentramos no nível computacional de explicação na ciência cognitiva. A complexidade dos problemas de decisão é um tópico amplamente pesquisado na ciê ncia da computação teórica e, nos últimos tempos, também se tornou influente na discussão sobre modelagem de tarefas cognitivas na ciência cognitiva (ver, por e xemplo, Frixione 2001; van Rooij 2008). No que é chamado de nível computacional de explicação, as tarefas cognitivas são entendidas como o emprego de capacidades cognitivas para transferir estados de entrada (por exemplo, percepções) para estados de saída (por exemplo, decisões) (Cummins 2000). Na distinção amplamente utilizada por Marr (1977, 1982), são identificados três níveis de explicação de tarefas cognitivas: o nível computacional, o nível algorítmico e o nível de implementação (Tabela 1). No contexto dos problemas matemáticos, os três níveis podem ser entendidos, respectivamente, como a caracterização computacional do problema, o algoritmo utilizado para resolver o problema e a ação neuronal envolvida na sua resolução. A distinção de Marr tornou-se amplamente aceita como a estrutura básica para o estudo das capacidades cognitivas (ver, por exemplo, Newell 1982; Pylyshyn 1984; Horgan e Tienson 1996; Frixione 2001; van Rooij 2008).

Marr (1982) argumentou que, para o progresso máximo na ciência cognitiva, o foco deveria estar no nível computacional. Sua visão (p. 27) era que ao estudar o nível computacional, também obtemos uma melhor compreensão dos níveis algorítmico e de implementação. De acordo com o Princípio da Racionalidade, apresentado por Newell (1982), os agentes escolhem ações que sabem que levarão aos seus objetivos. Depois disso, Anderson (1990) argumentou que através do processo evolutivo a competência humana em tarefas cognitivas foi otimizada, explicando assim por que o estudo do nível computacional também fornecerá explicações sobre os níveis algorítmico e de implementação. O processo evolutivo e a racionalidade são pensados para garantir que os atos cognitivos humanos sejam geralmente otimizados para a tarefa e, assim, o foco pode estar no nível computacional de explicação. À medida que os cientistas cognitivos desenvolveram a modelagem computacional de tarefas cognitivas, o foco de fato mudou mais para o nível computacional do que para os níveis algorítmico e de implementação (ver, por exemplo, Isaac et al. 2014; Szymanik 2016; para um exemplo recente, ver Piantadosi et al. 2016).[4]

[4] Isso não significa sugerir que os pesquisadores que trabalham em modelagem cognitiva ignoram ou não tê m interesse nos níveis algorítmicos e de implementação, apenas que as explicações em nível computacional constituem atualmente a área mais importante na modelagem computacional de tarefas cognitivas.

In this paper, I call the approach that combines computational-level explanations with results from computational complexity theory the *computational complexity approach* to cognitive complexity. In many ways, it is an understandable development. First, it is a practical fact that computational-level modeling is a field in which great progress has been made. We know much less about the actual algorithms used in cognitive tasks, let alone the neuronal activity that correlates with them. Second, the results from computational complexity are often easily applicable to the study of cognitive complexity. As we will see, there are classes of problems which have been shown to take prohibitive time to solve. When applied to cognitive complexity, such results immediately rule out many algorithms as explanations of cognitive tasks. Third, in focusing on the computational level, there is a natural line of development in terms of abstraction. When making the Chomskyan move from performance to competence, the purpose is to get rid of the inter-individual variations in order to establish the general nature of a cognitive ability. In applying the results from the mathematical study of computational complexity, we are taking one abstraction step further and talk about the computational properties of the mathematical functions we believe to model cognitive tasks. This way, if we model a cognitive task accurately, we can determine its complexity objectively. Perhaps the human performance on algorithmic and implementational levels is less than optimal, but the focus should be on modeling the cognitive task itself, not the full variety of its practical implementations.

We can thus see the computational level as representing competence whereas the algorithmic and implementational levels are associated with performance. This was also how Marr viewed the matter when presenting his three-level distinction:

> Chomsky's (1965) theory of transformational grammar is a true computational theory [...] It is concerned solely with specifying what the syntactic decomposition of an English sentence should be, and not at all with how that decomposition should be achieved. Chomsky himself was very clear about this - it is roughly his distinction between competence and performance (Marr 1982, p. 28)

When it comes to mathematical problem solving, the corresponding view means specifying what the solution to a problem should be, and not how that solution should be achieved. The advantages of this approach are immediately visible. Let us think of a simple mathematical decision problem, such as whether $27 + 38 = 65$ is true. People may apply different algorithms for solving it, but it would be quite

Table 1 Marr's three levels of explanation in cognitive science

Marr's level	Level of explanation	Characterization
1	Computation	What the cognitive task is functionally, i.e., in terms of its input and output
2	Algorithmic	How the task is performed, i.e., how the output is calculated for each input
3	Implementation	How the algorithm is implemented on the physical level

Neste artigo, chamo a abordagem que combina explicações em nível co
mputacional com resultados da teoria da complexidade computacional de abordagem da complexidade computacional para a complexidade cognitiva. Em muitos aspectos, é um desenvolvimento compreensível. Primeiro, é um facto prático que a modelação a nível computacional é um campo em que se registaram grandes progressos. Sabemos muito menos sobre os algoritmos reais usados em tarefas cognitivas, muito menos sobre a atividade neuronal que se correlaciona com eles. Em segundo lugar, os resultados da complexidade computacional são frequentemente facilmente aplicáveis ao estudo da complexidade cognitiva. Como veremos, há classes de problemas que c
omprovadamente levam um tempo proibitivo para serem resolvidos. Quando aplicados à complexidade cognitiva, tais resultados excluem imediatamente muitos algoritmos como explicações de tarefas cognitivas. Terceiro, ao focar no nível computacional, há uma linha natural de desenvolvimento em termos de abstração. Ao fazer o cho
mskyano passar do desempenho para a competência, o objetivo é livrar-se das variações interindividuais, a fim de estabelecer a natureza geral de uma habilidade cognitiva. Ao aplicar os resultados do estudo matemático da complexidade computacional, estamos dando um passo adiante na abstração e falando sobre as propriedades computacionais das funções matemáticas que acreditamos modelarem tarefas cognitivas. Dessa forma, se modelarmos uma tarefa cognitiva com precisão, poderemos determinar objetivamente sua complexidade. Talvez o desempenho humano nos níveis algorítmico e de implementação seja inferior ao ideal, mas o foco deveria estar na modelagem da tarefa cognitiva em si, e não na variedade completa de suas implementações práticas.

Podemos assim ver o nível computacional como representando a competência, enquanto os níveis algorítmico e de implementação estão associados ao desempenho. Foi também assim que Marr viu a questão ao apresentar a sua distinção de três níveis:

> A teoria da gramática transformacional de Chomsky (1965) é uma verdadeira teoria computacional [...] Ela se preocupa apenas em especificar como deveria ser a decomposição sintática de uma sentença em inglês, e de forma alguma em co
> mo essa decomposição deveria ser alcançada. O próprio Chomsky foi muito claro sobre isto - é aproximadamente a sua distinção entre competência e desempenho (Marr 1982, p. 28)

Quando se trata de resolução de problemas matemáticos, a visão correspondente significa especificar qual deveria ser a solução para um problema, e não como essa solução deveria ser alcançada. As vantagens desta abordagem são imediatamente visíveis. Vamos pensar em um problema de decisão matemática simples, como se $27 + 38 = 65$ é verdadeiro. As pessoas podem aplicar algoritmos diferentes para resolvê-lo, mas seria bastante

Tabela 1 Os três níveis de explicação de Marr na ciência cognitiva

Nível de Marr	Nível de explicação	Caracterização
1	Computação	O que a tarefa cognitiva é funcionalmente, isto é, em termos de sua entrada e saída
2	Algorítmico	Como a tarefa é executada, ou seja, como a saída é calculada para cada entrada
3	Implementação	Como o algoritmo é implementado no nível físico

strange to claim that all the algorithms are equally representative of human competence. From all the possible algorithms for solving the problem, there must be one (or perhaps more) that is optimal in that it requires the least amount of cognitive processing. As will be seen in the next section, this closely mirrors the computational complexity approach. In studying computational complexity, we are interested in finding optimal algorithms for solving a problem. If we accept the idea that cognitive abilities are (evolutionarily or otherwise) optimized for their tasks, it is understandable to equate the human competence in mathematical problem solving with a computationally optimal problem solving algorithm.

Indeed, in the cognitive domain of mathematical problem solving, I agree that this is how Marr's focus on the computational level should be interpreted. As explained by Frixione (2001), the computational-level approach allows us to focus on the characteristics of the functions that are proposed as models of particular cognitive tasks:

> The relation existing between a computational theory and the algorithmic level can be regarded as the relation between a function (in a mathematical sense) that is computable and a specific algorithm for calculating its values. The aim of a computational theory is to single out a function that models the cognitive phenomenon to be studied. Within the framework of a computational approach, such a function must be effectively computable. However, at the level of the computational theory, no assumption is made about the nature of the algorithms and their implementation. (Frixione 2001, p. 381)

This way, once the function is identified, we can focus on the properties of that function and leave aside considerations on algorithms and their implementation. But when we consider mathematical problems, we are already working within the paradigm of such functions. If we dismiss the algorithmic and implementational levels, we are left with explaining the task of taking a mathematical decision problem as the input and providing the correct answer (yes or no; true or false) as the output. When studying the *complexity* of this cognitive task, it is understandable to characterize it in terms of the computational complexity of the problem in question. This way, the number of computational steps required to reach a solution is seen as characterizing the complexity of the cognitive task. In the cognitive science literature, this is called *problem complexity* and it is often considered to be the central variable in the research on mathematical cognition (Ashcraft 1992, 1995; LeFevre et al. 2005).

Generally speaking, this approach is fruitful and reliable. To give an idea why that is the case, let us think of a simple example. It is easy to agree that the task of solving whether $27 + 38 = 65$ is true is cognitively less complex than $645 + 472 = 1117$, even without conducting any experiments. Since the latter sum involves more computational steps—as seen, for example, when calculating the sums with the standard method learned in school—it must be more complex. From the point of view of computational complexity theory, however, the two problems here are in fact instances of the same computational problem; namely, the addition of integers. In complexity theory, as will be seen in the next section, we can study differences in complexity between general mathematical problems. So, for example, we can know that the general problem of finding the best strategy for the game reversi (Othello)

estranho afirmar que todos os algoritmos são igualmente representativos da competência humana. De todos os algoritmos possíveis para resolver o problema, deve haver um (ou talvez mais) que seja ideal, pois requer a menor quantidade de processamento cognitivo. Como será visto na próxima seção, isso reflete de perto a abordagem da complexidade computacional. Ao estudar a complexidade computacional, estamos interessados em encontrar algoritmos ótimos para resolver u m problema. Se aceitarmos a ideia de que as capacidades cognitivas são (evolutivamente ou não) optimizadas para as suas tarefas, é compreensível equiparar a competência humana na resolução de problemas matemáticos a um algoritmo de resolução de problemas computacionalmente óptimo.

Na verdade, no domínio cognitivo da resolução de problemas matemáticos, concordo que é assim que o foco de Marr no nível computacional deve ser interpretado. Conforme explicado por Frixione (2001), a abordagem em nível computacional permite-nos focar nas características das funções que são propostas como modelos de tarefas cognitivas particulares:

> A relação existente entre uma teoria computacional e o nível algorítmico pode ser considerada como a relação entre uma função (no sentido matemático) que é computável e um algoritmo específico para calcular seus valores. O objetivo de uma teoria computacional é destacar uma função que modele o fenômeno cognitivo a ser estudado. Dentro da estrutura de uma abordagem computacional, tal função deve ser efetivamente computável. No entanto, ao nível da teoria computacional, nenhuma suposição é feita sobre a natureza dos algoritmos e sua implementação. (Frixione 2001, p. 381)

Desta forma, uma vez identificada a função, podemos focar nas propriedades dessa função e deixar de lado considerações sobre algoritmos e sua implementação. Mas quando consideramos problemas matemáticos, já estamos trabalhando dentro do paradigma de tais funções. Se descartarmos os níveis algorítmico e de implementação, resta-nos explicar a tarefa de tomar um problema de decisão matemática como entrada e fornecer a resposta correta (sim ou não; verdadeiro ou falso) como saída. Ao estudar a complexidade desta tarefa cognitiva, é compreensível caracterizá-la em termos da complexidade computacional do problema em questão. Dessa forma, o número de etapas computacionais necessárias para chegar a uma solução é visto como uma caracterização da complexidade da tarefa cognitiva. Na literatura das ciências cognitivas, isso é chamado de complexidade do problema e é frequentemente considerado a variável central na pesquisa sobre cognição matemática (Ashcraft 1992, 1995; LeFevre et al. 2005).

De modo geral, esta abordagem é frutífera e confiável. Para se ter uma ideia de por que isso acontece, pensemos em um exemplo simples. É fácil concordar que a tarefa de resolver se $27 + 38 = 65$ é verdadeiro é cognitivamente menos complexa do que $645 + 472 = 1117$, mesmo sem realizar quaisquer experiências. Dado que esta última soma envolve mais passos computacionais – como se vê, por exemplo, ao calcular as somas com o método padrão aprendido na escola – deve ser mais complexa. Do ponto de vista da teoria da complexidade computacional, entretanto, os dois problemas aqui são, na verdade, instâncias do mesmo problema computacional; ou seja, a adição de inteiros. Na teoria da complexidade, como veremos na próxima seção, podemos estudar diferenças de complexidade entre problemas matemáticos gerais. Assim, por exemplo, podemos saber que o problema geral de encontrar a melhor estratégia para o jogo reversi (Otelo)

is less complex than that of chess (Iwata and Kasai 1994; Fraenkel and Lichtenstein 1981).

Importantly, this result can be established without basing it on any experiments on reversi and chess players. Although there is obviously a great deal of details concerning the neuronal activity and the actual algorithms that are manifested in solving such problems, the computational-level explanations are informative without involving the other two levels. This way, focusing on the computational-level has a great practical advantage: whereas the algorithmic and implementational levels may be extremely difficult to study, research on computational complexity has an established and highly fruitful methodology when it comes to mathematical (as well as many other) problems.

However, this is not to suggest that the computational level is preferred merely due to pragmatic considerations. Marr (1982) and Anderson (1990) have stressed that there is greater explanatory power in understanding the computational characteristics of a particular cognitive task than in explaining the algorithmic and physical manifestations of the task. Indeed, Marr argued that the computational level of explanation is in fact crucial when we try to explain cognitive tasks on the algorithmic and implementational levels:

> an algorithm is likely to be understood more readily by understanding the nature of the problem being solved than by examining the mechanism (and the hardware) in which it is embodied. (Marr 1982, p. 27)

Seen in our present mathematical context, the implications of this approach are clear. If we want to examine the way human beings solve mathematical problems, we should focus on understanding the computational nature of those problems. This way, the computational complexity approach to studying mathematical problems can be explicitly formulated as the primary route to explaining human mathematical problem solving competence.

In the first part of this paper, I will evaluate the computational complexity approach to the cognitive complexity of mathematical problem solving. In the next two sections, we will see how useful it can be in limiting the space of possible functions that can work as characterizations of human cognitive capacities. In addition, much of the information we can gather from the research on computational complexity of mathematical problems can be used to assess the complexity of the cognitive tasks involved in solving them. Overall, there is nothing necessarily wrong with the general guideline that the more complex a problem is computationally, the more complex it is for humans to solve it.

However, there is no compelling argument why this should *always* be the case. The evolutionary argument for the optimization of cognitive tasks, for example, while perhaps appealing in many cases, seems to be a bad fit for mathematical problem solving. On an evolutionary scale, the emergence of mathematics appears to be too recent an event to allow for the optimization of problem solving strategies (Fabry 2019). While an accurate timing of the emergence of the first finger counting and other body part systems is impossible, the earliest known written systems of numerals are from around 3000–2000 BC (Ifrah 1998). Of course mathematical cognition may apply cognitive patterns that have been evolutionarily developed for

é menos complexo que o do xadrez (Iwata e Kasai 1994; Fraenkel e Lichtenstein 1981).

É importante ressaltar que este resultado pode ser estabelecido sem baseá-lo em quaisquer experimentos com reversi e jogadores de xadrez. Embora haja obviamente muitos detalhes sobre a atividade neuronal e os algoritmos reais que se manifestam na resolução de tais problemas, as explicações em nível computacional são informativas sem envolver os outros dois níveis. Dessa forma, focar no nível computacional tem uma grande vantagem prática: enquanto os níveis algorítmico e de implementação podem ser extremamente difíceis de estudar, a pesquisa em complexidade computacional tem uma metodologia estabelecida e altamente frutífera quando se trata de problemas matemáticos (assim como de muitos outros).

No entanto, isto não significa que o nível computacional seja preferido apenas devido a considerações pragmáticas. Marr (1982) e Anderson (1990) enfatizaram que há maior poder explicativo na compreensão das características computacionais de uma tarefa cognitiva específica do que na explicação das manifestações algorítmicas e físicas da tarefa. Na verdade, Marr argumentou que o nível computacional de explicação é de facto crucial quando tentamos explicar tarefas cognitivas nos níveis algorítmico e de implementação:

> é provável que um algoritmo seja entendido mais facilmente ao compreender a natureza do problema que está sendo resolvido do que ao examinar o mecanismo (e o hardware) no qual ele está incorporado. (março de 1982, p. 27)

Vistas em nosso contexto matemático atual, as implicações desta abordagem são claras. Se quisermos examinar a forma como os seres humanos resolvem problemas matemáticos, devemos nos concentrar na compreensão da natureza computacional desses problemas. Desta forma, a abordagem da complexidade computacional para estudar problemas matemáticos pode ser explicitamente formulada como a principal via para explicar a competência humana na resolução de problemas matemáticos.

Na primeira parte deste artigo, avaliarei a abordagem da complexidade computacional à complexidade cognitiva da resolução de problemas matemáticos. Nas próximas duas seções, veremos quão útil pode ser para limitar o espaço de possíveis funções que podem funcionar como caracterizações das capacidades cognitivas humanas. Além disso, muitas das informações que podemos recolher da investigação sobre a complexidade computacional de problemas matemáticos podem ser utilizadas para avaliar a complexidade das tarefas cognitivas envolvidas na sua resolução. No geral, não há nada necessariamente errado com a diretriz geral de que quanto mais complexo um problema for computacionalmente, mais complexo será para os hu manos resolvê-lo.

No entanto, não há nenhum argumento convincente de que isso deva ser sempre assim. O argumento evolucionista para a otimização de tarefas cognitivas, por exemplo, embora talvez atraente em muitos casos, parece não ser adequado para a resolução de problemas matemáticos. Numa escala evolutiva, o surgimento da matemática parece ser um evento demasiado recente para permitir a otimização de estratégias de resolução de problemas (Fabry 2019). Embora seja impossível determinar com precisão o momento do surgimento da contagem dos primeiros dedos e de outros sistemas de partes do corpo, os primeiros sistemas escritos de numerais conhecidos datam de cerca de 3.000 a 2.000 a.C. (Ifrah 1998). É claro que a cognição matemática pode aplicar padrões cognitivos que foram desenvolvidos evolutivamente para

other purposes, but with this relatively recent history it seems quite plausible that our mathematical problem solving ability can include aspects that are suboptimal. Indeed, in Sect. 6 I will argue that such suboptimal algorithms play an important role in actual human problem solving processes.

3 Computational Complexity

Before we consider the limitations of the computational complexity approach to explaining cognitive complexity of human mathematical problem solving, we should first establish what explanations based on computational complexity *can* achieve. Let us think of a simple cognitive task, such as finding the largest number out of a small set of numbers. For a set of two numbers, the task consists of taking the numbers a, b as the input and then giving as output the correct answer $f(x)$, where $x \in \{a, b\}$. Thus we have three components: the input (the domain), the output (the image) and the function modeling the cognitive phenomenon of finding the largest number. On the computational level, it is then a straight-forward matter to analyze the complexity of the task of finding the largest of two distinct numbers. We can move the pursuit to the mathematical study of computability and determine how much resources the task requires.

As a naive initial description, we can calculate the steps it takes to get the correct answer. Given the input a, b, we go through the process characterized by the following algorithm:

(1) Is $a < b$? If yes, move to (2). If no, move to (3)
(2) Output a.
(3) Output b.

Although actual human problem solvers may use a variety of algorithms for solving the problem, this three-line pseudo program appears to characterize the problem solving task. In standard mathematical notation, the computational model of the cognitive task is thus the function $f(a, b) = max(a, b)$.

For a set of three distinct numbers a, b, c we get the following algorithm:

(1) Is $a < b$? If yes, move to (3). If no, move to (2)
(2) Is $a < c$? If yes, move to (6). If no, move to (4)
(3) Is $b < c$? If yes, move to (6). If no, move to (5)
(4) Output a.
(5) Output b.
(6) Output c.

From these simple examples, it is already easy to see the potential fruitfulness of the computational complexity approach. Clearly the case of three numbers appears to be a cognitively more complex problem, as seen from the fact that the pseudo program for solving it consists of six lines, compared to the three lines of the two-number problem.

outros propósitos, mas com esta história relativamente recente parece bastante plausível que a nossa capacidade de resolução de problemas matemáticos possa incluir aspectos que são subóptimos. Na verdade, na seita. 6 Argumentarei que tais algoritmos subótimos desempenham um papel importante nos processos reais de resolução de problemas hu manos.

3 Complexidade Computacional

Antes de considerarmos as limitações da abordagem da complexidade computacional para explicar a complexidade cognitiva da resolução de problemas matemáticos humanos, devemos primeiro estabelecer o que as explicações baseadas na complexidade computacional podem alcançar. Pensemos numa tarefa cognitiva si mples, como encontrar o maior número num pequeno conjunto de números. Para um conjunto de dois números, a tarefa consiste em tomar os números a, b como entrada e então dar como saída a resposta correta f(x), onde x ∈ {a, b} . Assim temos três componentes: a entrada (o domínio), a saída (a imagem) e a função que modela o fenómeno cognitivo de encontrar o maior número. No nível computacional, é então uma questão simples analisar a complexidade da tarefa de encontrar o maior de dois números distintos. Podemos passar para o estudo matemático da computabilidade e determinar quantos recursos a tarefa requer.

Como uma descrição inicial ingênua, podemos calcular as etapas necessárias para obter a resposta correta. Dada a entrada a, b, passamos pelo processo caracterizado pelo seguinte algoritmo:

(1) É a < b ? Se sim, vá para (2). Se não, vá para (3) (2)
Saída a. (3) Saída b.

Embora os solucionadores de problemas humanos reais possam usar uma variedade de algoritmos para resolver o problema, este pseudoprograma de três linhas parece caracterizar a tarefa de resolução de problemas. Na notação matemática padrão, o modelo computacional da tarefa cognitiva é, portanto, a função f (a, b) = max(a, b).

Para um conjunto de três números distintos a, b, c, obtemos o seguinte algoritmo:

(1) É a < b ? Se sim, vá para (3). Se não, vá para (2) (2)
É a < c ? Se sim, vá para (6). Se não, vá para (4) (3) É b
< c ? Se sim, vá para (6). Se não, vá para (5) (4) Saída
a. (5) Saída b. (6) Saída c.

A partir destes exemplos simples, já é fácil ver a potencial fecundidade da abordagem da complexidade computacional. É evidente que o caso de três números parece ser um proble ma cognitivamente mais complexo, como pode ser visto pelo facto de o pseudoprograma para o resolver consistir em seis linhas, em comparação com as três linhas do problema de dois números.

We have not given any consideration to the algorithmic level, i.e., how human reasoners actually compute the output of a given input. Yet the way we found that characterizing the complexities of the two problems seems to be apt from the cognitive perspective, as well. As an important upshot, we can now study the question of complexity of the two cognitive tasks by studying the complexity of the two functions modeling it computationally, i.e., $f(a,b) = max(a,b)$ and $f(a,b,c) = max(a,b,c)$. Since an algorithm computing the latter function takes up more resources than the former, we can state with confidence that the cognitive task of solving the latter problem is more complex than the former. It is of course possible that for some individual, the latter problem is easier to solve than the former. But with the focus on competence, such cases are seen as anomalies that do not need to be taken into consideration when we are studying the general cognitive task. In essence, we have eliminated possible inter-individual variation from the explanations, thus moving from studying performance to studying competence, as Marr and Chomsky intended.

But just what do we mean when we talk about the complexity of mathematical problems? In the above case, we characterized it in terms of the length of the algorithm that solves the problem. However, clearly there are many algorithms of different lengths that could solve a particular problem. What we are interested in is not just any algorithm that gives the correct solution, but the one that demands the *least* resources. The most popular paradigm for determining this is studying complexity in terms of *Turing machines*. Turing (1936) presented the machine as a way to study computation theoretically. In a nutshell, a Turing machine takes as its input symbols on a tape, one at a time. The Turing machine is always in some inner state and based on the input symbol and the state, the machine is given instructions to read and write symbols on the tape, move the tape, and change to a new inner state (or remain in the same state). The set of these instructions is called an *algorithm*, giving us an explicit definition of the concept. Since the Turing machine is a theoretical construct, no limits are made to the size of the tape. Most importantly, according to the generally accepted *Church-Turing thesis*, if there is a mechanical procedure for solving a problem, then there is a Turing machine that can solve it (Turing 1936; Church 1936).

The Turing machine has become the established paradigm for studying algorithmic, mechanical procedures. In the study of complexity in theoretical computer science, researchers are not interested in the computing capacities of particular computers. Instead, they want to study the inherent complexities of different tasks free from the limitations of physical computers. Under this approach, the complexity of a mathematical problem can be characterized by the complexity of the *least* complex Turing machine (i.e., the algorithm run by a Turing machine that takes a minimum of resources) that solves the problem. Such an algorithm is called *optimal*.[5]

[5] Here it should be noted that for practical purposes, optimality may include considerations of various aspects, such as stability, verifiability, simplicity of coding, etc. The context of Turing machines here is a purely theoretical one in which such matters are ignored, important as they are in practice. It should also be noted that optimal algorithms are not unique. In theoretical computer science, an algorithm is called (asymptotically) optimal if it never performs more than a constant factor worse than the best possible algorithm. There can thus many (even an infinite number of) optimal algorithms. Finally, although the method of characterizing complexity of a problem in terms of an optimal algorithm for solving it is commonplace, we know from Blum's speedup theorem (Blum 1967) that it is not possible in all cases

Não levamos em consideração o nível algorítmico, isto é, como os raciocinadores humanos realmente calculam a saída de uma determinada entrada. No entanto, a f orma como descobrimos que caracterizar as complexidades dos dois problemas parece ser adequada também do ponto de vista cognitivo. Como um resultado importante, podemos agora estudar a questão da complexidade das duas tarefas cognitivas estudando a complexidade das duas funções que as modelam computacionalmente, ou seja, f (a, b) =max(a, b) e f (a, b, c) =max (a, b, c) . Uma vez que um algoritmo que computa a última função consome mais recursos do que o primeiro, podemos afirmar com confiança que a tarefa cognitiva de resolver o último problema é mais complexa do que o primeiro. É claro que é possível que, para algum indivíduo, o último problema seja mais fácil de resolver do que o primeiro. Mas com o foco na competência, tais casos são vistos como anomalias que não precisam ser levadas em consideração quando estudamos a tarefa cognitiva geral. Em essência, eliminamos possíveis variações interindividuais das explicações, passando assim do estudo do desempenho para o estudo da competência, como pretendiam Marr e Chomsky.

Mas o que queremos dizer quando falamos sobre a complexidade dos problemas matemáticos? No caso acima, caracterizamo-lo em termos do comprimento do algorit mo que resolve o problema. No entanto, é evidente que existem muitos algoritmos de comprimentos diferentes que poderiam resolver um problema específico. O que nos interessa não é qualquer algoritmo que forneça a solução correta, mas aquele que exija menos recursos. O paradigma mais popular para determinar isso é o estudo da complexidade em termos de máquinas de Turing. Turing (1936) apresentou a máquina como uma forma de estudar a computação teoricamente. Resumindo, uma máquina de Turing toma como entrada símbolos em uma fita, um de cada vez. A máquina de Turing está sempre em algum estado interno e com base no símbolo de entrada e no estado, a máquina recebe instruções para ler e escrever símbolos na fita, mover a fita e mudar para um novo estado interno (ou permanecer no mesmo estado). O conjunto dessas instruções é denominado algoritmo, o que nos dá uma definição explícita do conceito. Como a máquina de Turing é uma construção teórica, não há limites para o tamanho da fita. Mais importante ainda, de acordo com a tese geralmente aceite de Church-Turing, se existe um procedimento mecânico para resolver um problema, então existe uma máquina de Turing que o pode resolver (Turing 1936; Church 1936).

A máquina de Turing tornou-se o paradigma estabelecido para o estudo de procedimentos algorítmicos e mecânicos. No estudo da complexidade na ciência da computação teórica, os pesquisadores não estão interessados nas capacidades computacionais de computadores específicos. Em vez disso, querem estudar as complexidades inerentes de diferentes tarefas, livres das limitações dos computadores físicos. Sob esta abordagem, a complexidade de um problema matemático pode ser caracterizada pela complexidade da máquina de Turing menos complexa (ou seja, o algoritmo executado por uma máquina de Turing que consome um mínimo de rec ursos) que resolve o problema. Esse algoritmo é chamado de ótimo.[5]

[5] Aqui deve-se notar que, para fins práticos, a otimização pode incluir considerações de vários aspectos, como estabilidade, verificabilidade, simplicidade de codificação, etc. O contexto das máquinas de Turing aqui é puramente teórico, no qual tais questões são ignoradas, por mais importantes que sejam na prática. Deve-se notar também que algoritmos ótimos não são únicos. Na ciência da computação teórica, um algoritmo é chamado (assintoticamente) ótimo se ele nunca executa mais do que um fator constante pior do que o melhor algoritmo possível. Portanto, podem existir muitos (até mesmo um número infinito de) algoritmos ótimos. Finalmente, embora o método de caracterizar a complexidade de um problema em termos de um algoritmo ótimo para resolvê-lo seja comum, sabemos pelo teorema da aceleração de Blum (Blum 1967) que não é possível em todos os casos

The Turing machine is closely connected to the computational-level approach in cognitive science. It can be seen as providing a theoretical framework that connects the study of computational complexity and computational-level explanations in cognitive science. Marr's work on the computational level was influenced by Newell and Simon (1976, 1980, 1982), who argued that cognitive science should focus on functional explanations of what they called *physical symbol systems*, i.e., general classes of systems capable of manipulating symbols. Marr then developed this idea into the computational level of explanation. For Newell and Simon, the concept of physical symbol system is specified as an "instance of a universal [Turing] machine" (Newell and Simon 1976, p. 117). This way, the computational level of explanation has been from the very beginning in close relation to Turing machines and therefore also the study of computational complexity.

When it comes to Turing machines, there are two main ways of measuring the complexity of algorithms: the *time* and the *space* it takes to run an algorithm. Since the Turing machine is an abstract model, time and space are not measured in seconds or bits. Instead, they are measured as a function of the size of the input. This has proven to be a highly fruitful approach and as perhaps the most important result, we can divide computational problems into *complexity classes*.

Taking time as the measure, one of the most important complexity classes is called **P** and it is defined as the class of decision problems that can be solved by a deterministic Turing machine in polynomial time (Cobham 1964; Edmonds 1965). An algorithm (i.e., a Turing machine) is said to run for polynomial time if its running time has an upper bound of a polynomial function of the size of the input for the algorithm. This means that if the size of the input is n, the running time has an upper bound of some function n^k for some constant k.

Another important complexity class is called **EXP** (or **EXPTIME**) and it is the class of decision problems that are solvable by a deterministic Turing machine in *exponential* time. An algorithm runs for exponential time if its running time has a lower bound of some exponential function of the size of the input, i.e., for input size of n, the running time has a lower bound of some function $2^{p(n)}$ where $p(n)$ is some polynomial function of n.

P and **EXP** form an important pair of complexity classes for two reasons. First, according to the widely accepted Cobham's (or Cobham-Edmonds) thesis, **P** is generally seen as the class of problems that can be feasibly solved by a computer. Second, it has been proven that the complexity class **EXP** is strictly greater than **P**. Algorithms for solving problems in **P** are called *efficient*, or *tractable*. Algorithms for solving problems in **EXP** (that are not in **P**), on the other hand, are *inefficient* or *intractable.* As we will see, this distinction is very important when we consider the computational complexity of functions that model cognitive tasks. For computer science, the distinction is crucial. A simple example of an efficient algorithm

Footnote 5 (continued)
to define the computational complexity of functions in terms of optimal algorithms for solving them (I thank an anonymous reviewer for bringing up this point).

A máquina de Turing está intimamente ligada à abordagem de nível co mputacional na ciência cognitiva. Pode ser visto como um quadro teórico que liga o estudo da complexidade computacional e as explicações a nível computacional na ciência cognitiva. O trabalho de Marr no nível computacional foi influenciado por Newell e Simon (1976, 1980, 1982), que argumentaram que a ciência cognitiva deveria s e concentrar em explicações funcionais do que chamavam de sistemas de símbolos físicos, ou seja, classes gerais de sistemas capazes de manipular símbolos. Marr então desenvolveu essa ideia no nível computacional de explicação. Para Newell e Simon, o conceito de sistema de símbolos físicos é especificado como uma "instância de uma máquina [Turing] universal" (Newell e Simon 1976, p. 117). Desta forma, o nível de explicação computacional esteve desde o início em estreita relação com as máquinas de Turing e, portanto, também com o estudo da complexidade computacional.

Quando se trata de máquinas de Turing, existem duas maneiras principais de medir a complexidade dos algoritmos: o tempo e o espaço necessários para executar um algoritmo. Como a máquina de Turing é um modelo abstrato, o tempo e o espaço não são medidos em segundos ou bits. Em vez disso, eles são medidos em função do tamanho da entrada. Esta provou ser uma abordagem altamente frutífera e talvez como o resultado mais importante, podemos dividir os problemas computacionais em classes de complexidade.

Tomando o tempo como medida, uma das classes de complexidade mais importantes é chamada P e é definida como a classe de problemas de decisão que podem ser resolvidos por uma máquina de Turing determinística em tempo polinomial (Cobham 1964; Edmonds 1965). Diz-se que um algoritmo (isto é, uma máquina de Turing) é executado em tempo polinomial se seu tempo de execução tiver um limite superior de uma função polinomial do tamanho da entrada do algoritmo. Isso significa que se o tamanho da entrada for n, o tempo de execução terá um limite superior de alguma função n^k para alguma constante k.

Outra importante classe de complexidade é chamada EXP (ou EXPTIME) e é a classe de problemas de decisão que podem ser resolvidos por uma máquina de Turing determinística em tempo exponencial. Um algoritmo é executado em tempo exponencial se seu tempo de execução tiver um limite inferior de alguma função exponencial do tamanho da entrada, ou seja, para tamanho de entrada de n, o tempo de execução tiver um limite inferior de alguma função $2^{p(n)}$ onde p(n) é alguma função polinomial de n.

P e EXP formam um importante par de classes de complexidade por dois motivos. Primeiro, de acordo com a tese amplamente aceita de Cobham (ou Cobham-Edmonds), P é geralmente visto como a classe de problemas que podem ser resolvidos de forma viável por um computador. Segundo, foi provado que a classe de complexidade EXP é estritamente maior que P. Algoritmos para resolver problemas em P são chamados de eficientes ou tratáveis. Algoritmos para resolução de problemas em EXP (que não estão em P), por outro lado, são ineficientes ou intratáveis. Como veremos, esta distinção é muito importante quando consideramos a complexidade computacional das funções que modelam tarefas cognitivas. Para a ciência da computação, a distinção é crucial. Um exemplo simples de um algoritmo eficiente

Nota de rodapé 5 (continuação) para definir a complexidade computacional das funções em termos de algoritmos ideais para resolvê-las (agradeço a um revisor anônimo por abordar este ponto).

is multiplying integers, which takes roughly n^2 steps of computation for two n-digit integers with the standard algorithm that computers use. The operation of factoring integers into primes, on the other hand, is a good example of an inefficient algorithm. An n-digit integer takes about $2^{n^{1/3}}$computational steps (Pomerance 1996).

On first glance, this may look like a terribly useless characterization of efficient and inefficient algorithms. We can have an algorithm that runs for $n^{99999999999999999999}$ computational steps, which is sure to take longer than the $2^{n^{1/3}}$computational steps for prime factoring for any feasible input size n. Yet according to Cobham's thesis, it is the former algorithm that counts as efficient. There are also rates of growth that are neither polynomial nor exponential, such as $n^{\log n}$. So why do computer scientists use such a theoretically flawed distinction? The short answer is that in most cases it *works*. Based on empirical evidence of computation, polynomial time came to be a good characterization for what was considered efficient in practice and exponential time for what was considered in practice inefficient (Aaronson 2012).[6]

However, the distinction between **P** and **EXP** is not the only interesting difference in complexity classes. For our purposes, a particularly interesting case is the complexity class **NP**, which is defined as the class of decision problems that can be solved by a *non-deterministic* Turing machine in polynomial time.[7] It is easy to see that **P** is a subclass of **NP**, but it has proven to be difficult to show whether it is a *proper* subclass of it, i.e., whether **P = NP**. Although this is perhaps the most famous unsolved problem in theoretical computer science, it is a generally accepted conjecture that the class **NP** is strictly greater than **P**. This conjecture plays an important role because it is related to the important concept of a computationally *hard* problem. Hard (or **NP**-hard) problems are understood to be those problems that are at least as hard to solve as the hardest problems in **NP** (i.e., those that are not in **P**). A problem is said to be **NP**-complete if it is both **NP**-hard and in **NP**.

All this is important for us for two reasons. First, since the conjecture $\mathbf{P} \neq \mathbf{NP}$ is generally accepted, the complexity class **NP** is de facto the lowest complexity class of problems which are thought to be computationally intractable. Second, the class of **NP**-complete problems includes many famous mathematical problems, such as graph coloring and the traveling salesman problem (in its decision version). It also includes many familiar games, such as Rubik's cube (finding an optimal solution) and Sudoku (generalized). In short, many of the kind of problem solving tasks that human beings engage themselves in are **NP**-complete.

If it were the case that **P = NP**, we would know that all those problems would in fact be included in the complexity class **P**, and thus be solvable by efficient algorithms. There would still of course remain complexity classes strictly greater than **P** (for example, the above-mentioned **EXP**), but computational problems currently seen as computable only by inefficient algorithms would no longer be considered

[6] A well-known example of this practice is the use of cryptography in computers, which is based on the widely-shared, but unproven, conjecture by computer scientists that multiplying integers is efficient while factoring them is inefficient.

[7] Whereas a deterministic Turing machine is programmed to have at most one action for each situation, a non-deterministic Turing machine can choose the action from several options.

está multiplicando números inteiros, o que leva aproximadamente n^2 etapas de cálculo para dois números inteiros de n dígitos com o algoritmo padrão que os computadores usam. A operação de fatorar números inteiros em primos, por outro lado, é um bom exemplo de algoritmo ineficiente. Um número inteiro de n dígitos leva cerca de $2^{n^{1/3}}$etapas computacionais (Pomerance 1996).

À primeira vista, isso pode parecer uma caracterização terrivelmente inútil de algoritmos eficientes e ineficientes. Podemos ter um algoritmo que execute $n^{99999999999999999999}$ etapas computacionais, o que certamente levará mais tempo do que $2^{n^{1/3}}$ etapas computacionais para fatoração primária para qualquer tamanho de entrada viável n. No entanto, de acordo com a tese de Cobham, é o primeiro algoritmo que conta como eficiente. Existem também taxas de crescimento que não são polinomiais nem exponenciais, como $n^{\log n}$. Então, por que os cientistas da computação usam uma distinção tão teoricamente falha? A resposta curta é que na maioria dos casos funciona. Com base em evidências empíricas de computação, o tempo polinomial passou a ser uma boa caracterização para o que era considerado eficiente na prática e o tempo exponencial para o que era considerado ineficiente na prática (Aaronson 2012).[6]

Contudo, a distinção entre P e EXP não é a única diferença interessante nas classes de complexidade. Para nossos propósitos, um caso particularmente interessante é a classe de complexidade NP, que é definida como a classe de problemas de decisão que podem ser resolvidos por uma máquina de Turing não determinística em tempo polinomial.[7] É fácil ver que P é uma subclasse de NP, mas provou ser difícil mostrar se é uma subclasse adequada dela, ou seja, se P = NP. Embora este seja talvez o problema não resolvido mais famoso na ciência da computação teórica, é uma conjectura geralmente aceita que a classe NP é estritamente maior que P. Esta conjectura desempenha um papel importante porque está relacionada ao importante conceito de um problema computacionalmente difícil. Problemas difíceis (ou NP-difíceis) são entendidos como aqueles problemas que são pelo menos tão difíceis de resolver quanto os problemas mais difíceis em NP (ou seja, aqueles que não estão em P). Um problema é dito NP-completo se for NP-difícil e em NP.

Tudo isto é importante para nós por duas razões. Primeiro, uma vez que a conjectura $\mathbf{P} \neq \mathbf{NP}$ é geralmente aceita, a classe de complexidade NP é de fato a classe de problemas de menor complexidade que são considerados computacionalmente intratáveis. Em segundo lugar, a classe de problemas NP-completos inclui muitos problemas matemáticos famosos, como a coloração de grafos e o problema do caixeiro viajante (em sua versão de decisão). Também inclui muitos jogos familiares, como o cubo de Rubik (encontrar uma solução ideal) e o Sudoku (generalizado). Em suma, muitos dos tipos de tarefas de resolução de problemas em que os seres humanos se envolvem são NP-completos.

Se fosse o caso de P = NP, saberíamos que todos esses problemas seriam de fato incluídos na classe de complexidade P e, portanto, seriam solucionáveis por algoritmos eficientes. É claro que ainda permaneceriam classes de complexidade estritamente maiores que P (por exemplo, o EXP mencionado acima), mas problemas computacionais atualmente vistos como computáveis apenas por algoritmos ineficientes não seriam mais considerados

[6] Um exemplo bem conhecido dessa prática é o uso de criptografia em computadores, que se baseia na conjectura amplamente compartilhada, mas não comprovada, por cientistas da computação de que multiplicar números inteiros é eficiente, enquanto fatorá-los é ineficiente.[7] Enquanto uma máquina de Turing determinística é programada para ter no máximo uma ação para cada situação, uma máquina de Turing não determinística pode escolher a ação entre várias opções.

intractable. Here, however, we follow the consensus and accept the conjecture that **P** = **NP** is indeed false. Thus we get the following basic distinction: problems belonging to **P** are efficiently solvable, whereas problems that are **NP**-complete (or harder) are not. Although strictly speaking at many points in this paper we need to appeal to the conjecture that **P** ≠ **NP**, this will not be explicitly mentioned. As we will see, this conjecture that the complexity class **P** can be identified with solvable problems has had great influence in computational modeling of cognitive tasks.[8]

4 Tractable Cognition Thesis

In computer science, an algorithm refers generally to a finitely describable well-defined procedure which takes an input and after a finite number of steps produces some output and halts. From a modern perspective, the Turing machine gave a highly intuitive characterization of algorithms: essentially, they are the kind of procedures that can be run by digital computers. In this paper, we are interested in algorithms from two perspectives. First, in computational complexity theory, we are interested in optimal algorithms for solving a mathematical problem. Second, from the cognitive perspective, we are interested in algorithms that model human cognitive capacities. As was characterized in Sect. 2, in the computational complexity approach to modeling cognitive tasks those two approaches are merged.

In short, we want to study algorithms for computing functions that work as models of human cognitive capacities. We can define that A is an algorithm for computing a function $f : I \rightarrow O$ if and only if A is an algorithm, and for any input $i \in I$ it is the case that A produces output $A(i) \in O$ such that $A(i) = F(i)$. When we talk about mathematical *problems* in this paper, we are in fact interested in algorithms for solving problems. Thus, solving a problem P means to correctly determine the output of function f_p for all inputs in its domain. In this manner, problems and functions are treated here essentially as synonyms and considerations on the complexity of a problem should be understood as dealing with the computational complexity of a problem in terms of an optimal algorithm for solving it, i.e., computing the output of the function for all the inputs in the domain.

Now the question is: what kind of functions can feasibly model human cognitive capacities? In mathematical problem solving, the complexity classes give us a highly fruitful framework for studying this question. For example, to say that a problem P is **NP**-hard means that computing the solution for every input in the domain of P is not bounded by a polynomial function of the size of the input. This means that an algorithm A for computing the function $f_P : I \rightarrow O$ for all $i \in I$ takes nonpolynomial time. Because of this prohibitive amount of time that non-polynomial algorithms take to run, they are called computationally intractable. Consequently, f_p cannot feasibly model the problem solving capacity of solving P. This way, computationally

[8] For the current state of the art in the rapidly advancing field of complexity classes, the reader is referred to Scott Aaronson's excellent website "Complexity Zoo" (https://complexityzoo.uwaterloo.ca/Complexity_Zoo).

intratável. Aqui, entretanto, seguimos o consenso e aceitamos a conjectura de que P = NP é de fato falso. Assim obtemos a seguinte distinção básica: problemas pertencentes a P são eficientemente solucionáveis, enquanto problemas que são NP-completos (ou mais difíceis) não o são. Embora, estritamente falando, em muitos pontos deste artigo precisemos apelar para a conjectura de que **P** ≠ **NP** , isso não será mencionado explicitamente. Como veremos, esta conjectura de que a classe de complexidade P pode ser identificada com problemas solucionáveis teve grande influência na modelagem computacional de tarefas cognitivas.[8]

4 Tese de Cognição Tratável

Na ciência da computação, um algoritmo refere-se geralmente a um procedimento bem definido, finitamente descritível, que recebe uma entrada e, após um número finito de etapas, produz alguma saída e para. De uma perspectiva moderna, a máquina de Turing forneceu uma caracterização altamente intuitiva dos algoritmos: essencialmente, eles são o tipo de procedimentos que podem ser executados por computadores digitais. Neste artigo, estamos interessados em algoritmos sob duas perspectivas. Primeiro, na teoria da complexidade computacional, estamos interessados em algoritmos ótimos para resolver um problema matemático. Em segundo lugar, do ponto de vista cognitivo, estamos interessados em algoritmos que modelem as capacidades cognitivas humanas. Como foi caracterizado na Sec. 2, na abordagem da complexidade computacional para modelar tarefas cognitivas, essas duas abordagens são mescladas.

Em suma, queremos estudar algoritmos para funções computacionais que funcionem como modelos de capacidades cognitivas humanas. Podemos definir que A é um algoritmo para calcular uma função f : I → O se e somente se A for um algoritmo, e para qualquer entrada i ∈I é o caso de A produzir a saída A(i) ∈O tal que A(i) =F (i) . Quando falamos sobre problemas matemáticos neste artigo, estamos na verdade interessados em algoritmos para resolução de problemas. Assim, resolver um proble ma P significa determinar corretamente a saída da função f_p para todas as entradas em seu domínio. Desta forma, problemas e funções são tratados aqui essencialmente como sinônimos e considerações sobre a complexidade de um problema devem ser entendidas como lidando com a complexidade computacional de um problema em termos de um algoritmo ótimo para resolvê-lo, ou seja, computando a saída da função para todas as entradas no domínio.

Agora a questão é: que tipo de funções podem modelar de forma viável as capacidades cognitivas humanas? Na resolução de problemas matemáticos, as classes de complexidade nos fornecem uma estrutura altamente frutífera para o estudo desta questão. Por exemplo, dizer que um problema P é NP-difícil significa que calcular a solução para cada entrada no domínio de P não é limitado por uma função polinomial do tamanho da entrada. Isso significa que um algoritmo A para calcular a função f_P : I → O para todo i ∈I leva tempo não polinomial. Por causa dessa quantidade proibitiva de tempo que os algoritmos não polinomiais levam para ser executados, eles são chamados de computacionalmente intratáveis. Consequentemente, f_p não pode modelar de forma viável a capacidade de resolução de problemas de resolução de P. Desta forma, computacionalmente

[8] Para conhecer o estado da arte atual no campo de rápido avanço das classes de complexidade, o leitor deve consultar o excelente site "Complexity Zoo" de Scott Aaronson (https://compl exity zoo.uwate rloo.ca/ Compl exity _Zoo).

intractable algorithms in general are not considered to be feasible Marr's algorithmic-level explanations for cognitive processes (Frixione 2001).

It should be noted that there are some differing voices on this matter. NP-hard functions have been suggested as models at least for visual search (Tsotsos 1990) and analogical reasoning (Thagard 2000). In addition, some researchers have suggested that humans are able to solve the **NP**-hard Euclidean Traveling Salesperson Problem near-optimally in a short time (Graham et al. 2000; Dry et al. 2012). However, we should not confuse the actual use of **NP**-hard functions as models with them being *accurate* models by the strict criteria applied here. Neither should we believe that near-optimal performance in **NP**-hard tasks implies that the cognitive processing can only be modeled with **NP**-complete functions. It is conceivable that in many problems we can reach very good solutions with an approximate algorithm, but these should not be confused with proper exact solutions. In addition, the results presented in Ormerod and Chronicle (1999) suggest that human performance is often not even nearly optimal in the Euclidean Traveling Salesperson Problem.

This way, even though there may not be full consensus over the issue, it is generally accepted that computationally intractable algorithms cannot accurately model human cognitive tasks. This has become known as the *tractable cognition thesis* in the literature (van Rooij 2008; Isaac et al. 2014). According to the thesis, when we are looking for functions that model cognitive capacities, we should limit ourselves to those functions that can be computed by tractable algorithms. Standardly, based on Cobham's thesis, this has been understood as the *P-cognition thesis,* stating that we should limit our considerations to functions that compute problems in the complexity class **P** (Arora and Barak 2007).[9]

From this perspective, it may seem strange that many of the problems accepted to be **NP**-complete are actively studied and strategies for solutions are presented (Markman and Gentner 2000; Chater et al. 2006). How is this possible if the required algorithms are conjectured to be intractable? The answer can be found in *heuristic* procedures (Garey and Johnson 1979; Ausiello et al. 1999). The idea behind them is that while a heuristic procedure H is known to not compute a function f exactly, there is some close relation between the function f and the function f_H, which *is* computed exactly by H. This relation cannot be equality, so it must be the case that for some $i \in I$, $f(i) \neq f_H(i)$. Normally, the purpose of heuristic algorithms is to render **NP**-hard problems solvable in polynomial time (i.e., make them **P**-hard) by using an approximative function the relative error of which we can determine to be under some threshold "not too far from the optimum" (Papadimitriou 1994). Such "quick-and-dirty" algorithms are commonly used in many practical applications, in particular optimization tasks.

[9] It should be noted that there has been criticism against the P-cognition thesis as the appropriate formulation of the tractable cognition thesis. van Rooij (2008) argues that with suitable parameters concerning the size of input (see Downey and Fellows 1999 for more), also super-polynomial time computation can be feasible in modeling cognitive tasks. van Rooij calls this the "FPT-cognition thesis", for *fixed-parameter tractable*.

algoritmos intratáveis em geral não são considerados viáveis nas explicações de nível algorítmico de Marr para processos cognitivos (Frixione 2001).

Deve-se notar que existem algumas vozes divergentes sobre este assunto. Funções NP-difíceis foram sugeridas como modelos pelo menos para pesquisa visual (Tsotsos 1990) e raciocínio analógico (Thagard 2000). Além disso, alguns investigadores sugeriram que os seres humanos são capazes de resolver o problema euclidiano do caixeiro viajante quase NP-difícil num curto espaço de tempo (Graham et al. 2000; Dry et al. 2012). No entanto, não devemos confundir o uso real de funções NP-difíceis como modelos com elas sendo modelos precisos pelos critérios estritos aqui aplicados. Também não devemos acreditar que o desempenho quase ideal em tarefas NP-difíceis implica que o processamento cognitivo só pode ser modelado com funções NP-completas. É concebível que em muitos problemas possamos alcançar soluções muito boas com um algoritmo aproximado, mas estas não devem ser confundidas com soluções exatas adequadas. Além disso, os resultados apresentados em Ormerod e Chronicle (1999) sugerem que o desempenho humano muitas vezes não chega nem perto do ideal no Problema Euclidiano do Caixeiro Viajante.

Dessa forma, mesmo que não haja consenso total sobre a questão, é geralmente aceito que algoritmos computacionalmente intratáveis não podem modelar com precisão as tarefas cognitivas humanas. Isso se tornou conhecido como a tese da cognição tratável na literatura (van Rooij 2008; Isaac et al. 2014). De acordo com a tese, quando procuramos funções que modelem capacidades cognitivas, devemos nos li mitar àquelas funções que podem ser computadas por algoritmos tratáveis. Normalmente, com base na tese de Cobham, isso tem sido entendido como a tese da cognição P, afirmando que devemos limitar nossas considerações a funções que computam problemas na classe de complexidade P (Arora e Barak 2007).[9]

Desta perspectiva, pode parecer estranho que muitos dos problemas aceitos como NP-completos sejam ativamente estudados e estratégias para soluções sejam apresentadas (Markman e Gentner 2000; Chater et al. 2006). Como isso é possível se os algoritmos necessários são considerados intratáveis? A resposta pode ser encontrada em procedimentos heurísticos (Garey e Johnson 1979; Ausiello et al. 1999). A ideia por trás deles é que, embora se saiba que um procedimento heurístico H não calcula exatamente uma função f, existe alguma relação estreita entre a função f e a função f_H, que é calculada exatamente por H. Essa relação não pode ser de igualdade, então deve ser o caso de que para algum $i \in I$, $f(i) \neq f_H(i)$. Normalmente, o objetivo dos algoritmos heurísticos é tornar problemas NP-difíceis solucionáveis em tempo polinomial (ou seja, torná-los P-difíceis) usando uma função aproximativa cujo erro relativo podemos determinar estar abaixo de algum limite "não muito longe do ótimo" (Papadimitriou 1994). Esses algoritmos "rápidos e sujos" são comumente usados em muitas aplicações práticas, em tarefas específicas de otimização.

[9] Deve-se notar que tem havido críticas contra a tese da cognição P como a formulação apropriada da tese da cognição tratável. van Rooij (2008) argumenta que com parâmetros adequados relativos ao tamanho da entrada (ver Downey e Fellows 1999 para mais informações), também a computação de tempo superpolinomial pode ser viável na modelagem de tarefas cognitivas. van Rooij chama isso de "tese da cognição FPT", para parâmetro fixo tratável.

Do such heuristic algorithms offer us tools to go beyond tractable cognition? Although this may be seen as a feasible way to handle intractable algorithms, as van Rooij et al. (2012) have argued, there are fundamental problems involved in such claims if we consider the matter from a philosophical rather than a practical perspective. First, the introduction of H makes the computational and algorithm-level explanations inconsistent. Since for some $i \in I$ it must be the case that $f(i) \neq f_H(i)$, we can confirm f_H as the algorithm-level solution to a problem only if f is *dis*confirmed as the computational level solution to the problem, and vice versa. Second, H must fail as a cognitive explanation of solving f, because the cognizer with the procedure H does not compute the function f, but rather f_H, which is by definition another problem.

All this should of course not be confused with thinking that human cognizers do not *use* heuristic procedures, nor that they are irrelevant for the study of cognitive complexity. Instead, what van Rooij et al. (2012) argued was that if we appeal to heuristic procedures as ways out of intractable algorithms, we are subtly—or indeed often not so subtly—moving the goalposts. Van Rooij and her colleagues go through many proposed counter arguments (such as heuristic procedures providing partial explanations, optimism from many successful cases where $f(i) = f_H(i)$, and perhaps most strongly, approximations), but their argument seems to be left unharmed. Heuristic procedures do not give us a way to make intractable algorithms tractable, regardless of their great usefulness in achieving just that in the kind of approximative manner sufficient for many practical purposes.

From the considerations above, it becomes clear that heuristic algorithms do not allow us to drop the P-cognition thesis. We may discuss the details, but it seems that we should adopt some form of the tractable cognition thesis. Based on the physiological limitations of our brains (as well as our computing tools), algorithms above some complexity level cannot be feasibly implemented by us. Consequently, there must be some level of complexity after which problems are no longer computable for us. This is an extremely important point and the way we can analyze it in terms of complexity classes shows the great fruitfulness of the computational complexity approach to modeling cognitive tasks. A vast class of functions can be feasibly dismissed as potential models of human cognitive tasks due to their prohibitive complexity. Equally importantly, considering the topic of this paper, a vast class of mathematical problems can be deemed to be unfeasible for humans to solve efficiently.

Throughout these considerations, we should keep in mind the pragmatic character of the complexity classes. For small inputs, **NP** or **EXP**-hard problems can be perfectly solvable for human beings, whereas **P**-hard problems are unsolvable for large enough inputs. The complexity classes, as well as the tractable cognition thesis based on them, should be seen more as guidelines for evaluating complexity than strict results. Thus the P-cognition thesis should not be understood to claim that **NP**-hard functions, for example, cannot be used to model cognitive capacities. Rather, the thesis implies that for each such model, ultimately—perhaps for larger input—there must be a **P**-hard function that models the capacity more accurately.

But this pragmatic nature of the complexity classes notwithstanding, the acceptance of the tractable cognition thesis has very strong implications for philosophy of mathematics and beyond. Indeed, if mathematical problems beyond a certain

Esses algoritmos heurísticos nos oferecem ferramentas para ir além da cognição tratável? Embora isso possa ser visto como uma forma viável de lidar com algoritmos intratáveis, como van Rooij et al. (2012) argumentaram, existem problemas fundamentais envolvidos em tais afirmações se considerarmos a questão de uma perspectiva filosófica e não prática. Primeiro, a introdução de H torna inconsistentes as explicações computacionais e em nível de algoritmo. Como para algum $i \in I$ deve ser o caso de $f(i) \neq f_H(i)$, só podemos confirmar f_H como a solução em nível de algoritmo para um problema se f for desconfirmado como a solução em nível computacional para o problema, e vice-versa. Em segundo lugar, H deve falhar como explicação cognitiva da resolução de f, porque o conhecedor com o procedimento H não calcula a função f, mas sim f_H, que é por definição outro problema.

É claro que tudo isso não deve ser confundido com o pensamento de que os conhecedores humanos não usam procedimentos heurísticos, nem que eles são irrelevantes para o estudo da complexidade cognitiva. Em vez disso, o que van Rooij et al. (2012) argumentou foi que, se apelarmos a procedimentos heurísticos como formas de sair de algoritmos intratáveis, estaremos sutilmente – ou na verdade, muitas vezes não tão sutilmente – movendo as metas. Van Rooij e seus colegas analisam muitos contra-argumentos propostos (como procedimentos heurísticos que fornecem explicações parciais, otimismo de muitos casos bem-sucedidos onde $f(i) = H(i)$, e talvez mais fortemente, aproximações), mas seu argumento parece ter sido deixado ileso. Os procedimentos heurísticos não nos dão uma forma de tornar tratáveis algoritmos intratáveis, independentemente da sua grande utilidade para conseguir exatamente isso, de uma forma aproximativa suficiente para muitos propósitos práticos.

A partir das considerações acima, fica claro que os algoritmos heurísticos não nos permitem abandonar a tese da cognição-P. Podemos discutir os detalhes, mas parece que deveríamos adotar alguma forma de tese da cognição tratável. Com base nas limitações fisiológicas dos nossos cérebros (bem como nas nossas ferramentas computacionais), algoritmos acima de algum nível de complexidade não podem ser implementados por nós de forma viável. Conseqüentemente, deve haver algum nível de complexidade após o qual os problemas não serão mais computáveis para nós. Este é um ponto extremamente importante e a forma como podemos analisá-lo em termos de classes de complexidade mostra a grande fecundidade da abordagem da complexidade computacional para modelar tarefas cognitivas. Uma vasta classe de funções pode ser descartada como modelos potenciais de tarefas cognitivas humanas devido à sua complexidade proibitiva. Igualmente importante, considerando o tópico deste artigo, uma vasta classe de problemas matemáticos pode ser considerada inviável para humanos resolverem de forma eficiente.

Ao longo destas considerações, devemos ter em mente o caráter pragmático das classes de complexidade. Para entradas pequenas, problemas difíceis de NP ou EXP podem ser perfeitamente solucionáveis para seres humanos, enquanto problemas difíceis de P são insolúveis para entradas grandes o suficiente. As classes de complexidade, bem como a tese de cognição tratável baseada nelas, devem ser vistas mais como diretrizes para avaliar a complexidade do que como resultados estritos. Assim, a tese da cognição-P não deve ser entendida como afirmando que as funções NP-difíceis, por exemplo, não podem ser usadas para modelar capacidades cognitivas. Em vez disso, a tese implica que, em última análise, para cada um desses modelos - talvez para dados maiores - deve haver uma função P-hard que modele a capacidade com mais precisão.

Mas, apesar desta natureza pragmática das classes de complexidade, a aceitação da tese da cognição tratável tem implicações muito fortes para a filosofia da matemática e para além dela. Na verdade, se problemas matemáticos além de um certo

complexity are thought to be unsolvable, this is a strong conclusion generally for philosophy of mind and epistemology. It imposes explicit limits on what our cognitive capacities can achieve, as well on the class of problems whose truth-values we can know.

5 Complexity Within P and the Computational Paradigm

As fruitful as the computational approach to both cognitive modeling and mathematical problem solving has proven to be, it is clear that the limits it imposes are in practice rather weak. Even by accepting the P-cognition thesis we are left with an enormous class of functions that can potentially model human cognitive capacities, namely all functions computable in polynomial time. Finding the greatest common divisor (GCD) of two integers, for example, is known (in its decision form) to be in the complexity class **P**, yet a sequential solution for it can take a prohibitive amount of time. It could be the case that the GCD could be solved faster by parallel processing, yet there is no known way of parallelizing the computation effectively.[10] The algorithm for solving GCD may thus be seen as unfeasible as a model of human cognitive capacities.

Although the case of GCD is not clear, it reminds us of an important point: we should not treat all mathematical problems in the complexity class **P** as being the same in terms of complexity. Crucially, even if a problem were solvable in polynomial time, it could be in practice beyond the human capacity to solve it. All the P-cognition thesis gives us is an upper bound for human problem solving capacity. But within **P** there are problems with very different computational complexities. When studying the complexity of human mathematical problem solving ability, we should be interested in those differences. If it is possible to identify within **P** complexity measures that characterize mathematical problems in a more fine-grained manner, we would get important information also for identifying the cognitive tasks involved in the problem solving process.

There are many ways to expand our approach in order to be able to study complexity within **P**. We can, for example, stay within the prevalent paradigm of studying complexity in terms of the time or space needed to solve a problem. We simply need to look for complexity measures within **P**. In addition to the complexity class **NC** mentioned above, one interesting complexity class is **L**, the class of problems decidable in logarithmic memory. It is known that both **NC** and **L** are subsets of **P**, but it is not known whether **NC** = **P** or **L** = **P**. These problems may turn out to be as difficult as the problem **P** = **NP**, but it is conceivable that results from the study of

[10] More technically, although it is known that there are parallel algorithms faster than the standard sequential Euclidean algorithm (Chor and Goldreich 1990), it is not known whether GCD is in the complexity class **NC** ("Nick's Class"), which is the class of problems decidable in polylogarithmic time with a polynomial number of parallel processors. Furthermore, it is not known whether **NC** = **P** (Arora and Barak 2007).

complexidade são considerados insolúveis, esta é uma conclusão forte em geral para a filosofia da mente e a epistemologia. Impõe limites explícitos ao que as nossas capacidades cognitivas podem alcançar, bem como à classe de problemas cujos valores de verdade podemos conhecer.

5 Complexidade dentro de P e o paradigma computacional

Por mais frutuosa que tenha provado ser a abordagem computacional tanto para a modelação cognitiva como para a resolução de problemas matemáticos, é claro que os limites que ela impõe são, na prática, bastante fracos. Mesmo aceitando a tese da cognição P, ficamos com uma enorme classe de funções que podem potencialmente modelar as capacidades cognitivas humanas, nomeadamente todas as funções computáveis em tempo polinomial. Encontrar o máximo divisor comum (MDC) de dois inteiros, por exemplo, é conhecido (em sua forma de decisão) por estar na classe de complexidade P, mas uma solução sequencial para ele pode levar um tempo proibitivo. Poderia ser que o GCD pudesse ser resolvido mais rapidamente através do processamento paralelo, mas não existe nenhuma maneira conhecida de paralelizar a computação de forma eficaz.[10] O algoritmo para resolver o GCD pode, portanto, ser visto como inviável como um modelo de capacidades cognitivas humanas.

Embora o caso do GCD não seja claro, ele nos lembra um ponto importante: não devemos tratar todos os problemas matemáticos da classe de complexidade P como sendo iguais em termos de complexidade. Crucialmente, mesmo que um problema pudesse ser resolvido em tempo polinomial, na prática poderia estar além da capacidade humana de resolvê-lo. Tudo o que a tese da cognição-P nos dá é um limite superior para a capacidade humana de resolução de problemas. Mas dentro de P existem problemas com complexidades computacionais muito diferentes. Ao estudar a complexidade da capacidade humana de resolver problemas matemáticos, deveríamos estar interessados nessas diferenças. Se for possível identificar dentro de P medidas de complexidade que caracterizam problemas matemáticos de forma mais refinada, obteríamos informações importantes também para identificar as tarefas cognitivas envolvidas no processo de resolução de problemas.

Há muitas maneiras de expandir a nossa abordagem para podermos estudar a complexidade dentro de P. Podemos, por exemplo, permanecer dentro do paradigma predominante de estudar a complexidade em termos do tempo ou espaço necessário para resolver um problema. Precisamos simplesmente procurar medidas de complexidade dentro de P. Além da classe de complexidade NC mencionada acima, u ma classe de complexidade interessante é L, a classe de problemas decidíveis em memória logarítmica. Sabe-se que tanto NC quanto L são subconjuntos de P, mas não se sabe se NC = P ou L = P. Estes problemas podem revelar-se tão difíceis como o proble ma P = NP, mas é concebível que resulte do estudo de

10 Mais tecnicamente, embora se saiba que existem algoritmos paralelos mais rápidos que o algoritmo euclidiano sequencial padrão (Chor e Goldreich 1990), não se sabe se o GCD está na classe de complexidade NC ("Classe de Nick"), que é a classe de problemas decidíveis em tempo polilogarítmico com um número polinomial de processadores paralelos. Além disso, não se sabe se NC = P (Arora e Barak 2007).

complexity of classes provides us with more tools to distinguish between complexity classes within **P**.[11]

However, it could also turn out that in order to have maximally useful computational characterizations of mathematical problems for the study of cognitive complexity, we need to introduce more fine-grained measures of complexity than provided by complexity classes like **NC, L, P**, **NP** and **EXP**. This can be done by studying the functions that are upper bounds for the running time of an algorithm that solves a problem. Or in a pragmatic approach, we can measure the actual running times of algorithms on physical computers. Finally, we could also introduce another notion of complexity, such as Kolmogorov complexity, to work as the relevant unit of measure.[12] Alternatively, complexity for particular cases can also be measured in terms of computational steps, as in the example in the beginning of Sect. 3 of this paper. All these approaches have their difficulties, but a wider research paradigm with a plurality of measures of complexity can give us tools to introduce more informative distinctions between the complexity of mathematical problems.

What all such approaches have in common, however, is that they do not focus on the *human* aspect of mathematical problem solving. As we have seen, we can do a great deal of important work purely theoretically in the computational approach to cognitive modeling. But obviously there are limits to this: at some point we need to study the actual human cognitive capacities in order to find out which functions can model them. To assess the computational complexity of different cognitive tasks, we obviously first need to model those cognitive tasks as mathematical functions. The interesting thing about mathematical problem solving, however, is that we often seem to be able assess the complexity in a purely a priori manner. By studying the two simple problems presented in Sect. 3, for example, it is quite understandable to deem the three-number case as cognitively more complex than the two-number case, without ever conducting empirical research on how human reasoners in practice solve the problems.

It should be noted that while it may seem that cognitive complexity of mathematical problem solving can be studied independently of all empirical data, the matter may not be that simple. It is certainly true that the complexity of the cognitive task of solving a problem is often assessed based on the computational complexity of the problem, which can give the impression that the assessment is purely a priori. However, it seems feasible that this methodology would not be used if it were in conflict with empirical data. There are at least two ways in which the seemingly a priori

[11] There are complexity classes that are known to be strictly smaller than **P**, such as $\mathbf{AC^0}$, a class of circuit complexity. But these classes are generally very weak and thus of little use in distinguishing between mathematical problems. $\mathbf{AC^0}$, for example, does not even contain integer multiplication (Vollmer 1999).

[12] Kolmogorov complexity refers to the length of the shortest computer program that has an informative object, such as a string of symbols, as its output. For example, the string of symbols "aaaaaaaaaaaaaaaaaaaa" has a lower Kolmogorov complexity than the string of symbols "keehfydo38d-krislero29s". Both strings are 20 symbols long, but whereas the second string cannot be described with a shorter string, the former can. For instance, the English description "20 times a" is 10 symbols long. However, Kolmogorov complexity is not without problems. While it may seem like a straight-forward concept, it has turned out that determining the Kolmogorov complexity of even short strings of symbols is a highly difficult task (Soler-Toscano et al. 2014).

a complexidade das classes nos fornece mais ferramentas para distinguir entre classes de complexidade em P.[11]

No entanto, também poderia acontecer que, para termos caracterizações computacionais de problemas matemáticos extremamente úteis para o estudo da complexidade cognitiva, precisaríamos introduzir medidas de complexidade mais refinadas do que as fornecidas por classes de complexidade como NC, L, P, NP e EXP. I sto pode ser feito estudando as funções que são limites superiores para o tempo de execução de um algoritmo que resolve um problema. Ou, numa abordagem prag mática, podemos medir os tempos reais de execução de algoritmos em computadores físicos. Finalmente, poderíamos também introduzir outra noção de complexidade, co mo complexidade de Kolmogorov, para funcionar como a unidade de medida relevante. [12] Alternativamente, a complexidade para casos particulares também pode ser medida em termos de etapas computacionais, como no exemplo no início da Seção. 3 deste artigo. Todas estas abordagens têm as suas dificuldades, mas um paradigma de investigação mais amplo, com uma pluralidade de medidas de complexidade, pode dar-nos ferramentas para introduzir distinções mais informativas entre a co mplexidade dos problemas matemáticos.

O que todas essas abordagens têm em comum, contudo, é que não se concentram no aspecto humano da resolução de problemas matemáticos. Como vimos, podemos realizar muitos trabalhos importantes puramente teoricamente na abordagem computacional da modelagem cognitiva. Mas obviamente há limites para isto: em algum momento precisaremos estudar as capacidades cognitivas humanas reais para descobrir quais funções podem modelá-las. Para avaliar a complexidade co mputacional de diferentes tarefas cognitivas, obviamente precisamos primeiro modelar essas tarefas cognitivas como funções matemáticas. O interessante sobre a resolução de problemas matemáticos, contudo, é que muitas vezes parecemos ser capazes de avaliar a complexidade de uma maneira puramente a priori. Ao estudar os dois problemas simples apresentados na Seção. 3, por exemplo, é perfeitamente compreensível considerar o caso dos três números como cognitivamente mais co mplexo do que o caso dos dois números, sem nunca realizar investigação empírica sobre como os raciocinadores humanos resolvem os problemas na prática.

Deve-se notar que embora possa parecer que a complexidade cognitiva da resolução de problemas matemáticos pode ser estudada independentemente de todos os dados empíricos, a questão pode não ser tão simples. É certamente verdade que a complexidade da tarefa cognitiva de resolver um problema é muitas vezes avaliada co m base na complexidade computacional do problema, o que pode dar a impressão de que a avaliação é puramente a priori. Contudo, parece viável que esta metodologia não seja utilizada se estiver em conflito com os dados empíricos. Existem pelo menos duas maneiras pelas quais o aparentemente a priori

11 Existem classes de complexidade que são estritamente menores que P, como $\mathbf{AC^0}$, uma classe de complexidade de circuito. Mas estas classes são geralmente muito fracas e, portanto, de pouca utilidade na distinção entre problemas matemáticos. $\mathbf{AC^0}$, por exemplo, nem sequer contém multiplicação de inteiros (Vollmer 1999). 12 A complexidade de Kolmogorov refere-se ao comprimento do programa de computador mais curto que tem um objeto informativo, como uma sequência de símbolos, como saída. Por exemplo, a sequência de símbolos "aaaaaaaaaaaaaaaaaaaa" tem uma complexidade de Kolmogorov menor do que a sequência de símbolos "keehfydo38d-krislero29s". Ambas as strings têm 20 símbolos, mas enquanto a segunda string não pode ser descrita com uma string mais curta, a primeira pode. Por exemplo, a descrição em inglês " 20 vezes a" tem 10 símbolos. No entanto, a complexidade de Kolmogorov não está isenta de problemas. Embora possa parecer um conceito simples, descobriu-se que determinar a complexidade de Kolmogorov, mesmo de sequências curtas de símbolos, é uma tarefa altamente difícil (Soler-Toscano et al. 2014).

assessment of cognitive complexity can in fact be based on empirical results. First, all the data we have about cognitive capacities in general can be used to justify the assessment of the complexity of mathematical tasks. The limits of working memory, for example, impose limits on our problem solving capacity, as do physical limitations in using external tools. We do not need to empirically study each mathematical problem if we can establish that a certain class of problems puts more demands on working memory than another class. Second, the general method of using computational complexity to characterize the complexity of the problem solving tasks only works because in many cases we know the human competence to be modeled reasonably well by computationally optimal algorithms. Empirical data on the competence in the addition of (small) natural numbers, for example, implies that an optimal algorithm can be used to characterize the complexity of the human cognitive task (Fuson 1992).

This way, what may seem like a priori assessment of cognitive abilities may in fact be at least partly based on empirical considerations. Although the empirical data may not be explicitly acknowledged, using computational complexity to explain the complexity of cognitive tasks may be empirically justified. This empirical aspect notwithstanding, however, in practice the exclusive focus on the computational level of explanation can make the approach largely a priori. In a purely computational-level approach it is natural to assume that human competence can be modeled by optimal algorithms for solving mathematical problems, rather than studying empirically what kind of problem solving algorithms actual human reasoners use.

While this computational-level approach has clear advantages, I submit that there should be limits to how strong and wide the application of the a priori computational methodology should be. As fruitful as the computational complexity paradigm may be, we should not dismiss the possibility that human competence in mathematical problem solving may indeed differ in important and systematic ways from the optimal algorithms studied in the computational complexity approach. In the rest of this paper, I will argue that by including considerations on the algorithmic level, we can get a more informative framework for studying the actual human problem solving capacity. Furthermore, I will show that the algorithmic-level approach does not move the discussion from competence to performance. Instead, we get a theoretical framework that is better-equipped for explaining human competence by including considerations of the algorithms that are cognitively optimal for human reasoners.[13]

[13] In this way, the approach here resembles that of Peacocke (1986), who proposed amending the Marr classification by adding a level "1.5" of cognitive explanation (1 being the computational level and 2 the algorithmic). He argued that the purely extensional computational approach corresponds neither to Chomsky's competence nor even to the scientific practice that Marr intended. If we are concerned only with the output and the input, we are left with a multitude of characterizations of the functions involved. For example, if we think of functions simply as Cartesian sets of ordered pairs (i.e. purely extensionally), we learn nothing about the intensional intricacies of the function. But surely such purely syntactic pairing of inputs with outputs (with, e.g., a look-up table) is a inferior characterization (cognitively) than a description of a function that implies how it is cognitively computed. In this way, the characterization of the function may already point us toward the algorithms used in cognitive processing.

a avaliação da complexidade cognitiva pode, de fato, basear-se em resultados e mpíricos. Primeiro, todos os dados que temos sobre as capacidades cognitivas em geral podem ser usados para justificar a avaliação da complexidade das tarefas matemáticas. Os limites da memória de trabalho, por exemplo, impõem limites à nossa capacidade de resolução de problemas, assim como as limitações físicas na utilização de ferramentas externas. Não precisamos estudar empiricamente cada problema matemático se pudermos estabelecer que uma determinada classe de problemas exige mais da memória de trabalho do que outra classe. Em segundo lugar, o método geral de utilização da complexidade computacional para caracterizar a complexidade das t arefas de resolução de problemas só funciona porque em muitos casos sabemos que a competência humana é razoavelmente bem modelada por algoritmos computacionalmente óptimos. Dados empíricos sobre a competência na adição de (pequenos) números naturais, por exemplo, implicam que um algoritmo ideal pode ser usado para caracterizar a complexidade da tarefa cognitiva humana (Fuson 1992).

Desta forma, o que pode parecer uma avaliação a priori das capacidades cognitivas pode, na verdade, basear-se, pelo menos parcialmente, em considerações empíricas. Embora os dados empíricos possam não ser explicitamente reconhecidos, o uso da complexidade computacional para explicar a complexidade das tarefas cognitivas pode ser justificado empiricamente. Apesar deste aspecto empírico, na prática, o foco exclusivo no nível computacional de explicação pode tornar a abordagem em grande parte a priori. Numa abordagem de nível puramente computacional, é natural assumir que a competência humana pode ser modelada por algoritmos óptimos para resolver problemas matemáticos, em vez de estudar empiricamente que tipo de algoritmos de resolução de problemas os verdadeiros raciocinadores humanos usam.

Embora esta abordagem a nível computacional tenha vantagens claras, defendo que deveria haver limites para o quão forte e ampla deveria ser a aplicação da metodologia computacional a priori. Por mais frutífero que possa ser o paradigma da complexidade computacional, não devemos descartar a possibilidade de que a competência humana na resolução de problemas matemáticos possa de facto diferir de formas importantes e sistemáticas dos algoritmos óptimos estudados na abordagem da complexidade computacional. No restante deste artigo, argumentarei que, ao incluir considerações no nível algorítmico, podemos obter uma estrutura mais informativa para estudar a real capacidade humana de resolução de problemas. Além disso, mostrarei que a abordagem em nível algorítmico não transfere a discussão da competência para o desempenho. Em vez disso, obtemos uma estrutura teórica mais bem equipada para explicar a competência humana, incluindo considerações sobre os algoritmos que são cognitivamente ideais para raciocinadores humanos.[13]

13 Desta forma, a abordagem aqui se assemelha à de Peacocke (1986), que propôs alterar a classificação de Marr adicionando um nível "1,5" de explicação cognitiva (sendo 1 o nível computacional e 2 o algorítmico). Ele argumentou que a abordagem computacional puramente extensional não corresponde nem à competência de Chomsky nem mesmo à prática científica que Marr pretendia. Se estivermos preocupados apenas com a saída e a entrada, ficamos com uma infinidade de caracterizações das funções envolvidas. Por exemplo, se pensarmos nas funções simplesmente como conjuntos cartesianos de pares ordenados (ou seja, puramente extensionalmente), não aprenderemos nada sobre os meandros intensionais da função. Mas certamente esse emparelhamento puramente sintático de entradas com saídas (com, por exemplo, uma tabela de consulta) é uma caracterização inferior (cognitivamente) do que uma descrição de uma função que implica como ela é computada cognitivamente. Dessa forma, a caracterização da função já pode nos apontar para os algoritmos utilizados no processamento cognitivo.

6 Humanly Optimal Algorithms

When we study human mathematical problem solving abilities, it is quite obvious that we will encounter a great deal of suboptimal performance. Humans make errors and even when correct, they may use algorithms that are less than optimal. In the tradition of Chomsky and Marr, such use of suboptimal algorithms is deemed to be variation in performance and as such irrelevant for studying competence. Since that paradigm aims to explain general human competence when it comes to mathematical problem solving, in that respect it is not important that people do not always reach the full competence, nor that even competent problem-solvers are not always completely error-free and optimal.

However, when seen in the current setting of the computational complexity approach to modeling cognitive processes in mathematical problem solving, there is a potentially serious difficulty involved. In the study of computational complexity we are (in this regard) mostly interested in optimal algorithms. If we characterize the complexity of the cognitive task of solving a mathematical problem through the complexity of the problem, we are implicitly assuming that the competence of human reasoners can be characterized by a computationally optimal algorithm.

This approach seems to be unproblematic when it comes to some mathematical problems. Human competence in the addition of integers, for example, seems to use an optimal algorithm, and there is also evidence that this is the case in certain logical tasks (Szymanik 2016, Section 5). However, in what follows, I will argue that there is no reason to believe that this is *generally* the case. In fact, I will argue that there are several ways in which the human problem solving cognitive capacity differs essentially from the optimal algorithms for solving mathematical problems.[14]

There have been some arguments (e.g., Penrose 1989) to the effect that human mathematical ability could actually rise *above* the power of any algorithms, by a special intuition or mathematical insight. I consider those arguments to be dubious and will not focus on them here. I believe that mathematical intuition and insight are important subjects and they no doubt play an important role in mathematical problem solving. However, based on the definition of optimal algorithms, I cannot see how they could lead to super-optimal solutions to mathematical problems. If they did, we would presumably be able to model this "insightful" solution, which would then turn out to be an optimal solution.[15] Under the computational complexity approach—when possible—the complexity of mathematical problems is characterized by optimal algorithms. Such algorithms do not need to be unique, but by definition, every other algorithm for solving the problem takes equally long or longer to run than an optimal algorithm (or performs at best a constant factor better).[16] With human insight, we may be able to provide feasible hypotheses, partial

[14] An upshot of this approach is that we can also better understand why human competence sometimes can be modeled by an optimal algorithm and sometimes not.

[15] Of course this argument fails if we believe that human insight cannot be even in principle computationally modeled, as Penrose does.

[16] Or it requires as much as or more space, or it has equal or higher Kolmogorov complexity, etc.

6 algoritmos humanamente ideais

Quando estudamos as habilidades humanas de resolução de problemas matemáticos, é bastante óbvio que encontraremos um grande desempenho abaixo do ideal. Os humanos cometem erros e, mesmo quando corretos, podem usar algoritmos que não são ideais. Na tradição de Chomsky e Marr, esse uso de algoritmos subótimos é considerado uma variação no desempenho e, como tal, irrelevante para o estudo da competência. Uma vez que esse paradigma visa explicar a competência humana geral quando se trata de resolução de problemas matemáticos, nesse aspecto não é i mportante que as pessoas nem sempre atinjam a competência total, nem que mesmo os solucionadores de problemas competentes nem sempre sejam completamente isentos de erros e óptimos.

No entanto, quando visto no cenário atual da abordagem da complexidade computacional para modelar processos cognitivos na resolução de problemas matemáticos, há uma dificuldade potencialmente séria envolvida. No estudo da complexidade computacional estamos (nesse aspecto) principalmente interessados em algoritmos ótimos. Se caracterizarmos a complexidade da tarefa cognitiva de resolver um problema matemático através da complexidade do problema, estaremos implicitamente assumindo que a competência dos raciocinadores humanos pode ser caracterizada por um algoritmo computacionalmente óptimo.

Esta abordagem parece não ser problemática quando se trata de alguns problemas matemáticos. A competência humana na adição de números inteiros, por exemplo, parece utilizar um algoritmo óptimo, e há também evidências de que este é o caso em certas tarefas lógicas (Szymanik 2016, Secção 5). Contudo, no que se segue, argumentarei que não há razão para acreditar que este seja geralmente o caso. Na verdade, argumentarei que existem várias maneiras pelas quais a capacidade cognitiva de resolução de problemas humanos difere essencialmente dos algoritmos ideais para resolver problemas matemáticos.[14]

Tem havido alguns argumentos (por exemplo, Penrose 1989) no sentido de que a capacidade matemática humana poderia realmente elevar-se acima do poder de qualquer algoritmo, por uma intuição especial ou visão matemática. Considero esses argumentos duvidosos e não vou me concentrar neles aqui. Acredito que a intuição e o insight matemáticos são assuntos importantes e, sem dúvida, desempenham um papel importante na resolução de problemas matemáticos. Contudo, com base na definição de algoritmos ótimos, não consigo ver como eles poderiam levar a soluções superótimas para problemas matemáticos. Se o fizessem, presumivelmente seríamos capazes de modelar esta solução "perspicaz", que se revelaria então uma solução óptima.[15] Sob a abordagem da complexidade computacional – quando possível – a complexidade dos problemas matemáticos é caracterizada por algoritmos óptimos. Esses algoritmos não precisam ser únicos, mas, por definição, todos os outros algoritmos para resolver o problema levam tanto tempo ou mais para serem executados do que um algoritmo ideal (ou executam melhor, na melhor das hipóteses, um fator constante).[16] Com o insight humano, podemos ser capazes de fornecer hipóteses viáveis, parciais.

14 Um resultado desta abordagem é que também podemos compreender melhor porque é que a competência humana por vezes pode ser modelada por um algoritmo óptimo e por vezes não. 15 É claro que este argumento falha se acreditarmos que o insight humano não pode nem mesmo em princípio ser modelado computacionalmente, como faz Penrose. 16 Ou requer tanto ou mais espaço, ou tem complexidade Kolmogorov igual ou superior, etc.

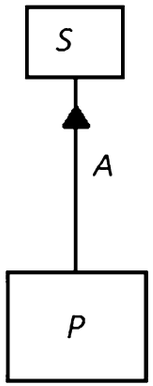


Fig. 1 An optimal algorithm A takes time t to get the solution S

or approximate solutions, or structures and strategies for solutions—and this process may take less time than an optimal algorithm. But we cannot have full step-by-step solutions that are faster than an optimal solution.

However, this does not mean that special human cognitive characteristics (what we might call "insight") do not play an important role in mathematical problem solving. Here I want to distinguish between optimal algorithms and *humanly* optimal algorithms. The key idea is that the solution that is most easily accessible to human cognizers may not always be a computationally optimal one.

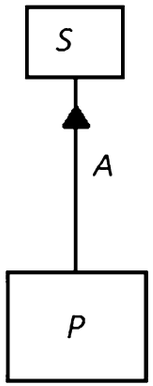
Let us say that an algorithm A is optimal for reaching the solution S to the problem P and it takes time t to run it (Fig. 1). Now by studying human performance in solving P, we may find out that even if the solution S is correct, the actual algorithm $A*$ may have been different from the optimal algorithm A (or an equivalent other optimal algorithm) and by definition the time taken $t*> t$ (Fig. 2).

In the computational-level approach of Chomsky and Marr, such a result would be dismissed due to it concerning performance rather than competence. But what if we acquired extensive data and found out that humans *generally* use a suboptimal algorithm like $A*$ to solve the problem? In that case there would clearly be a unique algorithm to describe the human competence, yet it would not correspond to an optimal algorithm. Such an example would show, contra Marr, that the explanation cannot focus mainly on the computational level. As we have seen, in the computational complexity approach optimal algorithms are used to model human cognitive competence. But if human performance generally follows a suboptimal algorithm, how can we retain this approach?

In the rest of this paper I will argue that indeed we cannot, and we must take into account also Marr's Level 2, the algorithmic level of explanation.[17] We will start by looking at three cases in which the humanly optimal algorithms for mathematical

[17] To be clear, I am not claiming that researchers in the cognitive sciences generally dismiss the importance of algorithmic-level explanations. Often considerations on the algorithmic and implementational levels of processing can also influence computational-level explanations, the Marrian focus on computational level of explanation notwithstanding.

Fig. 1 Um algoritmo ótimo A leva tempo t para obter a solução S



ou soluções aproximadas, ou estruturas e estratégias para soluções – e esse processo pode levar menos tempo do que um algoritmo ideal. Mas não podemos ter soluções passo a passo completas que sejam mais rápidas do que uma solução óptima.

Contudo, isto não significa que características cognitivas humanas especiais (o que poderíamos chamar de "insight") não desempenhem um papel importante na resolução de problemas matemáticos. Aqui quero distinguir entre algoritmos ótimos e algoritmos humanamente ótimos. A ideia principal é que a solução mais facilmente acessível aos conhecedores humanos pode nem sempre ser computacionalmente ideal.

Digamos que um algoritmo A é ótimo para alcançar a solução S para o problema P e leva um tempo t para executá-lo (Fig. 1). Agora, estudando o desempenho humano na resolução de P, podemos descobrir que mesmo que a solução S esteja correta, o algoritmo real A* pode ter sido diferente do algoritmo ótimo A (ou de outro algoritmo ótimo equivalente) e, por definição, do tempo gasto t*> t (Fig. 2).

Na abordagem de nível computacional de Chomsky e Marr, tal resultado seria descartado por se tratar mais de desempenho do que de competência. Mas e se adquirirmos dados extensos e descobrirmos que os humanos geralmente usam um algoritmo abaixo do ideal como A* para resolver o problema? Nesse caso, haveria claramente um algoritmo único para descrever a competência humana, mas não corresponderia a um algoritmo óptimo. Tal exemplo mostraria, contrariamente a Marr, que a explicação não pode concentrar-se principalmente no nível computacional. Co mo vimos, na abordagem da complexidade computacional são usados algoritmos ótimos para modelar a competência cognitiva humana. Mas se o desempenho humano geralmente segue um algoritmo abaixo do ideal, como podemos manter esta abordagem?

No restante deste artigo argumentarei que de fato não podemos, e devemos levar em conta também o Nível 2 de Marr, o nível algorítmico de explicação.[17] Começaremos examinando três casos em que os algoritmos humanamente ótimos para matemática

17 Para ser claro, não estou a afirmar que os investigadores das ciências cognitivas geralmente descartam a importância das explicações a nível algorítmico. Freqüentemente, considerações sobre os níveis algorítmicos e de implementação de processamento também podem influenciar as explicações em nível computacional, apesar do foco de Marrian no nível computacional de explicação.



Fig. 2 If the human algorithm $A*$ is not optimal, the time taken $t* > t$

problem solving are not the same as the algorithms with the lowest complexity, and assess the computational approach in each of them.

Before that, however, we need to ask how we should understand a "human" problem solver and what exactly is meant by "humanly optimal algorithm". Clearly human mathematical problem solving is in most cases aided by some tools, ranging from fingers to pens and paper to computer programs. Indeed, anything we can achieve with the help of computers is human endeavor and thus it could be argued that all mathematical problem solving is ultimately human problem solving. Perhaps we could limit human problem solving to mental operations carried out in the working memory without any external aids, but that would seem to be needlessly limiting given the importance of external tools for mathematical practice in all its levels. In the face of this, it may seem that any characterization of "human" problem solving is largely arbitrary. Moreover, many problem solving processes include several human beings. Should we limit the approach to individual problem solvers? But then how should we assess the cultural input of other people on that individual?

Similar questions may be asked about "humanly optimal" algorithms. Should we consider the tools used in problem solving processes? If so, how should we account for the differences in tools that different cultures have? For example, there can be different cultural emphasis how pen-and-paper methods and tools like abacus are used for solving arithmetical problems. Indeed, problem solving strategies in mental arithmetic also differ culturally—partly due to the different tools used—and this can be seen as differences in the brain region activation associated with mental arithmetic (Tang et al. 2006). How is it possible, from this background, to characterize a problem solving algorithm as "humanly optimal"?

These are all relevant questions and a full treatment of them is unfortunately not possible. We treat these issues in detail in (Fabry and Pantsar 2019). However, it should be pointed out here that while I do not want to suggest a particular limitations, I believe that even in the face of such questions, there can be meaningful characterizations of "human" problem solving and they can be explanatorily useful. What such characterizations should do, at the minimum, is pick out problem solving processes which are used generally enough, and in which the cognitive process is

Fig. 2 Se o algoritmo humano A* não for ideal, o tempo necessário t∗ > t



a resolução de problemas não são iguais aos algoritmos de menor complexidade e avaliam a abordagem computacional em cada um deles.

Antes disso, porém, precisamos perguntar como devemos entender um solucionador de problemas "humano" e o que exatamente significa "algoritmo humanamente ideal". É evidente que a resolução humana de problemas matemáticos é, na maioria dos casos, auxiliada por algumas ferramentas, que vão desde dedos a canetas e papel a programas de computador. Na verdade, tudo o que podemos alcançar com a ajuda de computadores é esforço humano e, portanto, pode-se argumentar que toda resolução de problemas matemáticos é, em última análise, resolução de problemas humanos. Talvez pudéssemos limitar a resolução de problemas humanos a operações mentais realizadas na memória de trabalho sem quaisquer ajudas externas, mas isso pareceria ser desnecessariamente li mitante, dada a importância das ferramentas externas para a prática matemática em todos os seus níveis. Perante isto, pode parecer que qualquer caracterização da resolução "humana" de problemas é em grande parte arbitrária. Além disso, muitos processos de resolução de problemas incluem vários seres humanos. Deveríamos limitar a abordagem aos solucionadores de problemas individuais? Mas então como deveríamos avaliar a contribuição cultural de outras pessoas sobre esse indivíduo?

Perguntas semelhantes podem ser feitas sobre algoritmos "humanamente ideais". Devemos considerar as ferramentas utilizadas nos processos de resolução de problemas? Se sim, como devemos explicar as diferenças nas ferramentas que as diferentes culturas possuem? Por exemplo, pode haver diferentes ênfases culturais na forma como métodos e ferramentas de caneta e papel, como o ábaco, são usados para resolver problemas aritméticos. Na verdade, as estratégias de resolução de problemas em aritmética mental também diferem culturalmente – em parte devido às diferentes ferramentas utilizadas – e isto pode ser visto como diferenças na activação da região cerebral associada à aritmética mental (Tang et al. 2006). Como é possível, a partir deste contexto, caracterizar um algoritmo de resolução de problemas como "humanamente ideal"?

Todas estas são questões relevantes e, infelizmente, não é possível um tratamento completo delas. Tratamos essas questões detalhadamente em (Fabry e Pantsar 2019). No entanto, deve ser salientado aqui que, embora eu não queira sugerir limitações específicas, acredito que, mesmo diante de tais questões, pode haver caracterizações significativas da resolução de problemas "humanos" e elas podem ser explicativamente úteis. O que tais caracterizações deveriam fazer, no mínimo, é selecionar processos de resolução de problemas que sejam usados de forma bastante geral e nos quais o processo cognitivo seja

essential to reaching the solution. Importantly for the purposes of this paper, under such a characterization, simply typing an input to a computer program and reading an output does not count as a human problem solving process.

Similarly, I want to introduce the term "humanly optimal algorithm" as a characterization that we can use as a tool in modeling the problem solving strategies that human cognizers use, rather than something that could be universally defined for particular mathematical problems. As will be discussed in Sect. 7, what is humanly optimal depends on how our problem solving processes are culturally determined. As such, the general use of the term "humanly optimal algorithm" should be understood as a tool that can be used to introduce a new approach to modeling human problem solving processes and characterizing their complexity. The important idea is that humanly optimal algorithms are not necessarily computationally optimal, but cognitively optimal for human cognitive agents with specific learning trajectories. I will argue that this approach allows for more accurate characterizations than the computational-level approach focusing on computational complexity. In the final section of this paper, we will see in more detail how the concept of humanly optimal algorithm can be used in pursuing more accurate models of human problem solving processes, and therefore also more accurate characterizations of their cognitive complexity. But first, let us take a look at some examples of the ways in which human problem solving processes can be computationally suboptimal.

Example 1: Fast computer algorithms

In many cases, it is understandable that computer algorithms for solving mathematical problems are modeled after the algorithms we have learned in school. In the case of addition, for example, the most common human algorithm would also appear to be an optimal one. Also for multiplication, there would seem to be nothing wrong with using the standard schoolbook algorithm. Multiplying integers with the standard method we have learned in school takes roughly n^2 steps for two n-digit integers, thus making it a relatively low-complex problem in **P**. However, Schönhage and Strassen (1971) have shown that there are also more advanced algorithms for multiplying large integers, ones that have lower upper bounds for the running time. The so-called Fast Fourier transforms are another algorithmic method that greatly reduces the complexity of computing discrete Fourier transforms of sequences. Yet another example can be found in matrix multiplication, in which the Strassen algorithm is quicker for large matrices than the standard method we learn in school. Multiplying two matrices of the size 2×2 with the standard method learned requires 8 multiplications. The Strassen algorithm, however, manages to reduce this to 7 multiplications by adding new addition and subtraction operations. In this way, the algorithm is actually more complex in that it requires more steps, but since multiplications take more time than addition and subtraction, for large matrices the Strassen algorithm becomes faster than the standard algorithm. Even faster algorithms for matrix multiplication have been invented, thus making the standard method increasingly less optimal in terms of time-complexity (Skiena 2008).

Due to limits in space, it is not possible to present the details of these algorithms, but in all three cases, human problem solvers do not generally use the faster

essencial para chegar à solução. É importante ressaltar que, para os propósitos deste artigo, sob tal caracterização, simplesmente digitar uma entrada em um programa de computador e ler uma saída não conta como um processo humano de resolução de problemas.

Da mesma forma, quero introduzir o termo "algoritmo humanamente ótimo" como uma caracterização que podemos usar como uma ferramenta na modelagem das estratégias de resolução de problemas que os conhecedores humanos usam, em vez de algo que poderia ser universalmente definido para problemas matemáticos específicos. Como será discutido na Seção. 7, o que é humanamente ideal depende de como os nossos processos de resolução de problemas são culturalmente determinados. Como tal, o uso geral do termo "algoritmo humanamente ótimo" deve ser entendido como uma ferramenta que pode ser usada para introduzir uma nova abordagem para modelar processos humanos de resolução de problemas e caracterizar a sua complexidade. A ideia importante é que algoritmos humanamente ótimos não são necessariamente computacionalmente ótimos, mas cognitivamente ótimos para agentes cognitivos humanos com trajetórias de aprendizagem específicas. Argumentarei que esta abordagem permite caracterizações mais precisas do que a abordagem de nível computacional focada na complexidade computacional. Na seção final deste artigo, veremos com mais detalhes como o conceito de algoritmo humanamente ótimo pode ser usado na busca de modelos mais precisos de processos humanos de resolução de problemas e, portanto, também em caracterizações mais precisas de sua complexidade cognitiva. Mas primeiro, vamos dar uma olhada em alguns exemplos de como os processos humanos de resolução de problemas podem ser computacionalmente abaixo do ideal.

Exemplo 1: Algoritmos de computador rápidos

Em muitos casos, é compreensível que os algoritmos de computador para resolver problemas matemáticos sejam modelados a partir dos algoritmos que aprendemos na escola. No caso da adição, por exemplo, o algoritmo humano mais comum também parece ser o ideal. Também para a multiplicação, parece não haver nada de errado em usar o algoritmo padrão do livro escolar. Multiplicar números inteiros com o método padrão que aprendemos na escola leva aproximadamente n^2 passos para dois números inteiros de n dígitos, tornando-o assim um problema de complexidade relativamente baixa em P. No entanto, Schönhage e Strassen (1971) mostraram que também existem algoritmos mais avançados para multiplicar números inteiros grandes, aqueles que tê m limites superiores inferiores para o tempo de execução. As chamadas transformadas rápidas de Fourier são outro método algorítmico que reduz bastante a complexidade do cálculo de transformadas discretas de Fourier de sequências. Ainda outro exemplo pode ser encontrado na multiplicação de matrizes, em que o algoritmo de Strassen é mais rápido para matrizes grandes do que o método padrão que aprendemos na escola. Multiplicar duas matrizes de tamanho 2×2 com o método padrão aprendido requer 8 multiplicações. O algoritmo Strassen, entretanto, consegue reduzir isso para 7 multiplicações adicionando novas operações de adição e subtração. Desta forma, o algoritmo é na verdade mais complexo porque requer mais passos, mas como as multiplicações levam mais tempo do que a adição e a subtração, para matrizes grandes o algoritmo de Strassen torna-se mais rápido que o algoritmo padrão. Algoritmos ainda mais rápidos para multiplicação de matrizes foram inventados, tornando o método padrão cada vez menos ideal em termos de complexidade de tempo (Skiena 2008).

Devido aos limites de espaço, não é possível apresentar os detalhes destes algoritmos, mas em todos os três casos, os solucionadores de problemas humanos geralmente não usam o algoritmo mais rápido.

algorithm to solve the task. Thus all three examples appear to give us clear cases where algorithms with lower time-complexities are not used by human problem solvers. This would seem to go against the basic paradigm of the computational complexity approach to mathematical problem solving: for some mathematical problems, the lowest-complexity algorithms cannot feasibly characterize the cognitive processes of human problem solvers.

However, it could be that all three cases are relevant in a limited manner, since the advantage of the fast algorithms only starts to show with operations that involve very large numbers. The Karatsuba algorithm, for example—which used to be the fastest multiplication algorithm before the Schönhage-Strassen algorithm—only starts to outperform the standard algorithm when the integers are hundreds of bits long (Karatsuba and Ofman 1962). It is unfeasible that a human problem solver (as characterized above) would ever perform this kind of multiplication, and it could thus still be the case that for all the relevant multiplications—those that human beings could in principle engage in—the standard algorithm is also a computationally optimal one, and not merely humanly optimal. Thus we should perhaps look elsewhere for more widely relevant humanly optimal algorithms which are not computationally optimal.

Example 2: Diagrams

Based on the above considerations, it is unlikely that these kinds of simple mechanical operations are where we can find the kinds of humanly optimal algorithms we are after. For that, we should look at more complex mathematical tasks, in which the human cognitive task could differ importantly from an optimal computational solution. Unfortunately, however, the more complex a mathematical task is, the more problematic it becomes to study the cognitive processes that are involved in solving it. Still, mathematical practice gives us reason to believe that mathematical problem solving generally is not a case of applying an optimal algorithm. Instead of merely constructing step-by-step solutions, in mathematics a wide array of different cognitive resources are used. As analyzed by, e.g., Schoenfeld (1985), it is clear that problem solving is not a straight-forward process where different algorithms are tested until the correct one is iteratively hit upon. In Schoenfeld's analysis, mathematical problem solving draws from four factors: resources, heuristics, control and belief systems. While control (i.e., resource management) and belief systems (i.e., mathematical world view) form interesting research questions, here I am more interested in the first two. How do human cognizers typically use their body of knowledge about mathematics to solve a particular problem? And crucially to the matter at hand, can analyzing human problem solving from this perspective provide insight into cognitive complexity that goes beyond the computational complexity approach?

One strategy that humans constantly use in mathematical problem solving are different types of heuristic, or didactic, processes. These should not be confused with the heuristic algorithms we discussed earlier. Whereas heuristic algorithms in computer science provide partial or approximate solutions, humans use heuristic

algoritmo para resolver a tarefa. Assim, todos os três exemplos parecem nos dar casos claros em que algoritmos com menor complexidade de tempo não são usados por solucionadores de problemas humanos. Isto parece ir contra o paradigma básico da abordagem da complexidade computacional para a resolução de problemas matemáticos: para alguns problemas matemáticos, os algoritmos de menor complexidade não podem caracterizar de forma viável os processos cognitivos dos solucionadores de problemas humanos.

Contudo, pode ser que todos os três casos sejam relevantes de forma limitada, uma vez que a vantagem dos algoritmos rápidos só começa a aparecer com operações que envolvem números muito grandes. O algoritmo Karatsuba, por exemplo – que costumava ser o algoritmo de multiplicação mais rápido antes do algoritmo Schönhage-Strassen – só começa a superar o algoritmo padrão quando os números inteiros têm centenas de bits (Karatsuba e Ofman 1962). É inviável que um solucionador de problemas humano (conforme caracterizado acima) alguma vez realize este tipo de multiplicação e, portanto, ainda poderia ser o caso de que, para todas as multiplicações relevantes – aquelas em que os seres humanos poderiam, em princípio, se envolver – o algoritmo padrão fosse também um algoritmo computacionalmente óptimo, e não apenas humanamente óptimo. Assim, talvez devêssemos procurar em outro lugar algoritmos humanamente ótimos mais amplamente relevantes que não sejam computacionalmente ótimos.

Exemplo 2: Diagramas

Com base nas considerações acima, é improvável que seja neste tipo de operações mecânicas simples que possamos encontrar os tipos de algoritmos humanamente ótimos que procuramos. Para isso, deveríamos olhar para tarefas matemáticas mais complexas, nas quais a tarefa cognitiva humana poderia diferir significativamente de uma solução computacional ótima. Infelizmente, porém, quanto mais complexa é uma tarefa matemática, mais problemático se torna o estudo dos processos cognitivos envolvidos na sua resolução. Ainda assim, a prática matemática dá-nos razões para acreditar que a resolução de problemas matemáticos geralmente não é um caso de aplicação de um algoritmo óptimo. Em vez de meramente construir soluções passo a passo, na matemática é utilizada uma ampla gama de diferentes recursos cognitivos. Conforme analisado, por exemplo, por Schoenfeld (1985), fica claro que a resolução de problemas não é um processo direto, onde diferentes algoritmos são testados até que o correto seja atingido iterativamente. Na análise de Schoenfeld, a resolução de problemas matemáticos baseia-se em quatro factores: recursos, heurística, controlo e sistemas de crenças. Embora o controle (ou seja, a gestão de recursos) e os sistemas de cre nças (ou seja, a visão matemática do mundo) formem questões de pesquisa interessantes, aq ui estou mais interessado nas duas primeiras. Como os conhecedores humanos normalmente usam seu corpo de conhecimento sobre matemática para resolver um problema específico? E, crucialmente para o assunto em questão, será que a análise da resolução de problemas humanos a partir desta perspectiva pode fornecer informações sobre a complexidade cognitiva que vão além da abordagem da complexidade computacional?

Uma estratégia que os humanos usam constantemente na resolução de problemas matemáticos são os diferentes tipos de processos heurísticos ou didáticos. Estes não devem ser confundidos com os algoritmos heurísticos que discutimos anteriormente. Enquanto os algoritmos heurísticos na ciência da computação fornecem soluções parciais ou aproxi madas, os humanos usam algoritmos heurísticos

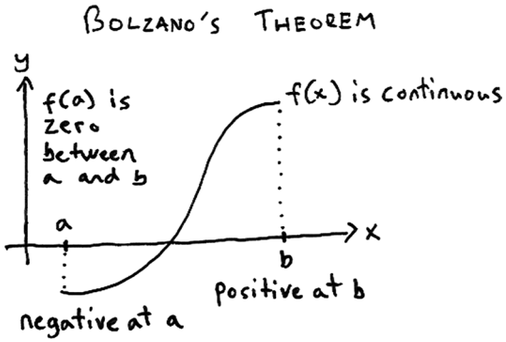


Fig. 3 A typical hand-drawn diagram of Bolzano's theorem

processes also in processes that lead to exact solutions.[18] Among these heuristic and didactic processes, drawing diagrams is probably the most prevalent. Ever since Polya's *How to solve it* (1945) and Hadamard's *The Psychology of Invention In The Mathematical Field* (1954) were first published, the study of heuristics and didactic methods of mathematical problem solving has been an important topic in mathematics education. Starting from perhaps Polya's most commonly applied rule "If you are having difficulty understanding a problem, try drawing a picture", some didactic rules have become the standard way in which mathematics is taught. The great usefulness of pictures and diagrams has been confirmed in an enormous number of studies (see, e.g., Jamnik 2001; Uesaka et al. 2007; Diezmann and English 2001). Yet applying already that simple rule of Polya takes us away from the computational complexity approach to characterizing human problem solving. Generally speaking, computers do not benefit from visualizing problems and adding this sort of didactic process to a problem solving computer program makes it less optimal, thus (from the computational perspective) needlessly increasing the complexity of the solution.[19] For human problem solvers, however, the matter is quite different. The computationally superfluous pictures, analogues and such can be absolutely crucial for finding the solution.

Let us consider a simple theorem from analysis, Bolzano's theorem, stating that if a continuous function gets both negative and positive values inside an interval, it must have a root in that interval. Most students of mathematics are familiar with some diagram explaining the idea of the theorem (Fig. 3). Furthermore, for most students, this kind of diagram helps grasp the content of the theorem (Zachariades et al. 2007). Yet it is clear that in the mathematical community, drawing this kind of a diagram would not be considered to be an acceptable solution to a problem. Instead, the diagram works as a didactic tool that helps us better understand the formal theorem.

[18] Of course in addition to the many applications humans have for heuristic processes that do *not* lead to exact solutions.

[19] Assuming that the computer is required to provide the solution to the problem in the initial mode of representation. Perhaps there could be an exception in highly complex computer programs, but it is enough for present purposes to note that changing the domain of representation is generally likely to add to the complexity of the algorithm.

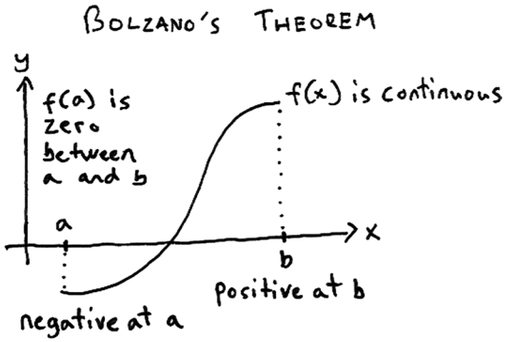


Fig. 3 Um diagrama típico desenhado à mão do teorema de Bolzano

processos também em processos que levam a soluções exatas.[18] Entre esses processos heurísticos e didáticos, desenhar diagramas é provavelmente o mais prevalente. Desde que How to resolve it (1945), de Polya, e The Psychology of Invention In The Mathematical Field (1954), de Hadamard, foram publicados pela primeira vez, o estudo de heurísticas e métodos didáticos de resolução de problemas matemáticos tem sido u m tópico importante na educação matemática. Partindo talvez da regra mais comummente aplicada de Polya "Se tiver dificuldade em compreender um problema, tente fazer um desenho", algumas regras didáticas tornaram-se a forma padrão de ensino da matemática. A grande utilidade das imagens e diagramas foi confirmada nu m enorme número de estudos (ver, por exemplo, Jamnik 2001; Uesaka et al. 2007; Diezmann e English 2001). No entanto, aplicar já essa regra simples de Polya nos afasta da abordagem da complexidade computacional para caracterizar a resolução humana de problemas. De modo geral, os computadores não se beneficiam da visualização de problemas e adicionar esse tipo de processo didático a um programa de computador para resolução de problemas torna-o menos ideal, aumentando assim (do ponto de vista computacional) desnecessariamente a complexidade da solução.[19] Para os solucionadores de problemas humanos, no entanto, a questão é bem diferente. As imagens, análogos e outros computacionalmente supérfluos podem ser absolutamente cruciais para encontrar a solução.

Consideremos um teorema simples de análise, o teorema de Bolzano, afirmando que se uma função contínua obtém valores negativos e positivos dentro de um i ntervalo, ela deve ter uma raiz nesse intervalo. A maioria dos estudantes de matemática está familiarizada com algum diagrama que explica a ideia do teorema (Fig. 3). Além disso, para a maioria dos estudantes, este tipo de diagrama ajuda a compreender o conteúdo do teorema (Zachariades et al. 2007). No entanto, é claro que na comunidade matemática, desenhar este tipo de diagrama não seria considerado u ma solução aceitável para um problema. Em vez disso, o diagrama funciona como uma ferramenta didática que nos ajuda a compreender melhor o teorema formal.

18 É claro que, além das muitas aplicações que os humanos têm para processos heurísticos que não levam a soluções exatas. 19 Supondo que o computador seja necessário para fornecer a solução do problema no modo inicial de representação. Talvez pudesse haver uma exceção em programas de computador altamente complexos, mas é suficiente para os presentes propósitos notar que a mudança do domínio de representação geralmente aumenta a complexidade do algoritmo.

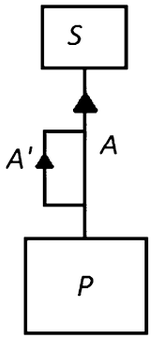


Fig. 4 An algorithm A' that employs diagrams adds to the complexity of the solution

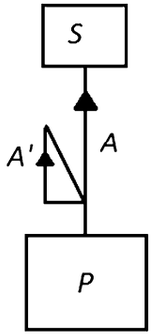


Fig. 5 A purely didactic algorithm A' does not advance the formal solution

But from the computational point of view, drawing the diagram only makes the algorithm human cognizers use more complex than an optimal algorithm (Fig. 4). Since *A'* introduces a parallel process, it must be the case that the time used by it is more than the time used by an optimal algorithm, i.e., *t'*> *t*.

In fact, in many cases, the diagram does not advance the formal solution at all. In such cases, the function of the diagram is purely didactic (Fig. 5). What the diagram does in such cases is assist us in finding the formal solution. For a problem solving computer program, this would mean that the initial representation of a mathematical problem is changed into another mode of representation, but ultimately the solution must be presented in terms of the initial representation. The solution via this process is generally computationally more complex than an optimal algorithm for solving the problem. In the case of Bolzano's theorem, for example, it seems clear that a theorem-proving computer program would not benefit from changing into a visual representation.

It is of course possible that diagrams and other didactic methods are not indispensable for all humans in solving any particular mathematical problem. Even if diagrams were generally useful, it is possible that all mathematical problems *could* be solved by some human reasoners without them. In that case we could in principle find for any problem a solution in which the human cognizer uses a computationally optimal algorithm. Indispensability, however, is too strong a

Fig. 4 Um algoritmo A' que emprega diagramas aumenta a complexidade da solução

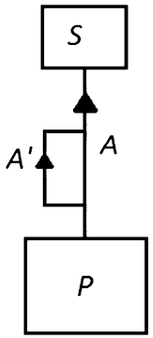
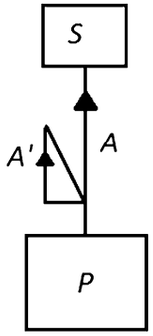


Fig. 5 Um algoritmo puramente didático A' não avança a solução formal



Mas do ponto de vista computacional, desenhar o diagrama apenas torna o algoritmo que os conhecedores humanos usam mais complexo do que um algoritmo ideal (Fig. 4). Como A' introduz um processo paralelo, deve acontecer que o tempo utilizado por ele seja maior que o tempo utilizado por um algoritmo ótimo, ou seja, t'> t.

Na verdade, em muitos casos, o diagrama não avança em nada a solução formal. Nestes casos, a função do diagrama é puramente didática (Fig. 5). O que o diagrama faz nesses casos é nos ajudar a encontrar a solução formal. Para um programa de computador para resolução de problemas, isto significaria que a representação inicial de um problema matemático é transformada em outro modo de representação, mas e m última análise a solução deve ser apresentada em termos da representação inicial. A solução por meio deste processo é geralmente computacionalmente mais complexa do que um algoritmo ideal para resolver o problema. No caso do teorema de Bolzano, por exemplo, parece claro que um programa de computador para provar teoremas não beneficiaria da transformação para uma representação visual.

É claro que é possível que diagramas e outros métodos didáticos não sejam i ndispensáveis para todos os seres humanos na resolução de qualquer problema matemático específico. Mesmo que os diagramas fossem geralmente úteis, é possível que todos os problemas matemáticos pudessem ser resolvidos por alguns raciocinadores humanos sem eles. Nesse caso, poderíamos, em princípio, encontrar para qualquer problema uma solução na qual o cognitor humano usasse um algoritmo computacionalmente ótimo. A indispensabilidade, no entanto, é um argumento demasiado forte

condition for the current purpose. What we want to know is whether there are problems in which human reasoners *generally* benefit from diagrams or other didactic and heuristic tools. With many problems, it seems highly probable that this is the case. If we are looking to establish what the human competence in certain problem solving task is, as indeed is the purpose in the paradigm established by Chomsky and Marr, it seems clear that we should be interested in this general trait, rather than possible outliers.

If our aim is to model human cognitive tasks accurately, we must include considerations on didactic processes such as constructing and interpreting diagrams. The problem is that such matters can be hard to study. The cognitive processes involved in solving even a moderately difficult mathematical problem are not easy to identify with brain scanning methods, thus making explanations on the implementational level problematic. On the algorithmic level, we can measure problem solving time for simple problems, and thus compare the cognitive efficiency of different algorithms used to reach the solution. For more difficult problems of the type that mathematicians actually deal with, however, also this method can be tricky. As the problems become more complex, it becomes increasingly difficult to trace all the parts of the algorithm that the problem solver uses. Often we have to rely either on observing mathematical practice or the testimony of the mathematicians. Both of these methods have significant drawbacks. Observing mathematical practice is likely to include many stages of thought process which are difficult, if not impossible, to detect. Questionnaires and interviews of mathematicians are also potentially problematic. Not only are there the usual problems with unreliability of introspective inquiry, but the kind of didactic, heuristic and intuitive aspects of mathematical thinking that we would wish to identify are often likely to be unconscious and difficult to get reliable data on.

That practical difficulty notwithstanding, however, heuristic and didactic processes like diagrams seem to give us a clear case in which the humanly optimal algorithms may differ from the optimal algorithms, as studied in a purely computational approach to mathematical problems. Consequently, heuristic and didactic processes are also a clear case in point that we should not work exclusively on Marr's computational level. In the computational complexity approach to mathematical problem solving on the computational level, such computationally suboptimal tools are by definition ignored. But given the great use that human problem solvers have for such cognitive tools, it becomes obvious that we should also be interested in the actual algorithms that human cognizers use. Indeed, to understand the human problem solving capacity as well as possible, it is the *humanly* optimal algorithms that we should be looking for—as suboptimal as they may be computationally.

It should be noted here that taking didactic processes into consideration does not in any way imply that the cognitive task of mathematical problem solving could not be computationally modeled. Rather, the argument here is that instead of focusing only on the input (the problem) and the output (the solution), we must be prepared to take into account all the relevant cognitive processes involved in mathematical problem solving. Some of these processes concern individual performance and should not be included in accounts of mathematical competence. But some processes, like those including diagrammatic reasoning, are likely to be generally applied by human

condição para o propósito atual. O que queremos saber é se existem problemas em que os raciocinadores humanos geralmente beneficiam de diagramas ou de outras ferramentas didáticas e heurísticas. Com muitos problemas, parece altamente provável que seja esse o caso. Se pretendemos estabelecer qual é a competência humana em determinadas tarefas de resolução de problemas, como de facto é o propósito do paradigma estabelecido por Chomsky e Marr, parece claro que deveríamos estar interessados nesta característica geral, e não em possíveis outliers.

Se o nosso objetivo é modelar com precisão as tarefas cognitivas humanas, deve mos incluir considerações sobre processos didáticos, como a construção e interpretação de diagramas. O problema é que tais assuntos podem ser difíceis de estudar. Os processos cognitivos envolvidos na resolução, mesmo de um problema matemático moderadamente difícil, não são fáceis de identificar com métodos de varredura cerebral, tornando problemáticas as explicações no nível de implementação. No nível algorítmico, podemos medir o tempo de resolução de problemas simples e, assim, comparar a eficiência cognitiva de diferentes algoritmos usados para chegar à solução. No entanto, para problemas mais difíceis, do tipo com que os matemáticos realmente lidam, este método também pode ser complicado. À medida que os problemas se t ornam mais complexos, torna-se cada vez mais difícil rastrear todas as partes do algoritmo que o solucionador de problemas utiliza. Muitas vezes temos que confiar na observação da prática matemática ou no testemunho dos matemáticos. Ambos os métodos têm desvantagens significativas. A observação da prática matemática provavelmente incluirá muitos estágios do processo de pensamento que são difíceis, se não impossíveis, de detectar. Questionários e entrevistas com matemáticos também são potencialmente problemáticos. Não só existem os problemas habituais de falta de fiabilidade da investigação introspectiva, mas o tipo de aspectos didáticos, heurísticos e intuitivos do pensamento matemático que gostaríamos de identificar são muitas vezes inconscientes e sobre os quais é difícil obter dados fiáveis.

Apesar dessa dificuldade prática, no entanto, processos heurísticos e didáticos como diagramas parecem nos dar um caso claro em que os algoritmos humanamente ótimos pode m diferir dos algoritmos ótimos, conforme estudados em uma abordagem puramente computacional para problemas matemáticos. Consequentemente, os processos heurísticos e didáticos também são um exemplo claro de que não devemos trabalhar exclusivamente no nível computacional de Marr. Na abordagem da complexidade computacional para a resolução de problemas matemáticos no nível computacional, tais ferramentas computacionalmente subótimas são, por definição, ignoradas. Mas dada a grande utilização que os solucionadores de problemas humanos têm para tais ferramentas cognitivas, torna-se óbvio que também deveríamos estar interessados nos algoritmos reais que os conhecedores humanos usam. Na verdade, para compreender da melhor forma possível a capacidade humana de resolução de problemas, são os algoritmos humanamente óptimos que deveríamos procurar – por mais sub-óptimos que possam ser computacionalmente.

Deve-se notar aqui que levar em consideração os processos didáticos não implica de forma alguma que a tarefa cognitiva de resolução de problemas matemáticos não possa ser modelada computacionalmente. Pelo contrário, o argumento aqui é que, em vez de nos concentrarmos apenas na entrada (o problema) e na saída (a solução), devemos estar preparados para ter em conta todos os processos cognitivos relevantes envolvidos na resolução de problemas matemáticos. Alguns destes processos dizem respeito ao desempenho individual e não devem ser incluídos nas contas de competência matemática. Mas alguns processos, como aqueles que incluem o raciocínio diagramático, são provavelmente aplicados geralmente por seres humanos.

problem solvers (at least those with a shared cultural background) and cannot be dismissed as dealing with performance. When it comes to modeling mathematical competence computationally, we need to identify these latter processes before we can know what we are modeling.

Example 3: The Distance effect

Above I have argued that diagrams and other heuristic and didactic methods contribute to humanly optimal algorithms. But there are also ways in which we use suboptimal algorithms which actually make solving problems unnecessarily hard for us. Diagrams concern a relatively sophisticated level of mathematical thinking, but we can see computationally suboptimal algorithms in use already on the very basic, unconscious level of treating mathematical concepts.

Consider the simple task of determining which of the following numbers is bigger:

4 5

Compare this now to the task of determining which of these numbers is bigger:

4 9

One would expect that the task is so simple that in both cases it takes equally long to get the answer. However, the data shows that even for adult subjects, the (4 5) pairing takes considerably longer than (4 9). This is called the *distance effect*: the greater the numerical distance between the two numbers, the quicker we are in solving the problem. The distance effect is usually explained in terms of our automatic tendency to process numerical symbols as quantities. When we process quantities we use the so-called *approximate number system* which is an estimation system that becomes less accurate as the distance between numbers becomes smaller (and the magnitude of numbers becomes larger) (Dehaene 1997/2011, pp. 62–64).[20]

Interestingly, we cannot get rid of the distance effect even if we are trained to solve these types of problems. The effect is also a remarkably strong one, as seen from the following example. Let us have the same task with the these numbers:

71 65

And these numbers:

79 65

Now we would certainly expect the solution to take an equally long time. After all, we can clearly grasp an optimal algorithm for solving the problem, which would be to first compare whether there is a difference in the first digit and only consider the second digit if no such difference exists. Yet also in this case there is a clear

[20] This connection between the distance effect and the approximate number system is widely accepted, but not universally so. Tzelgov and Ganor-Stern (2005), for example, argue that the distance effect could be due to lexical factors of how we have acquired number words. My argumentation in this paper, however, does not rely on the particular explanation of the distance effect.

solucionadores de problemas (pelo menos aqueles com uma formação cultural comum) e não podem ser descartados como lidando com desempenho. Quando se trata de modelar computacionalmente a competência matemática, precisamos identificar estes últimos processos antes de podermos saber o que estamos modelando.

Exemplo 3: O efeito Distância

Acima argumentei que diagramas e outros métodos heurísticos e didáticos contribuem para algoritmos humanamente ótimos. Mas também existem maneiras pelas quais usamos algoritmos abaixo do ideal que, na verdade, tornam a resolução de problemas desnecessariamente difícil para nós. Os diagramas dizem respeito a um nível relativamente sofisticado de pensamento matemático, mas podemos ver algoritmos computacionalmente abaixo do ideal em uso já no nível muito básico e inconsciente de tratamento de conceitos matemáticos.

Considere a tarefa simples de determinar qual dos seguintes números é maior:

4 5

Compare isso agora com a tarefa de determinar qual destes números é maior:

4 9

Seria de esperar que a tarefa fosse tão simples que em ambos os casos demorasse igualmente tempo a obter a resposta. No entanto, os dados mostram que mesmo para indivíduos adultos, o emparelhamento (4 5) demora consideravelmente mais tempo do que (4 9). Isto é chamado de efeito distância: quanto maior a distância numérica entre os dois números, mais rápido seremos na resolução do problema. O efeito distância é normalmente explicado em termos da nossa tendência automática de processar símbolos numéricos como quantidades. Quando processamos quantidades, usamos o chamado sistema numérico aproximado, que é um sistema de estimativa que se torna menos preciso à medida que a distância entre os números diminui (e a magnitude dos números aumenta) (Dehaene 1997/2011, pp. 62–64).[20]

Curiosamente, não podemos nos livrar do efeito distância mesmo se estivermos treinados para resolver esse tipo de problema. O efeito também é notavelmente forte, como pode ser visto no exemplo a seguir. Vamos ter a mesma tarefa com estes números:

71 65

E estes números:

79 65

Agora certamente esperaríamos que a solução levasse um tempo igualmente longo. Afinal, podemos compreender claramente um algoritmo ótimo para resolver o problema, que seria primeiro comparar se há diferença no primeiro dígito e só considerar o segundo dígito se tal diferença não existir. Mas também neste caso há uma clara

20 Esta ligação entre o efeito distância e o sistema numérico aproximado é amplamente aceite, mas não universalmente. Tzelgov e Ganor-Stern (2005), por exemplo, argumentam que o efeito da distância pode ser devido a fatores lexicais de como adquirimos palavras numéricas. A minha argumentação neste artigo, contudo, não se baseia na explicação específica do efeito distância.

difference, the pair (71 65) takes more time than the pair (79 65) (Hinrichs et al. 1981; Pinel et al. 2001).

In fact the distance effect can also be detected when asked whether two numbers are the *same*. When given numbers:

2 9

We are faster in determining that the numbers are not the same as with:

2 5

This is also the case when numerals were use instead of number symbols (TWO NINE takes shorter than TWO THREE) (Dehaene and Akhavein 1995). For some reason, we cannot treat number symbols or numerals without thinking of them as quantities and reverting to the approximate number system.

The distance effect has important consequences for the topic at hand. The kind of problems we have been dealing with in this example may not be particularly interesting as mathematical problems, but they show how we automatically assign meaning to number symbols and words.[21] Sometimes this may be beneficial. For example, we may be quick to establish that $23 + 56 = 15$ is false because the approximate sum is so far away from the suggested answer. But we cannot help associating this approximate quantity with number symbols even in cases in which it only adds to the complexity of the problem.

It is not the fact that we have different connotations for different number symbols that is important here. When we think about a mathematical problem, many of the connections we make—and thus much of the neural activity in our brain—can in fact be irrelevant for solving the problem. When solving whether $27 + 38 = 65$, I might for example make notice of the fact that I was born on the twenty-seventh, which does nothing to advance the solution. Another example are people who associate numbers with colors. This digit-color synesthesia makes it harder to identify numbers when they do not correspond to the synesthetic experience (Kadosh and Henik 2006). Of course it is just such incidental connections that we try to get rid of by moving from performance to competence. While our performance in the addition of integers may involve task-unrelated associations, our competence does not. From the point of view of the computational approach, we treat integers independent of their conceptual connections. If our actual algorithms for doing addition sometimes involves detours like birth-dates and colors, this does not need to be taken into consideration.

However, the empirical data strongly suggests that *whenever* human cognizers see number symbols, they automatically process them as magnitudes, and the effect is present in everyone familiar with number symbols. With this type of general cognitive tendency, we can longer dismiss the suboptimal algorithm as dealing with performance. Instead, our cognitive competence with integers includes the distance effect, and if we want to model the cognitive capacity accurately, also the distance effect must be included in the model.

[21] Provided of course that we are familiar with the meanings of those symbols and words.

diferença, o par (71 65) leva mais tempo que o par (79 65) (Hinrichs et al. 1981; Pinel et al. 2001).

Na verdade, o efeito da distância também pode ser detectado quando questionado se dois números são iguais. Quando dados números:

2 9

Somos mais rápidos em determinar que os números não são iguais aos de:

2 5

Este também é o caso quando numerais foram usados em vez de símbolos numéricos (DOIS NOVE é menor que DOIS TRÊS) (Dehaene e Akhavein 1995). Por alguma razão, não podemos tratar símbolos numéricos ou numerais sem pensar neles como quantidades e reverter para o sistema numérico aproximado.

O efeito distância tem consequências importantes para o tema em questão. O tipo de problemas com os quais estamos lidando neste exemplo pode não ser particularmente interessante como problema matemático, mas mostra como atri buímos automaticamente significado a símbolos numéricos e palavras.[21] Às vezes, isso pode ser benéfico. Por exemplo, podemos estabelecer rapidamente que $23 + 56 = 15$ é falso porque a soma aproximada está muito distante da resposta sugerida. Mas não podemos deixar de associar esta quantidade aproximada a símbolos numéricos, mesmo nos casos em que isso apenas aumenta a complexidade do problema.

Não é o fato de termos conotações diferentes para símbolos numéricos diferentes que é importante aqui. Quando pensamos num problema matemático, muitas das conexões que fazemos – e, portanto, grande parte da atividade neural no nosso cérebro – podem, de facto, ser irrelevantes para a resolução do problema. Ao resolver se $27 + 38 = 65$, posso, por exemplo, notar o fato de que nasci no dia 27, o que não contribui em nada para avançar na solução. Outro exemplo são as pessoas que associam números a cores. Esta sinestesia dígito-cor torna mais difícil a identificação de números quando eles não correspondem à experiência sinestésica (Kadosh e Henik 2006). É claro que são precisamente essas ligações incidentais que tentamos eliminar, passando do desempenho para a competência. Embora o nosso desempenho na adição de números inteiros possa envolver associações não relacionadas com tarefas, a nossa competência não o faz. Do ponto de vista da abordagem computacional, tratamos os números inteiros independentemente de suas conexões conceituais. Se nossos algoritmos reais para fazer adições às vezes envolvem desvios como datas de nascimento e cores, isso não precisa ser levado em consideração.

No entanto, os dados empíricos sugerem fortemente que sempre que os conhecedores humanos veem símbolos numéricos, eles automaticamente os processam como magnitudes, e o efeito está presente em todas as pessoas familiarizadas com símbolos numéricos. Com esse tipo de tendência cognitiva geral, não podemos mais descartar o algoritmo subótimo como lidando com desempenho. Em vez disso, a nossa competência cognitiva com números inteiros inclui o efeito distância, e se quisermos modelar a capacidade cognitiva com precisão, também o efeito distância deve ser incluído no modelo.

21 Desde que estejamos familiarizados com o significado desses símbolos e palavras.

Certainly there is nothing in the distance effect that suggests that it could not be computationally modeled, and such models have indeed been proposed (starting from Dehaene and Changeux 1993). But the modeling of it goes against much of what we would expect from an algorithm that takes numerical distance into account. For a computer algorithm to count the distance between two numbers, we should expect the *reverse* of the distance effect: the smaller the difference, the faster it is to compute. Thus the suboptimal problem solving algorithms due to the distance effect seem to be a genuinely human (and possibly animal)[22] phenomenon whose modeling requires moving beyond the computational-level approach and including the algorithmic level of explanation.

It could of course turn out to be the case that the distance effect does not play a role in human arithmetical calculations. It could merely be "noise" that disturbs arithmetical calculations by making us process numerical symbols as magnitudes. The arithmetical calculations themselves could follow algorithms that are computationally optimal. This is a possibility that should be empirically pursued. What we do know is that even for mathematically highly educated subjects, the total cognitive process of being presented with an arithmetical problem and solving it is not algorithmically optimal. When we study the human competence in solving arithmetical problems, this needs to be taken into account.

7 How to Study Humanly Optimal Algorithms

Above we have arrived at an important conclusion: the computational complexity of a mathematical problem cannot be equated with the complexity of a *humanly* optimal algorithm for solving it. As a direct consequence, the complexity of the cognitive task of solving a mathematical problem is not necessarily the same as the computational complexity of the problem. In Sect. 2, Marr's computational level of explanation was characterized in terms of identifying a function that models a cognitive capacity in terms of its input and output. Due to the considerations above, we have established that such modeling does not always give the best characterization of the cognitive capacity in question. We should also consider the algorithms that human cognizers use to compute the values of the function.

Therefore, in the Marrian three-level distinction between explanations in cognitive science, at least *some* cognitive tasks are not best characterized purely in terms of the computational-level approach. Moreover, the entire distinction between the computational and algorithmic levels becomes problematic: without including the algorithmic level we cannot identify the phenomena we aim to model on

[22] The distance effect itself is a general characteristic of the approximate number system and it has been detected in various nonhuman animals, including rats, pigeons, dolphins and apes (Dehaene 19972011, p. 16). However, most nonhuman animals cannot grasp symbolic representations of quantities, so it is difficult to see the distance effect as causing suboptimal problem solving strategies. Recently there have been experiments on monkeys that suggest an ability to make addition of symbolically represented magnitudes (Livingstone et al. 2014). In such case, it would not be unfeasible to think that the distance effect can cause the monkey to use a suboptimal problem solving algorithm.

Certamente não há nada no efeito da distância que sugira que ele não possa ser modelado computacionalmente, e tais modelos foram de fato propostos (a partir de Dehaene e Changeux 1993). Mas a modelagem vai contra muito do que esperaríamos de um algoritmo que leva em conta a distância numérica. Para que um algoritmo de computador conte a distância entre dois números, deveríamos esperar o inverso do efeito da distância: quanto menor a diferença, mais rápido será o cálculo. Assim, os algoritmos de resolução de problemas abaixo do ideal devido ao efeito da distância parecem ser um fenômeno genuinamente humano (e possivelmente animal)[22] cuja modelagem requer ir além da abordagem de nível computacional e incluir o nível algorítmico de explicação.

É claro que poderia acontecer que o efeito da distância não desempenhasse um papel nos cálculos aritméticos humanos. Poderia ser apenas um "ruído" que perturba os cálculos aritméticos, fazendo-nos processar símbolos numéricos como magnitudes. Os próprios cálculos aritméticos poderiam seguir algoritmos que sejam computacionalmente ótimos. Esta é uma possibilidade que deve ser investigada empiricamente. O que sabemos é que mesmo para indivíduos com elevado nível de formação matemática, o processo cognitivo total de ser apresentado a um problema aritmético e de o resolver não é algoritmicamente óptimo. Quando estudamos a competência humana na resolução de problemas aritméticos, isso precisa ser levado e m consideração.

7 Como estudar algoritmos humanamente ideais

Acima chegamos a uma conclusão importante: a complexidade computacional de um problema matemático não pode ser equiparada à complexidade de um algoritmo humanamente ideal para resolvê-lo. Como consequência direta, a complexidade da tarefa cognitiva de resolver um problema matemático não é necessariamente a mesma que a complexidade computacional do problema. Na Seita. 2, o nível computacional de explicação de Marr foi caracterizado em termos de identificação de uma função que modela uma capacidade cognitiva em termos de sua entrada e saída. Pelas considerações acima, estabelecemos que tal modelagem nem sempre dá a melhor caracterização da capacidade cognitiva em questão. Devemos também considerar os algoritmos que os conhecedores humanos usam para calcular os valores da função.

Portanto, na distinção marriana de três níveis entre explicações na ciência cognitiva, pelo menos algumas tarefas cognitivas não são melhor caracterizadas puramente em termos da abordagem de nível computacional. Além disso, toda a distinção entre os níveis computacional e algorítmico torna-se problemática: sem incluir o nível algorítmico não podemos identificar os fenómenos que pretendemos modelar.

22 O próprio efeito da distância é uma característica geral do sistema numérico aproximado e foi detectado em vários animais não humanos, incluindo ratos, pombos, golfinhos e macacos (Dehaene 1997-2011, p. 16). No entanto, a maioria dos animais não humanos não consegue compreender representações simbólicas de quantidades, por isso é difícil ver o efeito da distância como causa de estratégias de resolução de problemas abaixo do ideal. Recentemente, houve experiências em macacos que sugerem uma capacidade de adicionar magnitudes representadas simbolicamente (Livingstone et al. 2014). Nesse caso, não seria inviável pensar que o efeito da distância pode fazer com que o macaco utilize um algoritmo de resolução de problemas abaixo do ideal.

the computational level. In principle, there is of course nothing to prevent determining the complexity of the algorithms that human beings use in their cognitive tasks purely computationally. However, with too much focus on the computational approach, how can we find out what the human algorithms are? If the algorithms that most accurately model the problem solving strategies of human cognizers are not necessarily computationally optimal ones, we should already include considerations on algorithms—and possibly also their implementations—in the initial research question.

In fact this approach can already be seen in the actual computational modeling of human cognitive capacities. Sun (2008), for example, has noted that the distinction between Marr's three levels is relatively insignificant in modern computational modeling:

> One cannot specify algorithms without at least some considerations of possible implementations, and what is to be considered "computation" (i.e., what can be computed) relies on algorithms, especially the notion of algorithmic complexity, and so on. Therefore, one often has to consider computation, algorithms, and implementation together somehow (especially in relation to cognition). (Sun 2008, p. 15)

This developing modern paradigm fits well with the theoretical considerations of this paper. As we have seen, the computational approach can be needlessly limiting and may preclude finding important data about the algorithms used by human cognizers. Without including the algorithmic level, we may not be able to identify the cognitive phenomena we want to model in the first place. This way, dismissing the algorithmic level leads to problems also on the computational level.[23]

However, acknowledging the need for a wider methodological paradigm is only the first step. As important as expanding the study of cognitive complexity to include algorithmic aspects is, there are many understandable difficulties involved. Most importantly, the characteristics of advanced mathematical cognition can be very difficult to study. This is the case especially on the level of implementation, but also in terms of identifying the algorithms that humans use in problem solving (see, e.g., Tall 2002; Dowker 2016). Perhaps in their minds the test subjects make cognitive moves that do not show up as behavioral patterns, nor are the test subjects able to identify them introspectively (Peters and Bjalkebring 2015). The design of pertinent experiments is an enormous challenge and even with ideal circumstances for testing we cannot be entirely sure that we have accurately identified the algorithmic procedures. While that does not mean that we cannot gather reliable data at all, it does bring in the kind of inexactness that is a bad fit with the exact notions involved in the study of computational complexity (see, e.g., Carruthers and Stege 2013).

These practical difficulties of studying mathematical cognition should not be downplayed, yet at the same time they should not be thought of as an argument against engaging in the algorithmic level of explanation. From the perspective of

[23] In this, the present approach is similar to that of Varma (2014) who argues that restricting focus to one of Marr's levels of explanation can be detrimental for progress in cognitive science.

o nível computacional. Em princípio, é claro que não há nada que impeça a determinação da complexidade dos algoritmos que os seres humanos utilizam nas suas tarefas cognitivas de forma puramente computacional. Porém, com muito foco na abordagem computacional, como podemos descobrir o que são os algoritmos hu manos? Se os algoritmos que modelam com mais precisão as estratégias de resolução de problemas dos conhecedores humanos não são necessariamente aqueles computacionalmente óptimos, já deveríamos incluir considerações sobre algoritmos – e possivelmente também as suas implementações – na questão de investigação inicial.

Na verdade, esta abordagem já pode ser vista na modelagem computacional real das capacidades cognitivas humanas. Sun (2008), por exemplo, observou que a disti nção entre os três níveis de Marr é relativamente insignificante na modelagem computacional moderna:

> Não se pode especificar algoritmos sem pelo menos algumas considerações sobre possíveis implementações, e o que deve ser considerado "computação" (ou seja, o que pode ser computado) depende de algoritmos, especialmente a noção de complexidade algorítmica, e assim por diante. Portanto, muitas vezes é preciso considerar a computação, os algoritmos e a implementação juntos de alguma forma (especialmente em relação à cognição). (Dom de 2008, p. 15)

Este paradigma moderno em desenvolvimento se ajusta bem às considerações teóricas deste artigo. Como vimos, a abordagem computacional pode ser desnecessariamente limitante e pode impedir a descoberta de dados importantes sobre os algoritmos usados pelos conhecedores humanos. Sem incluir o nível algorítmico, talvez não consigamos identificar os fenómenos cognitivos que queremos modelar em primeiro lugar. Dessa forma, descartar o nível algorítmico leva a problemas também no nível computacional.[23]

No entanto, reconhecer a necessidade de um paradigma metodológico mais amplo é apenas o primeiro passo. Por mais importante que seja expandir o estudo da complexidade cognitiva para incluir aspectos algorítmicos, há muitas dificuldades compreensíveis envolvidas. Mais importante ainda, as características da cognição matemática avançada podem ser muito difíceis de estudar. Este é o caso especialmente ao nível da implementação, mas também em termos de identificação dos algoritmos que os humanos utilizam na resolução de problemas (ver, por exemplo, Tall 2002; Dowker 2016). Talvez, nas suas mentes, os sujeitos do teste façam movimentos cognitivos que não aparecem como padrões comportamentais, nem os sujeitos do teste sejam capazes de identificá-los introspectivamente (Peters e Bjalkebring 2015). O desenho de experimentos pertinentes é um enorme desafio e mesmo com circunstâncias ideais para testes não podemos ter certeza absoluta de que identifica mos com precisão os procedimentos algorítmicos. Embora isso não signifique que não possamos reunir dados confiáveis, traz consigo o tipo de inexatidão que não se ajusta às noções exatas envolvidas no estudo da complexidade computacional (ver, por exemplo, Carruthers e Stege 2013).

Estas dificuldades práticas de estudar a cognição matemática não devem ser subestimadas, mas ao mesmo tempo não devem ser pensadas como um argumento contra o envolvimento no nível algorítmico de explicação. Da perspectiva de

23 Nisso, a presente abordagem é semelhante à de Varma (2014), que argumenta que restringir o foco a um dos níveis de explicação de Marr pode ser prejudicial para o progresso na ciência cognitiva.

computational complexity, it may be asked why we should be interested in modeling suboptimal algorithms in the first place, whether they concern diagrams and other didactic and heuristic tools, or unconscious processes like those underlying the distance effect. After all, the computational complexity approach has an established fruitful methodology, and including algorithmic considerations only makes the problems more complicated and difficult to study. However, the fruitfulness and established methodology of the computational complexity approach should not lead to a streetlight effect in which the ease of explanation becomes a determining factor in establishing the research paradigm.[24]

However, aside from such practical worries, there is also a potentially important theoretical problem in algorithmic-level explanations. In the previous section, I proposed that we can consider an algorithm humanly optimal if it is generally beneficial for human problem solvers. But as was also mentioned in that section, there is no reason to believe that there are unique humanly optimal algorithms for mathematical problems. Indeed, since mathematical practices (including symbols, tools, etc.) vary in different cultures, it seems clear that we should be looking for optimal algorithms in culturally specific contexts. For example, let us consider the "Japanese" visual multiplication method (Fig. 6), in which orthogonal lines are drawn for ones, tens and hundreds, etc. and the result is reached by counting the intersections of the lines.[25]

Whereas that method may be heuristically less complex for children acquainted with it—at least with numbers where the digits are small like in Fig. 6—for children not acquainted with such visual methods this is usually not the case. In addition to such cultural differences, there are also individual differences in how helpful we find different heuristic methods. More visually inclined students, for example, are likely to find diagrams and other visual methods more helpful, and students with diverse learning abilities generally benefit from different heuristic and didactic tools (van Garderen et al. 2013).[26]

In this way, many mathematical practices are *enculturated*. Enculturation refers to the transformative process in which interactions with the surrounding culture influence the way cognitive practices develop (Menary 2015; Fabry 2018). Through the mechanism Menary (2014) calls "learning driven plasticity", new cognitive capacities can be acquired due to the neural plasticity of the human brain, which allows for both structural and functional changes (Dehaene 2009; Ansari 2012; Anderson 2014). In learning mathematics, the methods and practices of the surrounding culture thus play an important role in how the cognitive mathematical capacities

[24] The name "streetlight effect" comes from the old joke in which a drunk looks for his keys in the better visibility of a streetlight, instead of the place where he lost them.

[25] The origin of the name for this method, which has recently received a lot of attention on the Internet, is unknown, but most often it is referred to as the Japanese multiplication method. This use also seems to adopted in academic contexts. See, e.g., http://www.cemc.uwaterloo.ca/events/mathcircles/2013-14/Fall/Junior6_Multiplication_Nov5.pdf.

[26] The usefulness of visual heuristic methods can also be present without actual visual input, as seen in the "mental abacus" used in many especially Asian cultures for calculations (Frank and Barner 2012).

complexidade computacional, pode-se perguntar por que deveríamos estar interessados em modelar algoritmos subótimos em primeiro lugar, quer se trate de diagramas e outras ferramentas didáticas e heurísticas, quer de processos inconscientes como aqueles subjacentes ao efeito distância. Afinal, a abordagem da complexidade computacional tem uma metodologia frutífera estabelecida, e incluir considerações algorítmicas apenas torna os problemas mais complicados e difíceis de estudar. No entanto, a fecundidade e a metodologia estabelecida da abordagem da complexidade computacional não devem conduzir a um efeito de iluminação pública em que a facilidade de explicação se torne um factor determinante no estabelecimento do paradigma de investigação.[24]

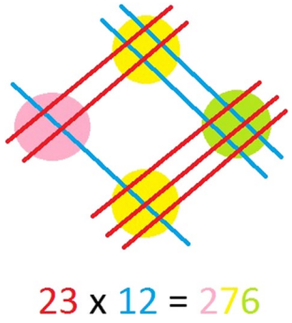
No entanto, além dessas preocupações práticas, há também um problema teórico potencialmente importante nas explicações em nível algorítmico. Na seção anterior, propus que podemos considerar um algoritmo humanamente ótimo se for geralmente benéfico para solucionadores de problemas humanos. Mas, como também foi mencionado naquela seção, não há razão para acreditar que existam algoritmos humanos ótimos únicos para problemas matemáticos. Na verdade, uma vez que as práticas matemáticas (incluindo símbolos, ferramentas, etc.) variam em diferentes culturas, parece claro que deveríamos procurar algoritmos óptimos em contextos culturalmente específicos. Por exemplo, consideremos o método de multiplicação visual "japonês" (Fig. 6), no qual linhas ortogonais são desenhadas para unidades, dezenas e centenas, etc. e o resultado é alcançado contando as interseções das linhas.[25]

Embora esse método possa ser heuristicamente menos complexo para crianças familiarizadas com ele – pelo menos com números onde os dígitos são pequenos como na Fig. 6 – para crianças não familiarizadas com tais métodos visuais este nor malmente não é o caso. Além dessas diferenças culturais, há também diferenças individuais na forma como consideramos os diferentes métodos heurísticos úteis. Alunos com maior inclinação visual, por exemplo, provavelmente acharão diagramas e outros métodos visuais mais úteis, e alunos com diversas habilidades de aprendizagem geralmente se beneficiam de diferentes ferramentas heurísticas e didáticas (van Garderen et al. 2013).[26]

Dessa forma, muitas práticas matemáticas são enculturadas. A enculturação refere-se ao processo transformativo no qual as interações com a cultura circundante influenciam a forma como as práticas cognitivas se desenvolvem (Menary 2015; Fabry 2018). Através do mecanismo que Menary (2014) chama de "plasticidade impulsionada pela aprendizagem", novas capacidades cognitivas podem ser adquiridas devido à plasticidade neural do cérebro humano, que permite mudanças estruturais e funcionais (Dehaene 2009; Ansari 2012; Anderson 2014). Na aprendizagem da matemática, os métodos e práticas da cultura circundante desempenham, portanto, um papel importante na forma como as capacidades matemáticas cognitivas se desenvolvem.

24 O nome "efeito de iluminação pública" vem da velha piada em que um bêbado procura suas chaves na melhor visibilidade de um poste de luz, e não no local onde as perdeu. 25 A origem do nome deste método, que recentemente recebeu muita atenção na Internet, é desconhecida, mas na maioria das vezes é referido como método de multiplicação japonês. Este uso também parece ser adotado em contextos acadêmicos. Veja, por exemplo, http://www.cemc.uwate rloo.ca/event s/mathc ircle s/2013-14/Fall/ Junio r6_Multi plica tion_ Nov5.pdf. 26 A utilidade dos métodos heurísticos visuais também pode estar presente sem a entrada visual real, como pode ser visto no "ábaco mental" usado em muitas culturas especialmente asiáticas para cálculos (Frank e Barner 2012).

Fig. 6 The "Japanese" visual multiplication method



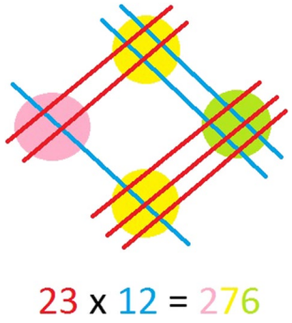
develop. This theoretical framework can help explain the different methods that human problem solvers use (Pantsar 2018, 2019).

The computational complexity approach to mathematical problem solving, and computational-level explanation based on it, is unable to account for the enculturated nature of problem solving processes. This is where the present approach has an advantage. By including the algorithmic level in the explanations, we can study which problem solving methods are cognitively beneficial for human problem solvers within a particular culture. We can also study how these methods vary interculturally. Perhaps for many mathematical problems there is relatively little cultural variation. With some problems, it may make sense to speak of universal, humanly optimal problem solving algorithms. In some cases, these may also coincide with computational optimal algorithms. However, we should not assume that this is generally the case. That is why humanly optimal algorithms must be considered conceptually separate from computationally optimal algorithms. For the same reason, cognitive complexity should be considered to be conceptually separate from computational complexity. Because of the expected cultural variation, humanly optimal algorithms—and hence cognitive complexity of mathematical problem solving—should be considered in cultural contexts. With these conceptual differences in place, our approach can account for the enculturated nature of mathematical practices.[27]

At this point, one might ask whether this cultural and individual variation is not about *performance* while we are interested in *competence*? But based on all the considerations we have been through in this paper, it is impossible to retain this distinction in the sense that there is *one* human competence in mathematical problem solving. Mathematical problem solving is an ability with great cultural and individual variation and a single notion of mathematical competence seems thus misplaced. The distinction between performance and competence is important to make, but instead of having a uniform notion of an optimal algorithm (and one corresponding notion of competence) we should look for general patterns in performance in order to establish a wider, more fine-grained conception of *enculturated competence*. With

[27] See Fabry and Pantsar (2019) for a detailed treatment of this topic.

Fig. 6 O método de multiplicação visual "japonês"



desenvolver. Este quadro teórico pode ajudar a explicar os diferentes métodos que os solucionadores de problemas humanos utilizam (Pantsar 2018, 2019).

A abordagem da complexidade computacional para a resolução de problemas matemáticos, e a explicação em nível computacional baseada nela, é incapaz de dar conta da natureza cultuada dos processos de resolução de problemas. É aqui que a abordagem actual tem uma vantagem. Ao incluir o nível algorítmico nas explicações, podemos estudar quais métodos de resolução de problemas são cognitivamente benéficos para solucionadores de problemas humanos dentro de uma cultura particular. Também podemos estudar como esses métodos variam interculturalmente. Talvez para muitos problemas matemáticos haja relativamente pouca variação cultural. Com alguns problemas, pode fazer sentido falar de algoritmos de resolução de problemas universais e humanamente ótimos. Em alguns casos, estes também podem coincidir com algoritmos computacionais ótimos. Contudo, não devemos presumir que este seja geralmente o caso. É por isso que algoritmos humanamente ótimos devem ser considerados conceitualmente separados de algoritmos computacionalmente ótimos. Pela mesma razão, a complexidade cognitiva deve ser considerada conceitualmente separada da complexidade computacional. Devido à variação cultural esperada, os algoritmos humanamente óptimos – e, portanto, a complexidade cognitiva da resolução de problemas matemáticos – devem ser considerados em contextos culturais. Com essas diferenças conceituais estabelecidas, nossa abordagem pode explicar a natureza enculturada das práticas matemáticas.[27]

Neste ponto, pode-se perguntar se esta variação cultural e individual não tem a ver com desempenho enquanto estamos interessados em competência. Mas com base em todas as considerações que fizemos neste artigo, é impossível manter esta distinção no sentido de que existe uma competência humana na resolução de problemas matemáticos. A resolução de problemas matemáticos é uma capacidade com grande variação cultural e individual e uma noção única de competência matemática parece, portanto, equivocada. É importante fazer a distinção entre desempenho e competência, mas em vez de termos uma noção uniforme de um algoritmo óptimo (e uma noção correspondente de competência) deveríamos procurar padrões gerais de desempenho, a fim de estabelecer uma concepção mais ampla e mais refinada de competência enculturada. Com

27 Ver Fabry e Pantsar (2019) para um tratamento detalhado deste tópico.

the focus on enculturated competence, we can have a theoretical framework that is more sensitive to particular culturally determined mathematical practices, such as constructing and interpreting diagrams.

In studying enculturated mathematical competence, it is clear that we should use as much information about the computational complexity of mathematical problems as we can. While the cultural and individual differences are important, much of the complexity of the cognitive task of solving a mathematical problem can indeed be traced to the computational complexity of that problem. In this way, the approach here is not meant to question the value of the established fruitful methods of studying complexity. Instead, I want to suggest that the established methodology can be augmented by including considerations of the actual, sometimes suboptimal, enculturated algorithms that human problem solvers use.

It should also be noted that mathematical competence is a much wider phenomenon than simple problem solving. Throughout this paper the focus has been on problem solving, and even that only in a very limited paradigm set by Turing machines, i.e., step-by-step algorithmic solutions. Actual mathematical problem solving is of course a much more complicated phenomenon in which there are many important socially and culturally determined aspects, such as clarity of expressions and proof structures. In addition, there are numerous epistemic considerations involved in mathematical cognition, whereas in this paper we have only focused on problem solving. From verifying the solutions to communicating and explaining them, enculturated mathematical competence includes various dimensions which have not been analyzed here. This is not an oversight, as I believe that research of mathematical practice is crucial in determining what enculturated mathematical competences are on higher levels of mathematics. Rather than dismissing the importance of such considerations, my purpose in this paper has been to show that we need a notion of humanly optimal algorithms and enculturated competence already when we seek to explain mathematical cognition in the limited paradigm of problem solving algorithms. In order for us to be able to study mathematics as a human phenomenon in all its richness, we may need to expand the paradigm even more.

One important aspect of developing this notion of enculturated competence is that we should also consider the question of human-computer interaction in mathematical problem solving. To present just one example, as computer-assisted problem solving becomes more prevalent, it makes sense to move the focus to the cognitive and computational complexity involved in the computer-assisted process. It is easy to predict that if mathematics relies increasingly on problem solving (or solution-checking) computer programs, many of the human heuristic and didactic tools, such as diagrams, may no longer play the same kind of role as they currently do. However, in this scenario we get new interesting questions concerning cognitive complexity, including considerations on the understandability of computer programs. Increasing human-computer interaction will undoubtedly change the way mathematics is practiced and learned, but it seems unrealistic to assume that the heuristic and didactic elements important for human understanding will disappear. In this way, although the study of cognitive complexity may evolve into new directions, there is no reason to believe that the cognitive complexity of human mathematical problem solving will become completely reducible to the computational complexity of the

Com o foco na competência enculturada, podemos ter um quadro teórico que é mais sensível a práticas matemáticas específicas culturalmente determinadas, como a construção e interpretação de diagramas.

Ao estudar a competência matemática enculturada, fica claro que devemos usar o máximo de informação possível sobre a complexidade computacional dos problemas matemáticos. Embora as diferenças culturais e individuais sejam importantes, grande parte da complexidade da tarefa cognitiva de resolver um problema matemático pode, de facto, ser atribuída à complexidade computacional desse problema. Desta forma, a abordagem aqui não pretende questionar o valor dos métodos frutíferos estabelecidos para estudar a complexidade. Em vez disso, quero sugerir que a metodologia estabelecida pode ser ampliada pela inclusão de considerações sobre os algoritmos enculturados reais, às vezes subótimos, que os solucionadores de problemas humanos usam.

Deve-se também notar que a competência matemática é um fenómeno muito mais amplo do que a simples resolução de problemas. Ao longo deste artigo, o foco tem sido a resolução de problemas, e mesmo isso apenas num paradigma muito limitado definido pelas máquinas de Turing, ou seja, soluções algorítmicas passo a passo. A resolução real de problemas matemáticos é, obviamente, um fenómeno muito mais complicado, no qual existem muitos aspectos importantes social e culturalmente determinados, tais como clareza de expressões e estruturas de prova. Além disso, existem inúmeras considerações epistêmicas envolvidas na cognição matemática, ao passo que neste artigo nos concentramos apenas na resolução de problemas. Desde a verificação das soluções até à sua comunicação e explicação, a competência matemática cultivada i nclui diversas dimensões que não foram aqui analisadas. Isto não é um descuido, pois acredito que a investigação da prática matemática é crucial para determinar quais são as competências matemáticas enculturadas nos níveis mais elevados da matemática. Em vez de descartar a importância de tais considerações, o meu objectivo neste artigo foi mostrar que precisamos de uma noção de algoritmos humanamente óptimos e de competência enculturada já quando procuramos explicar a cognição matemática no paradigma limitado dos algoritmos de resolução de problemas. Para que possamos estudar a matemática como um fenómeno humano em toda a sua riqueza, talvez seja necessário expandir ainda mais o paradigma.

Um aspecto importante do desenvolvimento desta noção de competência enculturada é que deveríamos também considerar a questão da interacção humano-computador na resolução de problemas matemáticos. Para apresentar apenas um exemplo, à medida que a resolução de problemas assistida por computador se torna mais predominante, faz sentido mudar o foco para a complexidade cognitiva e computacional envolvida no processo assistido por computador. É fácil prever que, se a matemática depender cada vez mais de programas de computador para a resolução de problemas (ou verificação de soluções), muitas das ferramentas heurísticas e didáticas humanas, como os diagramas, poderão já não desempenhar o mesmo tipo de papel que desempenham atualmente. Contudo, neste cenário surgem novas questões i nteressantes relativas à complexidade cognitiva, incluindo considerações sobre a compreensibilidade de programas de computador. O aumento da interacção humano-computador irá, sem dúvida, mudar a forma como a matemática é praticada e aprendida, mas parece irrealista assumir que os elementos heurísticos e didácticos importantes para a compreensão humana irão desaparecer. Desta forma, embora o estudo da complexidade cognitiva possa evoluir para novas direções, não há razão para acreditar que a complexidade cognitiva da resolução de problemas matemáticos humanos se tornará completamente redutível à complexidade computacional do

problems. Instead, what will emerge is a new enculturated notion of mathematical competence, one involving algorithms with their own particular characteristics. In that scenario, it is important to have a research paradigm that does not focus only on computational complexity and the computational level of explanation.

Acknowledgements Open access funding provided by University of Helsinki including Helsinki University Central Hospital. Funding was provided by Academy of Finland. I would like to thank Regina Fabry, Cyrille Imbert and Stefan Buijsman for their very helpful comments on earlier versions of this manuscript. I would also like to thank Gabriel Sandu for making this work possible as part of his project "Dependence and Independence in Logic" at the University of Helsinki.

Open Access This article is distributed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license, and indicate if changes were made.

References

Aaronson, S. (2012). Why philosophers and cognitive scientists should care about computational complexity. In J. Copeland, et al. (Eds.), *Computability: Gödel, Turing, Church, and beyond*. Cambridge: MIT Press.

Anderson, J. R. (1990). *The adaptive character of thought*. Hillsdale, NJ: Lawrence Erlbaum Associates.

Anderson, M. (2014). *After phrenology: Neural reuse and the interactive brain*. Cambridge, MA: MIT Press.

Ansari, D. (2012). Culture and education: New frontiers in brain plasticity. *Trends in Cognitive Sciences*, *16*(2), 93–95.

Arora, S., & Barak, B. (2007). *Computational complexity. A modern approach*. Cambridge: Cambridge University Press.

Ashcraft, M. H. (1992). Cognitive arithmetic: A review of data and theory. *Cognition*, *44*(1–2), 75–106.

Ashcraft, M. H. (1995). Cognitive psychology and simple arithmetic: A review and summary of new directions. *Mathematical Cognition*, *1*(1), 3–34.

Ausiello, G., Crescenzi, P., Gambosi, G., Kann, V., Marchetti-Spaccamela, A., & Protasi, M. (1999). *Complexity and approximation: Combinatorial optimization problems and their approximability properties*. Berlin: Springer.

Bieri, J. (1955). Cognitive complexity–simplicity and predictive behavior. *Journal of Abnormal and Social Psychology*, *51*, 263–268.

Blum, M. (1967). A machine-independent theory of the complexity of recursive functions. *Journal of the ACM (JACM)*, *14*(2), 322–336.

Carruthers, S., & Stege, U. (2013). On evaluating human problem solving of computationally hard problems. *Journal of Problem Solving*, *5*(2), 42–70.

Chater, N., Tenenbaum, J. B., & Yuille, A. (2006). Probabilistic models of cognition: Conceptual foundations. *Trends in Cognitive Sciences*, *10*(7), 287–291.

Chomsky, N. (1965). *Aspects of the theory of syntax*. Cambridge, MA: MIT Press.

Chor, B., & Goldreich, O. (1990). An improved parallel algorithm for integer GCD. *Algorithmica*, *5*(1–4), 1–10.

Church, A. (1936). An unsolvable problem of elementary number theory. *American Journal of Mathematics*, *58*(58), 345–363.

Cobham, A. (1964). The intrinsic computational difficulty of functions. In *Proceedings of the 1964 congress on logic, mathematics and the methodology of science* (pp. 24–30).

Cummins, R. (2000). How to solve it. How does it work? vs. What are the laws? Two conceptions of psychological explanation. In F. Keil & R. Wilson (Eds.), *Explanation and cognition* (pp. 117–145). Cambridge, MA: MIT Press.

Dehaene, S. (1997/2011). *The number sense: How the mind creates mathematics*, 2nd edn. 2011. New York: Oxford University Press.

problemas. Em vez disso, o que irá emergir é uma nova noção enculturada de competência matemática, envolvendo algoritmos com características próprias e particulares. Nesse cenário, é importante ter um paradigma de pesquisa que não se concentre apenas na complexidade computacional e no nível computacional de explicação.

Agradecimentos Financiamento de acesso aberto fornecido pela Universidade de Helsinque, incluindo o Hospital Central da Universidade de Helsinque. O financiamento foi fornecido pela Academia da Finlândia. Gostaria de agradecer a Regina Fabry, Cyrille Imbert e Stefan Buijsman pelos seus comentários muito úteis em versões anteriores deste manuscrito. Gostaria também de agradecer a Gabriel Sandu por tornar este trabalho possível como parte do seu projeto "Dependência e Independência na Lógica" na Universidade de Helsinque.

Acesso aberto Este artigo é distribuído sob os termos da Licença Internacional Creative Commons Attribution 4.0 (http://creat iveco mmons .org/licen ses/by/4.0/), que permite uso, distribuição e reprodução irrestritos em qualquer meio, desde que você dê crédito apropriado ao(s) autor(es) original(is) e à fonte, forneça um link para a licença Creative Commons e indique se alterações foram feitas.

Referências

Aaronson, S. (2012). Por que filósofos e cientistas cognitivos deveriam se preocupar com a complexidade computacional. Em J. Copeland, et al. (Eds.), Computabilidade: Gödel, Turing, Church e além. Cambridge: MIT Press. Anderson, JR (1990). O caráter adaptativo do pensamento. Hillsdale, NJ: Lawrence Erlbaum Associates. Anderson, M. (2014). Depois da frenologia: Reutilização neural e o cérebro interativo. Cambridge, MA: MIT Press. Ansari, D. (2012). Cultura e educação: Novas fronteiras na plasticidade cerebral. Tendências em Ciências Cognitivas, 16(2), 93–95. Arora, S. e Barak, B. (2007). Complexidade computacional. Uma abordagem moderna. Cambridge: Cambridge University Press. Ashcraft, MH (1992). Aritmética cognitiva: uma revisão de dados e teoria. Cognição, 44(1–2), 75–106. Ashcraft, MH (1995). Psicologia cognitiva e aritmética simples: uma revisão e um resumo de novos rumos. Cognição Matemática, 1(1), 3–34. Ausiello, G., Crescenzi, P., Gambosi, G., Kann, V., Marchetti-Spaccamela, A., & Protasi, M. (1999). Complexidade e aproximação: Problemas de otimização combinatória e suas propriedades de aproximabilidade. Berlim: Springer. Bieri, J. (1955). Complexidade cognitiva – simplicidade e comportamento preditivo. Jornal de Psicologia Anormal e Social, 51, 263–268. Blu m, M. (1967). Uma teoria independente da máquina sobre a complexidade das funções recursivas. Jornal do ACM (JACM), 14(2), 322–336. Carruthers, S. e Stege, U. (2013). Na avaliação da resolução humana de problemas computacionalmente difíceis. Jornal de Solução de Problemas, 5(2), 42–70. Chater, N., Tenenbaum, JB e Yuille, A. (2006). Modelos probabilísticos de cognição: Fundamentos conceituais. Tendências em Ciências Cognitivas, 10(7), 287–291. Chomsky, N. (1965). Aspectos da teoria da sintaxe. Cambridge, MA: MIT Press. Chor, B. e Goldreich, O. (1990). Um algoritmo paralelo aprimorado para GCD inteiro. Algorítmica, 5(1– 4), 1–10. Igreja, A. (1936). Um problema insolúvel da teoria elementar dos números. American Journal of Mathematics, 58(58), 345–363. Cobham, A. (1964). A dificuldade computacional intrínseca das funções. Em Anais do congresso de 1964 sobre lógica, matemática e metodologia da ciência (pp. 24–30). Cummins, R. (2000). Como resolver isso. Como funciona? vs. Quais são as leis? Duas concepções de explicação psicológica. Em F. Keil & R. Wilson (Eds.), Explicação e cognição (pp. 117–145). Cambridge, MA: MIT Press. Dehaene, S. (1997/2011). O sentido numérico: como a mente cria a matemática, 2ª ed. 2011. Nova York: Oxford University Press.

Dehaene, S. (2009). *Reading in the brain: The new science of how we read*. London: Penguin.
Dehaene, S., & Akhavein, R. (1995). Attention, automaticity, and levels of representation in number processing. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, *21*(2), 314.
Dehaene, S., & Changeux, J. P. (1993). Development of elementary numerical abilities: A neuronal model. *Journal of Cognitive Neuroscience*, *5*(4), 390–407.
Diezmann, C. M., & English, L. D. (2001). Promoting the use of diagrams as tools for thinking. In 2001 National Council of Teachers of Mathematics Yearbook: The role of representation in school mathematics (pp. 77–89). National Council of Teachers of Mathematics.
Dowker, A. (2016). Factors that influence improvement in numeracy, reading, and comprehension in the context of a numeracy intervention. *Frontiers in Psychology*, *7*, 1929.
Downey, R. G., & Fellows, M. R. (1999). *Parameterized complexity*. New York: Springer.
Dry, M., Preiss, K., & Wagemans, J. (2012). Clustering, randomness, and regularity: Spatial distributions and human performance on the traveling salesperson problem and minimum spanning tree problem. *The Journal of Problem Solving*, *4*(1), 2.
Edmonds, J. (1965). Paths, trees, and flowers. *Canadian Journal of Mathematics*, *17*(3), 449–467.
Fabry, R. E. (2018). Betwixt and between: The enculturated predictive processing approach to cognition. *Synthese*, *195*, 1–36.
Fabry, R. E. (2019). The cerebral, extra-cerebral bodily, and socio-cultural dimensions of enculturated arithmetical cognition. *Synthese*. https://doi.org/10.1007/s11229-019-02238-1.
Fabry, R. E. & Pantsar, M. (2019). A fresh look at research strategies in computational cognitive science: The case of enculturated mathematical problem solving. *Synthese*. https://doi.org/10.1007/s11229-019-02276-9.
Fraenkel, A. S., & Lichtenstein, D. (1981). Computing a perfect strategy for $\text{ n }\times \text{ n }$ chess requires time exponential in n. *Journal of Combinatorial Theory, Series A*, *31*(2), 199–214.
Frank, M., & Barner, D. (2012). Representing exact number visually using mental abacus. *Journal of Experimental Psychology*, *141*(1), 134–149.
Frixione, M. (2001). Tractable competence. *Minds and Machines*, *11*(3), 379–397.
Fuson, K. C. (1992). Research on learning and teaching addition and subtraction of whole numbers. In G. Leinhardt, R. T. Putnam, & R. A. Hattrup (Eds.), *Analysis of arithmetic for mathematics teaching* (pp. 53–187). Hillsdale, NJ, US: Lawrence Erlbaum Associates.
Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. San Francisco: W.H. Freeman.
Goldreich, O. (2008). Computational complexity: A conceptual perspective. *ACM Sigact News*, *39*(3), 35–39.
Graham, S. M., Joshi, A., & Pizlo, Z. (2000). The traveling salesman problem: A hierarchical model. *Memory & Cognition*, *28*(7), 1191–1204.
Hadamard, J. (1954). *The psychology of invention in the mathematical field*. Mineola, NY: Dover Publications.
Hinrichs, J. V., Yurko, D. S., & Hu, J. M. (1981). Two-digit number comparison: Use of place information. *Journal of Experimental Psychology: Human Perception and Performance*, *7*(4), 890.
Horgan, T., & Tienson, J. (1996). *Connectionism and the philosophy of psychology*. Cambridge, MA: MIT Press.
Ifrah, G. (1998). *The universal history of numbers: From prehistory to the invention of the computer,* Translated by Bellos, D., Harding, E. F., Wood, S., & Monk, I. London: Harville Press.
Isaac, A. M., Szymanik, J., & Verbrugge, R. (2014). Logic and complexity in cognitive science. In S. Smets & A. Baltag (Eds.), *Johan van Benthem on logic and information dynamics* (pp. 787–824). Berlin: Springer.
Iwata, S., & Kasai, T. (1994). The Othello game on an $\text{ n }\times \text{ n }$ board is PSPACE-complete. *Theoretical Computer Science*, *123*(2), 329–340.
Jamnik, M. (2001). *Mathematical reasoning with diagrams*. Chicago: University of Chicago Press.
Kadosh, R. C., & Henik, A. (2006). Color congruity effect: Where do colors and numbers interact in synesthesia? *Cortex*, *42*(2), 259–263.
Karatsuba, A., & Ofman, Y. (1962). Multiplication of many-digital numbers by automatic computers. *Physics-Doklady*, *7*(1963), 595–596.
LeFevre, J., DeStefano, D., Coleman, B., & Shanahan, T. (2005). Mathematical cognition and working memory. In J. Campbell (Ed.), *Handbook of mathematical cognition* (pp. 361–378). New York: Psychology Press.

Dehaene, S. (2009). Lendo no cérebro: a nova ciência de como lemos. Londres: Pinguim. Dehaene, S. e Akhavein, R. (1995). Atenção, automaticidade e níveis de representação no processamento de números. J ournal of Experimental Psychology: Aprendizagem, Memória e Cognição, 21(2), 314. Dehaene, S., & Changeux, J. P. (1993). Desenvolvimento de habilidades numéricas elementares: Um modelo neuronal. Jornal de Neurociência Cognitiva, 5(4), 390–407. Diezmann, CM e English, LD (2001). Promover o uso de diagramas co mo ferramentas de pensamento. Em 2001, Anuário do Conselho Nacional de Professores de Matemática: O papel da representação na matemática escolar (pp. 77-89). Conselho Nacional de Professores de Matemática. Dowker, A. (2016). Fatores que influenciam a melhoria em numeramento, leitura e compreensão no contexto de uma intervenção em numeramento. Frontiers in Psychology, 7, 1929. Downey, RG, & Fellows, MR (1999). Complexidade parametrizada. Nova York: Springer. Dry, M., Preiss, K., & Wagemans, J. (2012). Agrupamento, aleatoriedade e regularidade: distribuições espaciais e desempenho humano no problema do caixeiro viajante e no problema da árvore geradora mínima. The Journal of Problem Solving, 4(1), 2. Edmonds, J. (1965). Caminhos, árvores e flores. Jornal Canadense de Matemática, 17(3), 449–467. Fabry, RE (2018). Betwixt e entre: A abordagem de processamento preditivo enculturado para a cognição. Síntese, 195, 1–36. Fabry, RE (2019). As dimensões cerebrais, corporais extracerebrais e socioculturais da cognição aritmética enculturada. Síntese. https://doi.org/10.1007/s1122 9-019-02238 -1. Fabry, RE & Pantsar, M. (2019). Um novo olhar sobre estratégias de pesquisa em ciência cognitiva computacional: O caso da resolução de problemas matemáticos enculturados. Síntese. https://doi.org/10 1122 9 1007 019 02276 9 . /s - - - . Fraenkel, AS e Lichtenstein, D. (1981). Calcular uma estratégia perfeita para $\text{ n }\times \text{ n }$ xadrez requer tempo exponencial em n. Journal of Combinatorial Theory, Série A, 31(2), 199–214. Frank, M. e Barner, D. (2012). Representando o número exato visualmente usando o ábaco mental. Jornal de Psicologia Experimental, 141(1), 134–149. Frixione, M. (2001). Competência tratável. Mentes e Máquinas, 11(3), 379–397. Fuson, KC (1992). Pesquisa sobre aprendizagem e ensino de adição e subtração de números inteiros. Em G. Leinhardt, R. T. Putnam, & R. A. Hattrup (Eds.), Análise de aritmética para ensino de matemática (pp. 53–187). Hillsdale, NJ, EUA: Lawrence Erlbaum Associates. Garey, MR e Johnson, DS (1979). Computadores e intratabilidade: Um guia para a teoria da completude NP. São Francisco: W.H. Homem Livre. Goldreich, O. (2008). Complexidade computacional: uma perspectiva conceitual. ACM Sigact News, 39(3), 35–39. Graham, SM, Joshi, A. e Pizlo, Z. (2000). O problema do caixeiro viajante: um modelo hierárquico. Memória e Cognição, 28(7), 1191–1204. Hadamard, J. (1954). A psicologia da invenção no campo matemático. Mineola, NY: Publicações Dover. Hinrichs, JV, Yurko, DS e Hu, JM (1981). Comparação de números de dois dígitos: Uso de informações de local. Journal of Experimental Psychology: Percepção e Desempenho Humano, 7(4), 890. Horgan, T., & Tienson, J. (1996). Conexionismo e a filosofia da psicologia. Cambridge, MA: MIT Press. Ifrah, G. (1998). A história universal dos números: da pré-história à invenção do computador, traduzido por Bellos, D., Harding, E. F., Wood, S., & Monk, I. London: Harville Press. Isaac, AM, Szymanik, J. e Verbrugge, R. (2014). Lógica e complexidade nas ciências cognitivas. Em S. Smets & A. Baltag (Eds.), Johan van Benthem sobre lógica e dinâmica da informação (pp. 787–824). Berlim: Springer. Iwata, S. e Kasai, T. (1994). O jogo Othello em um tabuleiro $\text{ n }\times \text{ n }$ está completo no PSPACE. Ciência da Computação Teórica, 123(2), 329–340. Jamnik, M. (2001). Raciocínio matemático com diagramas. Chicago: Universidade de Chicago Press. Kadosh, RC e Henik, A. (2006). Efeito de congruência de cores: onde as cores e os números interagem na sinestesia? Córtex, 42(2), 259–263. Karatsuba, A. e Ofman, Y. (1962). Multiplicação de números multidigitais por computadores automáticos. Física-Doklady, 7(1963), 595–596. LeFevre, J., DeStefano, D., Coleman, B. e Shanahan, T. (2005). Cognição matemática e memória de trabalho. Em J. Campbell (Ed.), Manual de cognição matemática (pp. 361–378). Nova York: Psychology Press.

Livingstone, M. S., Pettine, W. W., Srihasam, K., Moore, B., Morocz, I. A., & Lee, D. (2014). Symbol addition by monkeys provides evidence for normalized quantity coding. *Proceedings of the National Academy of Sciences*, *111*(18), 6822–6827.
Markman, A. B., & Gentner, D. (2000). Structure mapping in the comparison process. *The American Journal of Psychology*, *113*(4), 501.
Marr, D. (1977). Artificial intelligence—A personal view. *Artificial Intelligence*, *9*(1), 37–48.
Marr, D. (1982). *Vision: A computational investigation into the human representation and processing of visual information*. San Francisco: W.H. Freeman and Company.
Menary, R. (2014). Neural plasticity, neuronal recycling and niche construction. *Mind & Language*, *29*(3), 286–303.
Menary, R. (2015). Mathematical cognition: A case of enculturation. In T. Metzinger & J. M. Windt (Eds.), *Open MIND* (pp. 1–20). Frankfurt am Main: MIND Group. https://doi.org/10.15502/9783958570818.
Newell, A. (1980). Physical symbol systems. *Cognitive Science*, *4*(2), 135–183.
Newell, A. (1982). The knowledge level. *Artificial Intelligence*, *18*(1), 87–127.
Newell, A., & Simon, H. A. (1976). Computer science as empirical inquiry (The 1975 ACM Turing Award Lecture). *Communications of the ACM*, *19*(3), 113–126.
Ormerod, T. C., & Chronicle, E. P. (1999). Global perceptual processing in problem solving: The case of the traveling salesperson. *Perception & Psychophysics*, *61*(6), 1227–1238.
Pantsar, M. (2018). Early numerical cognition and mathematical processes. *Theoria*, *33*(2), 285–304.
Pantsar, M. (2019). The Enculturated Dove From Proto-Arithmetic to Arithmetic. *Frontiers in Psychology*, *10*, 1454. https://doi.org/10.3389/fpsyg.2019.01454.
Papadimitriou, C. (1994). *Computational complexity*. Reading: Addison-Wesley.
Peacocke, C. (1986). Explanation in computational psychology: Language, perception and level 1.5. *Mind and Language*, *1*(2), 1010–1023.
Penrose, R. (1989). *The Emperor's new mind: Concerning computers, minds and the laws of physics*. Oxford: University Press.
Peters, E., & Bjalkebring, P. (2015). Multiple numeric competencies: When a number is not just a number. *Journal of Personality and Social Psychology*, *108*(5), 802.
Piantadosi, S. T., Tenenbaum, J. B., & Goodman, N. D. (2016). The logical primitives of thought: Empirical foundations for compositional cognitive models. *Psychological Review*, *123*(4), 392.
Pinel, P., Dehaene, S., Riviere, D., & LeBihan, D. (2001). Modulation of parietal activation by semantic distance in a number comparison task. *Neuroimage*, *14*(5), 1013–1026.
Polya, G. (1945). *How to solve it*. Princeton: Princeton University Press.
Pomerance, C. (1996). A tale of two sieves. *Notices of the American Mathematical Society*, *43*(12), 1473–1485.
Pylyshyn, Z. W. (1984). *Computation and cognition: Towards a foundation of cognitive science*. Cambridge, MA: MIT Press.
Schoenfeld, A. H. (1985). *Mathematical problem solving*. London: Academic Press.
Schönhage, A., & Strassen, V. (1971). Schnelle Multiplikation großer Zahlen. *Computing*, *7*, 281–292.
Skiena, S. (2008). *The algorithm design manual*. Berlin: Springer.
Soler-Toscano, F., Zenil, H., Delahaye, J. P., & Gauvrit, N. (2014). Calculating Kolmogorov complexity from the output frequency distributions of small turing machines. *PLoS ONE*, *9*(5), e96223.
Sun, R. (Ed.). (2008). Introduction to computational cognitive modeling. In *The Cambridge Handbook of Computational Psychology* (pp. 3–19). Cambridge: University Press.
Szymanik, J. (2016). *Quantifiers and cognition: Logical and computational perspectives, studies in linguistics and philosophy* (Vol. 96). Berlin: Springer.
Tall, D. (Ed.). (2002). *Advanced mathematical thinking. Mathematics education library* (Vol. 11). Dordrecht: Springer.
Tang, Y., Zhang, W., Chen, K., Feng, S., Ji, Y., Shen, J., et al. (2006). Arithmetic processing in the brain shaped by cultures. *Proceedings of the National Academy of Sciences*, *103*(28), 10775–10780.
Thagard, P. (2000). *Coherence in thought and action*. Cambridge, MA: Cambridge University Press.
Tsotsos, J. K. (1990). Analyzing vision at the complexity level. *Behavioral and Brain Sciences*, *13*, 423–469.
Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, *42*, 230–265.
Tzelgov, J., & Ganor-Stern, D. (2005). Automaticity in processing ordinal information. In J. Campbell (Ed.), *Handbook of Mathematical Cognition* (pp. 55–66). New York: Psychology Press.

Livingstone, MS, Pettine, WW, Srihasam, K., Moore, B., Morocz, IA, & Lee, D. (2014). A adição de símbolos por macacos fornece evidências de codificação de quantidade normalizada. Anais da Academia Nacional de Ciências, 111(18), 6822–6827. Markman, AB e Gentner, D. (2000). Mapeamento de estrutura no processo de comparação. The American Journal of Psychology, 113(4), 501. Marr, D. (1977). Inteligência artificial – uma visão pessoal. Inteligência Artificial, 9(1), 37–48. Marr, D. (1982). Visão: Uma investigação computacional sobre a representação humana e o processamento de informações visuais. São Francisco: W.H. Freeman e companhia. Menary, R. (2014). Plasticidade neural, reciclagem neuronal e construção de nichos. Mente e Linguagem, 29(3), 286–303. Menary, R. (2015). Cognição matemática: um caso de enculturação. Em T. Metzinger & JM Windt (Eds.), Open MIND (pp. 1 97839 58570 818 20 10 15502 –). Frankfurt am Main: Grupo MIND. https://doi.org/ . / . Newell, A. (1980). Sistemas de símbolos físicos. Ciência Cognitiva, 4(2), 135–183. Newell, A. (1982). O nível de conhecimento. Inteligência Artificial, 18(1), 87–127. Newell, A. e Simon, HA (1976). Ciência da computação como investigação empírica (The 1975 ACM Turing Award Lecture). Comunicações do ACM, 19(3), 113–126. Ormerod, TC e Chronicle, EP (1999). Processamento perceptivo global na resolução de problemas: o caso do caixeiro-viajante. Percepção e Psicofísica, 61(6), 1227–1238. Pantsar, M. (2018). Cognição numérica inicial e processos matemáticos. Teoria, 33(2), 285–304. Pantsar, M. (2019). A pomba enculturada da proto-aritmética à aritmética. Frontiers in Psychology, 10, 1454. https://doi.org/10.3389/fpsyg .2019.01454 . Papadimitriou, C. (1994). Complexidade computacional. Leitura: Addison-Wesley. Peacocke, C. (1986). Explicação em psicologia computacional: Linguagem, percepção e nível 1.5. Mente e Linguagem, 1(2), 1010–1023. Penrose, R. (1989). A nova mente do Imperador: Sobre computadores, mentes e as leis da física. Oxford: Imprensa Universitária. Peters, E. e Bjalkebring, P. (2015). Múltiplas competências numéricas: Quando um número não é apenas um número. Jornal de Personalidade e Psicologia Social, 108(5), 802. Piantadosi, ST, Tenenbaum, JB, & Goodman, ND (2016). Os primitivos lógicos do pensamento: fundamentos empíricos para modelos cognitivos composicionais. Revisão Psicológica, 123(4), 392. Pinel, P., Dehaene, S., Riviere, D., & LeBihan, D. (2001). Modulação da ativação parietal por distância semântica em tarefa de comparação numérica. Neuroimagem, 14(5), 1013–1026. Polya, G. (1945). Como resolver isso. Princeton: Princeton University Press. Pomerance, C. (1996). Uma história de duas peneiras. Avisos da American Mathematical Society, 43(12), 1473–1485. Pylyshyn, ZW (1984). Computação e cognição: Rumo a uma base da ciência cognitiva. Cambridge, MA: MIT Press. Schoenfeld, AH (1985). Resolução de problemas matemáticos. Londres: Academic Press. Schönhage, A. e Strassen, V. (1971). Schnelle Multiplicação maior Zahlen. Computação, 7, 281–292. Skiena, S. (2008). O manual de design de algoritmo. Berlim: Springer. Soler-Toscano, F., Zenil, H., Delahaye, JP e Gauvrit, N. (2014). Cálculo da complexidade de Kolmogorov a partir das distribuições de frequência de saída de pequenas máquinas de turing. PLoS UM, 9(5), e96223. Sol, R. (Ed.). (2008). Introdução à modelagem cognitiva computacional. No Manual de Psicologia Computacional de Cambridge (pp. 3–19). Cambridge: Imprensa Universitária. Szymanik, J. (2016). Quantificadores e cognição: Perspectivas lógicas e computacionais, estudos em linguística e filosofia (Vol. 96). Berlim: Springer. Alto, D. (Ed.). (2002). Pensamento matemático avançado. Biblioteca de educação matemática (Vol. 11). Dordrecht: Springer. Tang, Y., Zhang, W., Chen, K., Feng, S., Ji, Y., Shen, J., et al. (2006). Processamento aritmético no cérebro moldado por culturas. Anais da Academia Nacional de Ciências, 103(28), 10775–10780. Thagard, P. (2000). Coerência no pensamento e na ação. Cambridge, MA: Cambridge University Press. Tsotsos, J.K. (1990 13, 423 469). Analisando a visão no nível de complexidade. Ciências Comportamentais e do Cérebro, -. Turing, AM (1936). Em números computáveis, com aplicação ao Entscheidungsproblem. Proceedings of the London Mathematical Society, 42, 230–265. Tzelgov, J. e Ganor-Stern, D. (2005). Automaticidade no processamento de informações ordinais. Em J. Campbell (Ed.), Manual de Cognição Matemática (pp. 55–66). Nova York: Psychology Press.

Uesaka, Y., Manalo, E., & Ichikawa, S. I. (2007). What kinds of perceptions and daily learning behaviors promote students' use of diagrams in mathematics problem solving? *Learning and Instruction*, *17*(3), 322–335.

van Garderen, D., Scheuermann, A., & Jackson, C. (2013). Examining how students with diverse abilities use diagrams to solve mathematics word problems. *Learning Disability Quarterly*, *36*(3), 145–160.

van Hiel, A., & Mervielde, I. (2003). The measurement of cognitive complexity and its relationship with political extremism. *Political Psychology*, *24*(4), 781–801.

van Rooij, I. (2008). The tractable cognition thesis. *Cognitive Science*, *32*, 939–984.

van Rooij, I., Wright, C. D., & Wareham, T. (2012). Intractability and the use of heuristics in psychological explanations. *Synthese*, *187*(2), 471–487.

Varma, S. (2014). The subjective meaning of cognitive architecture: A marrian analysis. *Frontiers in Psychology*, *5*, 440.

Vollmer, H. (1999). *Introduction to circuit complexity. A uniform approach*., Texts in theoretical computer science Berlin: Springer.

Zachariades, T., Jones, K., Giannakoulias, E., Biza, I., Diacoumopoulus, D., & Souyoul, A. (2007). *Teaching calculus using dynamic geometric tools*. Southampton: University of Southampton.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Uesaka, Y., Manalo, E., & Ichikawa, SI (2007). Que tipos de percepções e comportamentos de aprendizagem diários promovem o uso de diagramas pelos alunos na resolução de problemas matemáticos? Aprendizagem e Instrução, 17(3), 322–335. van Garderen, D., Scheuermann, A. e Jackson, C. (2013). Examinar como alunos com diversas habilidades usam diagramas para resolver problemas matemáticos com palavras. Dificuldade de aprendizagem trimestralmente, 36(3), 145–160. van Hiel, A. e Mervielde, I. (2003). A medição da complexidade cognitiva e sua relação com o extremismo político. Psicologia Política, 24(4), 781–801. van Rooij, I. (2008). A tese da cognição tratável. Ciência Cognitiva, 32, 939–984. van Rooij, I., Wright, CD e Wareham, T. (2012). Intratabilidade e uso de heurísticas em explicações psicológicas. Síntese, 187(2), 471–487. Varma, S. (2014). O significado subjetivo da arquitetura cognitiva: uma análise marriana. Frontiers in Psychology, 5, 440. Voll mer, H. (1999). Introdução à complexidade do circuito. Uma abordagem uniforme., Textos em ciência da computação teórica Berlim: Springer. Zachariades, T., Jones, K., Giannakoulias, E., Biza, I., Diacoumopoulus, D., & Souyoul, A. (2007). Ensinar cálculo usando ferramentas geométricas dinâmicas. Southampton: Universidade de Southampton.

PublNota de Isher A Springer Nature permanece neutra em relação às reivindicações jurisdicionais publicadas d
mapse afiliações institucionais.